
VIII Appendix: Mathematical Background

Introduction

When we analyze algorithms, we often need to draw upon a body of mathematical tools. Some of these tools are as simple as high-school algebra, but others may be new to you. In Part I, we saw how to manipulate asymptotic notations and solve recurrences. This appendix comprises a compendium of several other concepts and methods we use to analyze algorithms. As noted in the introduction to Part I, you may have seen much of the material in this appendix before having read this book (although the specific notational conventions we use might occasionally differ from those you have seen elsewhere). Hence, you should treat this appendix as reference material. As in the rest of this book, however, we have included exercises and problems, in order for you to improve your skills in these areas.

Appendix A offers methods for evaluating and bounding summations, which occur frequently in the analysis of algorithms. Many of the formulas here appear in any calculus text, but you will find it convenient to have these methods compiled in one place.

Appendix B contains basic definitions and notations for sets, relations, functions, graphs, and trees. It also gives some basic properties of these mathematical objects.

Appendix C begins with elementary principles of counting: permutations, combinations, and the like. The remainder contains definitions and properties of basic probability. Most of the algorithms in this book require no probability for their analysis, and thus you can easily omit the latter sections of the chapter on a first reading, even without skimming them. Later, when you encounter a probabilistic analysis that you want to understand better, you will find Appendix C well organized for reference purposes.

Appendix D defines matrices, their operations, and some of their basic properties. You have probably seen most of this material already if you have taken a course in linear algebra, but you might find it helpful to have one place to look for our notation and definitions.

A Summations

When an algorithm contains an iterative control construct such as a **while** or **for** loop, we can express its running time as the sum of the times spent on each execution of the body of the loop. For example, we found in Section 2.2 that the j th iteration of insertion sort took time proportional to j in the worst case. By adding up the time spent on each iteration, we obtained the summation (or series)

$$\sum_{j=2}^n j .$$

When we evaluated this summation, we attained a bound of $\Theta(n^2)$ on the worst-case running time of the algorithm. This example illustrates why you should know how to manipulate and bound summations.

Section A.1 lists several basic formulas involving summations. Section A.2 offers useful techniques for bounding summations. We present the formulas in Section A.1 without proof, though proofs for some of them appear in Section A.2 to illustrate the methods of that section. You can find most of the other proofs in any calculus text.

A.1 Summation formulas and properties

Given a sequence $a_1, a_2, \dots, a_n$ of numbers, where n is a nonnegative integer, we can write the finite sum $a_1 + a_2 + \dots + a_n$ as

$$\sum_{k=1}^n a_k .$$

If $n = 0$, the value of the summation is defined to be 0. The value of a finite series is always well defined, and we can add its terms in any order.

Given an infinite sequence $a_1, a_2, \dots$ of numbers, we can write the infinite sum $a_1 + a_2 + \dots$ as

$$\sum_{k=1}^{\infty} a_k ,$$

which we interpret to mean

$$\lim_{n \rightarrow \infty} \sum_{k=1}^n a_k .$$

If the limit does not exist, the series **diverges**; otherwise, it **converges**. The terms of a convergent series cannot always be added in any order. We can, however, rearrange the terms of an **absolutely convergent series**, that is, a series $\sum_{k=1}^{\infty} a_k$ for which the series $\sum_{k=1}^{\infty} |a_k|$ also converges.

Linearity

For any real number c and any finite sequences $a_1, a_2, \dots, a_n$ and $b_1, b_2, \dots, b_n$,

$$\sum_{k=1}^n (ca_k + b_k) = c \sum_{k=1}^n a_k + \sum_{k=1}^n b_k .$$

The linearity property also applies to infinite convergent series.

We can exploit the linearity property to manipulate summations incorporating asymptotic notation. For example,

$$\sum_{k=1}^n \Theta(f(k)) = \Theta \left(\sum_{k=1}^n f(k) \right) .$$

In this equation, the Θ -notation on the left-hand side applies to the variable k , but on the right-hand side, it applies to n . We can also apply such manipulations to infinite convergent series.

Arithmetic series

The summation

$$\sum_{k=1}^n k = 1 + 2 + \dots + n ,$$

is an **arithmetic series** and has the value

$$\sum_{k=1}^n k = \frac{1}{2}n(n+1) \tag{A.1}$$

$$= \Theta(n^2) . \tag{A.2}$$

Sums of squares and cubes

We have the following summations of squares and cubes:

$$\sum_{k=0}^n k^2 = \frac{n(n+1)(2n+1)}{6}, \quad (\text{A.3})$$

$$\sum_{k=0}^n k^3 = \frac{n^2(n+1)^2}{4}. \quad (\text{A.4})$$

Geometric series

For real $x \neq 1$, the summation

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \cdots + x^n$$

is a **geometric** or **exponential series** and has the value

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}. \quad (\text{A.5})$$

When the summation is infinite and $|x| < 1$, we have the infinite decreasing geometric series

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}. \quad (\text{A.6})$$

Harmonic series

For positive integers n , the n th **harmonic number** is

$$\begin{aligned} H_n &= 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n} \\ &= \sum_{k=1}^n \frac{1}{k} \\ &= \ln n + O(1). \end{aligned} \quad (\text{A.7})$$

(We shall prove a related bound in Section A.2.)

Integrating and differentiating series

By integrating or differentiating the formulas above, additional formulas arise. For example, by differentiating both sides of the infinite geometric series (A.6) and multiplying by x , we get

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2} \quad (\text{A.8})$$

for $|x| < 1$.

Telescoping series

For any sequence $a_0, a_1, \dots, a_n$,

$$\sum_{k=1}^n (a_k - a_{k-1}) = a_n - a_0, \quad (\text{A.9})$$

since each of the terms $a_1, a_2, \dots, a_{n-1}$ is added in exactly once and subtracted out exactly once. We say that the sum *telescopes*. Similarly,

$$\sum_{k=0}^{n-1} (a_k - a_{k+1}) = a_0 - a_n.$$

As an example of a telescoping sum, consider the series

$$\sum_{k=1}^{n-1} \frac{1}{k(k+1)}.$$

Since we can rewrite each term as

$$\frac{1}{k(k+1)} = \frac{1}{k} - \frac{1}{k+1},$$

we get

$$\begin{aligned} \sum_{k=1}^{n-1} \frac{1}{k(k+1)} &= \sum_{k=1}^{n-1} \left(\frac{1}{k} - \frac{1}{k+1} \right) \\ &= 1 - \frac{1}{n}. \end{aligned}$$

Products

We can write the finite product $a_1 a_2 \cdots a_n$ as

$$\prod_{k=1}^n a_k.$$

If $n = 0$, the value of the product is defined to be 1. We can convert a formula with a product to a formula with a summation by using the identity

$$\lg \left(\prod_{k=1}^n a_k \right) = \sum_{k=1}^n \lg a_k.$$

Exercises**A.1-1**

Find a simple formula for $\sum_{k=1}^n (2k - 1)$.

A.1-2 ★

Show that $\sum_{k=1}^n 1/(2k - 1) = \ln(\sqrt{n}) + O(1)$ by manipulating the harmonic series.

A.1-3

Show that $\sum_{k=0}^{\infty} k^2 x^k = x(1 + x)/(1 - x)^3$ for $0 < |x| < 1$.

A.1-4 ★

Show that $\sum_{k=0}^{\infty} (k - 1)/2^k = 0$.

A.1-5 ★

Evaluate the sum $\sum_{k=1}^{\infty} (2k + 1)x^{2k}$.

A.1-6

Prove that $\sum_{k=1}^n O(f_k(i)) = O(\sum_{k=1}^n f_k(i))$ by using the linearity property of summations.

A.1-7

Evaluate the product $\prod_{k=1}^n 2 \cdot 4^k$.

A.1-8 ★

Evaluate the product $\prod_{k=2}^n (1 - 1/k^2)$.

A.2 Bounding summations

We have many techniques at our disposal for bounding the summations that describe the running times of algorithms. Here are some of the most frequently used methods.

Mathematical induction

The most basic way to evaluate a series is to use mathematical induction. As an example, let us prove that the arithmetic series $\sum_{k=1}^n k$ evaluates to $\frac{1}{2}n(n + 1)$. We can easily verify this assertion for $n = 1$. We make the inductive assumption that

it holds for n , and we prove that it holds for $n + 1$. We have

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^n k + (n+1) \\ &= \frac{1}{2}n(n+1) + (n+1) \\ &= \frac{1}{2}(n+1)(n+2) .\end{aligned}$$

You don't always need to guess the exact value of a summation in order to use mathematical induction. Instead, you can use induction to prove a bound on a summation. As an example, let us prove that the geometric series $\sum_{k=0}^n 3^k$ is $O(3^n)$. More specifically, let us prove that $\sum_{k=0}^n 3^k \leq c 3^n$ for some constant c . For the initial condition $n = 0$, we have $\sum_{k=0}^0 3^k = 1 \leq c \cdot 1$ as long as $c \geq 1$. Assuming that the bound holds for n , let us prove that it holds for $n + 1$. We have

$$\begin{aligned}\sum_{k=0}^{n+1} 3^k &= \sum_{k=0}^n 3^k + 3^{n+1} \\ &\leq c 3^n + 3^{n+1} \quad (\text{by the inductive hypothesis}) \\ &= \left(\frac{1}{3} + \frac{1}{c}\right) c 3^{n+1} \\ &\leq c 3^{n+1}\end{aligned}$$

as long as $(1/3 + 1/c) \leq 1$ or, equivalently, $c \geq 3/2$. Thus, $\sum_{k=0}^n 3^k = O(3^n)$, as we wished to show.

We have to be careful when we use asymptotic notation to prove bounds by induction. Consider the following fallacious proof that $\sum_{k=1}^n k = O(n)$. Certainly, $\sum_{k=1}^1 k = O(1)$. Assuming that the bound holds for n , we now prove it for $n + 1$:

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^n k + (n+1) \\ &= O(n) + (n+1) \quad \Leftarrow \text{wrong!!} \\ &= O(n+1) .\end{aligned}$$

The bug in the argument is that the “constant” hidden by the “big-oh” grows with n and thus is not constant. We have not shown that the same constant works for *all* n .

Bounding the terms

We can sometimes obtain a good upper bound on a series by bounding each term of the series, and it often suffices to use the largest term to bound the others. For

example, a quick upper bound on the arithmetic series (A.1) is

$$\begin{aligned}\sum_{k=1}^n k &\leq \sum_{k=1}^n n \\ &= n^2 .\end{aligned}$$

In general, for a series $\sum_{k=1}^n a_k$, if we let $a_{\max} = \max_{1 \leq k \leq n} a_k$, then

$$\sum_{k=1}^n a_k \leq n \cdot a_{\max} .$$

The technique of bounding each term in a series by the largest term is a weak method when the series can in fact be bounded by a geometric series. Given the series $\sum_{k=0}^n a_k$, suppose that $a_{k+1}/a_k \leq r$ for all $k \geq 0$, where $0 < r < 1$ is a constant. We can bound the sum by an infinite decreasing geometric series, since $a_k \leq a_0 r^k$, and thus

$$\begin{aligned}\sum_{k=0}^n a_k &\leq \sum_{k=0}^{\infty} a_0 r^k \\ &= a_0 \sum_{k=0}^{\infty} r^k \\ &= a_0 \frac{1}{1-r} .\end{aligned}$$

We can apply this method to bound the summation $\sum_{k=1}^{\infty} (k/3^k)$. In order to start the summation at $k = 0$, we rewrite it as $\sum_{k=0}^{\infty} ((k+1)/3^{k+1})$. The first term (a_0) is $1/3$, and the ratio (r) of consecutive terms is

$$\begin{aligned}\frac{(k+2)/3^{k+2}}{(k+1)/3^{k+1}} &= \frac{1}{3} \cdot \frac{k+2}{k+1} \\ &\leq \frac{2}{3}\end{aligned}$$

for all $k \geq 0$. Thus, we have

$$\begin{aligned}\sum_{k=1}^{\infty} \frac{k}{3^k} &= \sum_{k=0}^{\infty} \frac{k+1}{3^{k+1}} \\ &\leq \frac{1}{3} \cdot \frac{1}{1-2/3} \\ &= 1 .\end{aligned}$$

A common bug in applying this method is to show that the ratio of consecutive terms is less than 1 and then to assume that the summation is bounded by a geometric series. An example is the infinite harmonic series, which diverges since

$$\begin{aligned}\sum_{k=1}^{\infty} \frac{1}{k} &= \lim_{n \rightarrow \infty} \sum_{k=1}^n \frac{1}{k} \\ &= \lim_{n \rightarrow \infty} \Theta(\lg n) \\ &= \infty.\end{aligned}$$

The ratio of the $(k+1)$ st and k th terms in this series is $k/(k+1) < 1$, but the series is not bounded by a decreasing geometric series. To bound a series by a geometric series, we must show that there is an $r < 1$, which is a *constant*, such that the ratio of all pairs of consecutive terms never exceeds r . In the harmonic series, no such r exists because the ratio becomes arbitrarily close to 1.

Splitting summations

One way to obtain bounds on a difficult summation is to express the series as the sum of two or more series by partitioning the range of the index and then to bound each of the resulting series. For example, suppose we try to find a lower bound on the arithmetic series $\sum_{k=1}^n k$, which we have already seen has an upper bound of n^2 . We might attempt to bound each term in the summation by the smallest term, but since that term is 1, we get a lower bound of n for the summation—far off from our upper bound of n^2 .

We can obtain a better lower bound by first splitting the summation. Assume for convenience that n is even. We have

$$\begin{aligned}\sum_{k=1}^n k &= \sum_{k=1}^{n/2} k + \sum_{k=n/2+1}^n k \\ &\geq \sum_{k=1}^{n/2} 0 + \sum_{k=n/2+1}^n (n/2) \\ &= (n/2)^2 \\ &= \Omega(n^2),\end{aligned}$$

which is an asymptotically tight bound, since $\sum_{k=1}^n k = O(n^2)$.

For a summation arising from the analysis of an algorithm, we can often split the summation and ignore a constant number of the initial terms. Generally, this technique applies when each term a_k in a summation $\sum_{k=0}^n a_k$ is independent of n .

Then for any constant $k_0 > 0$, we can write

$$\begin{aligned}\sum_{k=0}^n a_k &= \sum_{k=0}^{k_0-1} a_k + \sum_{k=k_0}^n a_k \\ &= \Theta(1) + \sum_{k=k_0}^n a_k ,\end{aligned}$$

since the initial terms of the summation are all constant and there are a constant number of them. We can then use other methods to bound $\sum_{k=k_0}^n a_k$. This technique applies to infinite summations as well. For example, to find an asymptotic upper bound on

$$\sum_{k=0}^{\infty} \frac{k^2}{2^k} ,$$

we observe that the ratio of consecutive terms is

$$\begin{aligned}\frac{(k+1)^2/2^{k+1}}{k^2/2^k} &= \frac{(k+1)^2}{2k^2} \\ &\leq \frac{8}{9}\end{aligned}$$

if $k \geq 3$. Thus, the summation can be split into

$$\begin{aligned}\sum_{k=0}^{\infty} \frac{k^2}{2^k} &= \sum_{k=0}^2 \frac{k^2}{2^k} + \sum_{k=3}^{\infty} \frac{k^2}{2^k} \\ &\leq \sum_{k=0}^2 \frac{k^2}{2^k} + \frac{9}{8} \sum_{k=0}^{\infty} \left(\frac{8}{9}\right)^k \\ &= O(1) ,\end{aligned}$$

since the first summation has a constant number of terms and the second summation is a decreasing geometric series.

The technique of splitting summations can help us determine asymptotic bounds in much more difficult situations. For example, we can obtain a bound of $O(\lg n)$ on the harmonic series (A.7):

$$H_n = \sum_{k=1}^n \frac{1}{k} .$$

We do so by splitting the range 1 to n into $\lfloor \lg n \rfloor + 1$ pieces and upper-bounding the contribution of each piece by 1. For $i = 0, 1, \dots, \lfloor \lg n \rfloor$, the i th piece consists

of the terms starting at $1/2^i$ and going up to but not including $1/2^{i+1}$. The last piece might contain terms not in the original harmonic series, and thus we have

$$\begin{aligned}
 \sum_{k=1}^n \frac{1}{k} &\leq \sum_{i=0}^{\lfloor \lg n \rfloor} \sum_{j=0}^{2^i-1} \frac{1}{2^i + j} \\
 &\leq \sum_{i=0}^{\lfloor \lg n \rfloor} \sum_{j=0}^{2^i-1} \frac{1}{2^i} \\
 &= \sum_{i=0}^{\lfloor \lg n \rfloor} 1 \\
 &\leq \lg n + 1 .
 \end{aligned} \tag{A.10}$$

Approximation by integrals

When a summation has the form $\sum_{k=m}^n f(k)$, where $f(k)$ is a monotonically increasing function, we can approximate it by integrals:

$$\int_{m-1}^n f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x) dx . \tag{A.11}$$

Figure A.1 justifies this approximation. The summation is represented as the area of the rectangles in the figure, and the integral is the shaded region under the curve. When $f(k)$ is a monotonically decreasing function, we can use a similar method to provide the bounds

$$\int_m^{n+1} f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_{m-1}^n f(x) dx . \tag{A.12}$$

The integral approximation (A.12) gives a tight estimate for the n th harmonic number. For a lower bound, we obtain

$$\begin{aligned}
 \sum_{k=1}^n \frac{1}{k} &\geq \int_1^{n+1} \frac{dx}{x} \\
 &= \ln(n+1) .
 \end{aligned} \tag{A.13}$$

For the upper bound, we derive the inequality

$$\begin{aligned}
 \sum_{k=2}^n \frac{1}{k} &\leq \int_1^n \frac{dx}{x} \\
 &= \ln n ,
 \end{aligned}$$

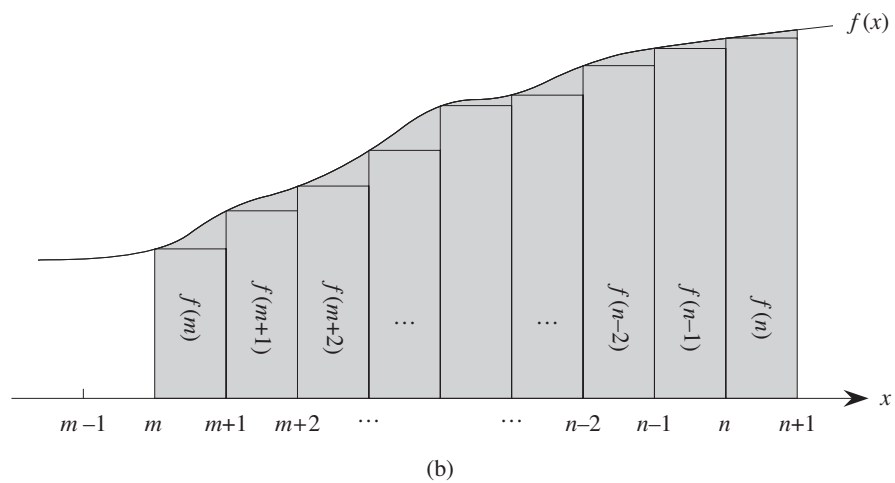
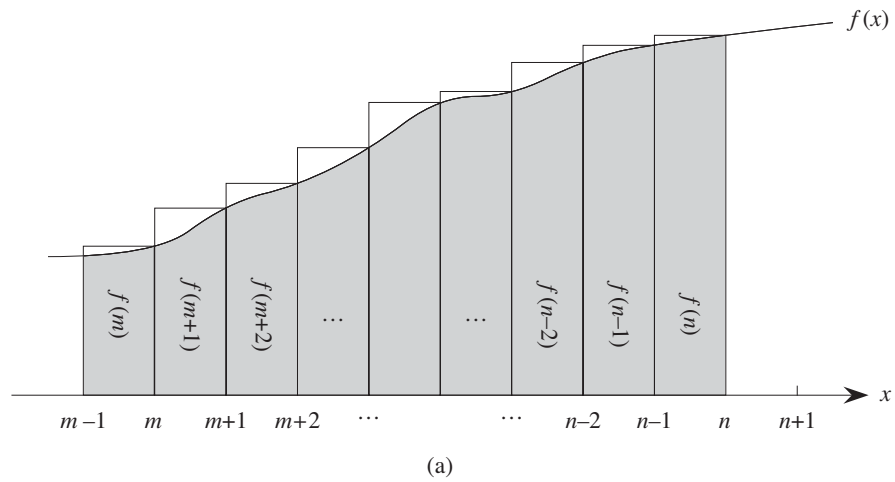


Figure A.1 Approximation of $\sum_{k=m}^n f(k)$ by integrals. The area of each rectangle is shown within the rectangle, and the total rectangle area represents the value of the summation. The integral is represented by the shaded area under the curve. By comparing areas in **(a)**, we get $\int_{m-1}^n f(x) dx \leq \sum_{k=m}^n f(k)$, and then by shifting the rectangles one unit to the right, we get $\sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x) dx$ in **(b)**.

which yields the bound

$$\sum_{k=1}^n \frac{1}{k} \leq \ln n + 1. \quad (\text{A.14})$$

Exercises

A.2-1

Show that $\sum_{k=1}^n 1/k^2$ is bounded above by a constant.

A.2-2

Find an asymptotic upper bound on the summation

$$\sum_{k=0}^{\lfloor \lg n \rfloor} \lceil n/2^k \rceil.$$

A.2-3

Show that the n th harmonic number is $\Omega(\lg n)$ by splitting the summation.

A.2-4

Approximate $\sum_{k=1}^n k^3$ with an integral.

A.2-5

Why didn't we use the integral approximation (A.12) directly on $\sum_{k=1}^n 1/k$ to obtain an upper bound on the n th harmonic number?

Problems

A-1 Bounding summations

Give asymptotically tight bounds on the following summations. Assume that $r \geq 0$ and $s \geq 0$ are constants.

a. $\sum_{k=1}^n k^r.$

b. $\sum_{k=1}^n \lg^s k.$

$$c. \sum_{k=1}^n k^r \lg^s k.$$

Appendix notes

Knuth [209] provides an excellent reference for the material presented here. You can find basic properties of series in any good calculus book, such as Apostol [18] or Thomas et al. [334].

B Sets, Etc.

Many chapters of this book touch on the elements of discrete mathematics. This appendix reviews more completely the notations, definitions, and elementary properties of sets, relations, functions, graphs, and trees. If you are already well versed in this material, you can probably just skim this chapter.

B.1 Sets

A *set* is a collection of distinguishable objects, called its *members* or *elements*. If an object x is a member of a set S , we write $x \in S$ (read “ x is a member of S ” or, more briefly, “ x is in S ”). If x is not a member of S , we write $x \notin S$. We can describe a set by explicitly listing its members as a list inside braces. For example, we can define a set S to contain precisely the numbers 1, 2, and 3 by writing $S = \{1, 2, 3\}$. Since 2 is a member of the set S , we can write $2 \in S$, and since 4 is not a member, we have $4 \notin S$. A set cannot contain the same object more than once,¹ and its elements are not ordered. Two sets A and B are *equal*, written $A = B$, if they contain the same elements. For example, $\{1, 2, 3, 1\} = \{1, 2, 3\} = \{3, 2, 1\}$.

We adopt special notations for frequently encountered sets:

- $\emptyset$ denotes the *empty set*, that is, the set containing no members.
- $\mathbb{Z}$ denotes the set of *integers*, that is, the set $\{\dots, -2, -1, 0, 1, 2, \dots\}$.
- $\mathbb{R}$ denotes the set of *real numbers*.
- $\mathbb{N}$ denotes the set of *natural numbers*, that is, the set $\{0, 1, 2, \dots\}$.²

¹A variation of a set, which can contain the same object more than once, is called a *multiset*.

²Some authors start the natural numbers with 1 instead of 0. The modern trend seems to be to start with 0.

If all the elements of a set A are contained in a set B , that is, if $x \in A$ implies $x \in B$, then we write $A \subseteq B$ and say that A is a **subset** of B . A set A is a **proper subset** of B , written $A \subset B$, if $A \subseteq B$ but $A \neq B$. (Some authors use the symbol “ $\subset$ ” to denote the ordinary subset relation, rather than the proper-subset relation.) For any set A , we have $A \subseteq A$. For two sets A and B , we have $A = B$ if and only if $A \subseteq B$ and $B \subseteq A$. For any three sets A , B , and C , if $A \subseteq B$ and $B \subseteq C$, then $A \subseteq C$. For any set A , we have $\emptyset \subseteq A$.

We sometimes define sets in terms of other sets. Given a set A , we can define a set $B \subseteq A$ by stating a property that distinguishes the elements of B . For example, we can define the set of even integers by $\{x : x \in \mathbb{Z} \text{ and } x/2 \text{ is an integer}\}$. The colon in this notation is read “such that.” (Some authors use a vertical bar in place of the colon.)

Given two sets A and B , we can also define new sets by applying **set operations**:

- The **intersection** of sets A and B is the set

$$A \cap B = \{x : x \in A \text{ and } x \in B\} .$$

- The **union** of sets A and B is the set

$$A \cup B = \{x : x \in A \text{ or } x \in B\} .$$

- The **difference** between two sets A and B is the set

$$A - B = \{x : x \in A \text{ and } x \notin B\} .$$

Set operations obey the following laws:

Empty set laws:

$$\begin{aligned} A \cap \emptyset &= \emptyset , \\ A \cup \emptyset &= A . \end{aligned}$$

Idempotency laws:

$$\begin{aligned} A \cap A &= A , \\ A \cup A &= A . \end{aligned}$$

Commutative laws:

$$\begin{aligned} A \cap B &= B \cap A , \\ A \cup B &= B \cup A . \end{aligned}$$

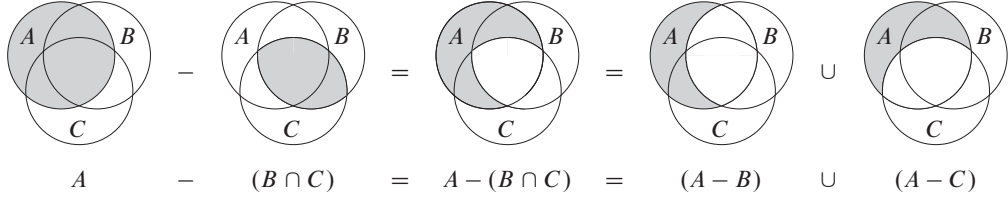


Figure B.1 A Venn diagram illustrating the first of DeMorgan's laws (B.2). Each of the sets A , B , and C is represented as a circle.

Associative laws:

$$\begin{aligned} A \cap (B \cap C) &= (A \cap B) \cap C, \\ A \cup (B \cup C) &= (A \cup B) \cup C. \end{aligned}$$

Distributive laws:

$$\begin{aligned} A \cap (B \cup C) &= (A \cap B) \cup (A \cap C), \\ A \cup (B \cap C) &= (A \cup B) \cap (A \cup C). \end{aligned} \tag{B.1}$$

Absorption laws:

$$\begin{aligned} A \cap (A \cup B) &= A, \\ A \cup (A \cap B) &= A. \end{aligned}$$

DeMorgan's laws:

$$\begin{aligned} A - (B \cap C) &= (A - B) \cup (A - C), \\ A - (B \cup C) &= (A - B) \cap (A - C). \end{aligned} \tag{B.2}$$

Figure B.1 illustrates the first of DeMorgan's laws, using a **Venn diagram**: a graphical picture in which sets are represented as regions of the plane.

Often, all the sets under consideration are subsets of some larger set U called the **universe**. For example, if we are considering various sets made up only of integers, the set $\mathbb{Z}$ of integers is an appropriate universe. Given a universe U , we define the **complement** of a set A as $\bar{A} = U - A = \{x : x \in U \text{ and } x \notin A\}$. For any set $A \subseteq U$, we have the following laws:

$$\begin{aligned} \overline{\bar{A}} &= A, \\ A \cap \bar{A} &= \emptyset, \\ A \cup \bar{A} &= U. \end{aligned}$$

We can rewrite DeMorgan's laws (B.2) with set complements. For any two sets $B, C \subseteq U$, we have

$$\begin{aligned}\overline{B \cap C} &= \overline{B} \cup \overline{C}, \\ \overline{B \cup C} &= \overline{B} \cap \overline{C}.\end{aligned}$$

Two sets A and B are **disjoint** if they have no elements in common, that is, if $A \cap B = \emptyset$. A collection $\mathcal{S} = \{S_i\}$ of nonempty sets forms a **partition** of a set S if

- the sets are **pairwise disjoint**, that is, $S_i, S_j \in \mathcal{S}$ and $i \neq j$ imply $S_i \cap S_j = \emptyset$, and
- their union is S , that is,

$$S = \bigcup_{S_i \in \mathcal{S}} S_i.$$

In other words, $\mathcal{S}$ forms a partition of S if each element of S appears in exactly one $S_i \in \mathcal{S}$.

The number of elements in a set is the **cardinality** (or **size**) of the set, denoted $|S|$. Two sets have the same cardinality if their elements can be put into a one-to-one correspondence. The cardinality of the empty set is $|\emptyset| = 0$. If the cardinality of a set is a natural number, we say the set is **finite**; otherwise, it is **infinite**. An infinite set that can be put into a one-to-one correspondence with the natural numbers $\mathbb{N}$ is **countably infinite**; otherwise, it is **uncountable**. For example, the integers $\mathbb{Z}$ are countable, but the reals $\mathbb{R}$ are uncountable.

For any two finite sets A and B , we have the identity

$$|A \cup B| = |A| + |B| - |A \cap B|, \quad (\text{B.3})$$

from which we can conclude that

$$|A \cup B| \leq |A| + |B|.$$

If A and B are disjoint, then $|A \cap B| = 0$ and thus $|A \cup B| = |A| + |B|$. If $A \subseteq B$, then $|A| \leq |B|$.

A finite set of n elements is sometimes called an **n -set**. A 1-set is called a **singleton**. A subset of k elements of a set is sometimes called a **k -subset**.

We denote the set of all subsets of a set S , including the empty set and S itself, by 2^S ; we call 2^S the **power set** of S . For example, $2^{\{a,b\}} = \{\emptyset, \{a\}, \{b\}, \{a,b\}\}$. The power set of a finite set S has cardinality $2^{|S|}$ (see Exercise B.1-5).

We sometimes care about setlike structures in which the elements are ordered. An **ordered pair** of two elements a and b is denoted (a, b) and is defined formally as the set $(a, b) = \{a, \{a, b\}\}$. Thus, the ordered pair (a, b) is *not* the same as the ordered pair (b, a) .

The **Cartesian product** of two sets A and B , denoted $A \times B$, is the set of all ordered pairs such that the first element of the pair is an element of A and the second is an element of B . More formally,

$$A \times B = \{(a, b) : a \in A \text{ and } b \in B\} .$$

For example, $\{a, b\} \times \{a, b, c\} = \{(a, a), (a, b), (a, c), (b, a), (b, b), (b, c)\}$. When A and B are finite sets, the cardinality of their Cartesian product is

$$|A \times B| = |A| \cdot |B| . \quad (\text{B.4})$$

The Cartesian product of n sets $A_1, A_2, \dots, A_n$ is the set of ***n*-tuples**

$$A_1 \times A_2 \times \cdots \times A_n = \{(a_1, a_2, \dots, a_n) : a_i \in A_i \text{ for } i = 1, 2, \dots, n\} ,$$

whose cardinality is

$$|A_1 \times A_2 \times \cdots \times A_n| = |A_1| \cdot |A_2| \cdots |A_n|$$

if all sets are finite. We denote an n -fold Cartesian product over a single set A by the set

$$A^n = A \times A \times \cdots \times A ,$$

whose cardinality is $|A^n| = |A|^n$ if A is finite. We can also view an n -tuple as a finite sequence of length n (see page 1166).

Exercises

B.1-1

Draw Venn diagrams that illustrate the first of the distributive laws (B.1).

B.1-2

Prove the generalization of DeMorgan's laws to any finite collection of sets:

$$\begin{aligned} \overline{A_1 \cap A_2 \cap \cdots \cap A_n} &= \overline{A_1} \cup \overline{A_2} \cup \cdots \cup \overline{A_n} , \\ \overline{A_1 \cup A_2 \cup \cdots \cup A_n} &= \overline{A_1} \cap \overline{A_2} \cap \cdots \cap \overline{A_n} . \end{aligned}$$

B.1-3 ★

Prove the generalization of equation (B.3), which is called the *principle of inclusion and exclusion*:

$$\begin{aligned}
 |A_1 \cup A_2 \cup \cdots \cup A_n| = & \\
 & |A_1| + |A_2| + \cdots + |A_n| \\
 & - |A_1 \cap A_2| - |A_1 \cap A_3| - \cdots \quad (\text{all pairs}) \\
 & + |A_1 \cap A_2 \cap A_3| + \cdots \quad (\text{all triples}) \\
 & \vdots \\
 & + (-1)^{n-1} |A_1 \cap A_2 \cap \cdots \cap A_n| .
 \end{aligned}$$

B.1-4

Show that the set of odd natural numbers is countable.

B.1-5

Show that for any finite set S , the power set 2^S has $2^{|S|}$ elements (that is, there are $2^{|S|}$ distinct subsets of S).

B.1-6

Give an inductive definition for an n -tuple by extending the set-theoretic definition for an ordered pair.

B.2 Relations

A **binary relation** R on two sets A and B is a subset of the Cartesian product $A \times B$. If $(a, b) \in R$, we sometimes write $a R b$. When we say that R is a binary relation on a set A , we mean that R is a subset of $A \times A$. For example, the “less than” relation on the natural numbers is the set $\{(a, b) : a, b \in \mathbb{N} \text{ and } a < b\}$. An n -ary relation on sets $A_1, A_2, \dots, A_n$ is a subset of $A_1 \times A_2 \times \cdots \times A_n$.

A binary relation $R \subseteq A \times A$ is **reflexive** if

$$a R a$$

for all $a \in A$. For example, “=” and “ $\leq$ ” are reflexive relations on $\mathbb{N}$, but “<” is not. The relation R is **symmetric** if

$$a R b \text{ implies } b R a$$

for all $a, b \in A$. For example, “=” is symmetric, but “<” and “ $\leq$ ” are not. The relation R is **transitive** if

$$a R b \text{ and } b R c \text{ imply } a R c$$

for all $a, b, c \in A$. For example, the relations “ $<$,” “ $\leq$,” and “ $=$ ” are transitive, but the relation $R = \{(a, b) : a, b \in \mathbb{N} \text{ and } a = b - 1\}$ is not, since $3 R 4$ and $4 R 5$ do not imply $3 R 5$.

A relation that is reflexive, symmetric, and transitive is an **equivalence relation**. For example, “ $=$ ” is an equivalence relation on the natural numbers, but “ $<$ ” is not. If R is an equivalence relation on a set A , then for $a \in A$, the **equivalence class** of a is the set $[a] = \{b \in A : a R b\}$, that is, the set of all elements equivalent to a . For example, if we define $R = \{(a, b) : a, b \in \mathbb{N} \text{ and } a + b \text{ is an even number}\}$, then R is an equivalence relation, since $a + a$ is even (reflexive), $a + b$ is even implies $b + a$ is even (symmetric), and $a + b$ is even and $b + c$ is even imply $a + c$ is even (transitive). The equivalence class of 4 is $[4] = \{0, 2, 4, 6, \dots\}$, and the equivalence class of 3 is $[3] = \{1, 3, 5, 7, \dots\}$. A basic theorem of equivalence classes is the following.

Theorem B.1 (An equivalence relation is the same as a partition)

The equivalence classes of any equivalence relation R on a set A form a partition of A , and any partition of A determines an equivalence relation on A for which the sets in the partition are the equivalence classes.

Proof For the first part of the proof, we must show that the equivalence classes of R are nonempty, pairwise-disjoint sets whose union is A . Because R is reflexive, $a \in [a]$, and so the equivalence classes are nonempty; moreover, since every element $a \in A$ belongs to the equivalence class $[a]$, the union of the equivalence classes is A . It remains to show that the equivalence classes are pairwise disjoint, that is, if two equivalence classes $[a]$ and $[b]$ have an element c in common, then they are in fact the same set. Suppose that $a R c$ and $b R c$. By symmetry, $c R b$, and by transitivity, $a R b$. Thus, for any arbitrary element $x \in [a]$, we have $x R a$ and, by transitivity, $x R b$, and thus $[a] \subseteq [b]$. Similarly, $[b] \subseteq [a]$, and thus $[a] = [b]$.

For the second part of the proof, let $\mathcal{A} = \{A_i\}$ be a partition of A , and define $R = \{(a, b) : \text{there exists } i \text{ such that } a \in A_i \text{ and } b \in A_i\}$. We claim that R is an equivalence relation on A . Reflexivity holds, since $a \in A_i$ implies $a R a$. Symmetry holds, because if $a R b$, then a and b are in the same set A_i , and hence $b R a$. If $a R b$ and $b R c$, then all three elements are in the same set A_i , and thus $a R c$ and transitivity holds. To see that the sets in the partition are the equivalence classes of R , observe that if $a \in A_i$, then $x \in [a]$ implies $x \in A_i$, and $x \in A_i$ implies $x \in [a]$. ■

A binary relation R on a set A is **antisymmetric** if
 $a R b$ and $b R a$ imply $a = b$.

For example, the “ $\leq$ ” relation on the natural numbers is antisymmetric, since $a \leq b$ and $b \leq a$ imply $a = b$. A relation that is reflexive, antisymmetric, and transitive is a **partial order**, and we call a set on which a partial order is defined a **partially ordered set**. For example, the relation “is a descendant of” is a partial order on the set of all people (if we view individuals as being their own descendants).

In a partially ordered set A , there may be no single “maximum” element a such that $b R a$ for all $b \in A$. Instead, the set may contain several **maximal** elements a such that for no $b \in A$, where $b \neq a$, is it the case that $a R b$. For example, a collection of different-sized boxes may contain several maximal boxes that don’t fit inside any other box, yet it has no single “maximum” box into which any other box will fit.³

A relation R on a set A is a **total relation** if for all $a, b \in A$, we have $a R b$ or $b R a$ (or both), that is, if every pairing of elements of A is related by R . A partial order that is also a total relation is a **total order** or **linear order**. For example, the relation “ $\leq$ ” is a total order on the natural numbers, but the “is a descendant of” relation is not a total order on the set of all people, since there are individuals neither of whom is descended from the other. A total relation that is transitive, but not necessarily reflexive and antisymmetric, is a **total preorder**.

Exercises

B.2-1

Prove that the subset relation “ $\subseteq$ ” on all subsets of $\mathbb{Z}$ is a partial order but not a total order.

B.2-2

Show that for any positive integer n , the relation “equivalent modulo n ” is an equivalence relation on the integers. (We say that $a \equiv b \pmod{n}$ if there exists an integer q such that $a - b = qn$.) Into what equivalence classes does this relation partition the integers?

B.2-3

Give examples of relations that are

- a.* reflexive and symmetric but not transitive,
- b.* reflexive and transitive but not symmetric,
- c.* symmetric and transitive but not reflexive.

³To be precise, in order for the “fit inside” relation to be a partial order, we need to view a box as fitting inside itself.

B.2-4

Let S be a finite set, and let R be an equivalence relation on $S \times S$. Show that if in addition R is antisymmetric, then the equivalence classes of S with respect to R are singletons.

B.2-5

Professor Narcissus claims that if a relation R is symmetric and transitive, then it is also reflexive. He offers the following proof. By symmetry, $a R b$ implies $b R a$. Transitivity, therefore, implies $a R a$. Is the professor correct?

B.3 Functions

Given two sets A and B , a **function** f is a binary relation on A and B such that for all $a \in A$, there exists precisely one $b \in B$ such that $(a, b) \in f$. The set A is called the **domain** of f , and the set B is called the **codomain** of f . We sometimes write $f : A \rightarrow B$; and if $(a, b) \in f$, we write $b = f(a)$, since b is uniquely determined by the choice of a .

Intuitively, the function f assigns an element of B to each element of A . No element of A is assigned two different elements of B , but the same element of B can be assigned to two different elements of A . For example, the binary relation

$$f = \{(a, b) : a, b \in \mathbb{N} \text{ and } b = a \bmod 2\}$$

is a function $f : \mathbb{N} \rightarrow \{0, 1\}$, since for each natural number a , there is exactly one value b in $\{0, 1\}$ such that $b = a \bmod 2$. For this example, $0 = f(0)$, $1 = f(1)$, $0 = f(2)$, etc. In contrast, the binary relation

$$g = \{(a, b) : a, b \in \mathbb{N} \text{ and } a + b \text{ is even}\}$$

is not a function, since $(1, 3)$ and $(1, 5)$ are both in g , and thus for the choice $a = 1$, there is not precisely one b such that $(a, b) \in g$.

Given a function $f : A \rightarrow B$, if $b = f(a)$, we say that a is the **argument** of f and that b is the **value** of f at a . We can define a function by stating its value for every element of its domain. For example, we might define $f(n) = 2n$ for $n \in \mathbb{N}$, which means $f = \{(n, 2n) : n \in \mathbb{N}\}$. Two functions f and g are **equal** if they have the same domain and codomain and if, for all a in the domain, $f(a) = g(a)$.

A **finite sequence** of length n is a function f whose domain is the set of n integers $\{0, 1, \dots, n-1\}$. We often denote a finite sequence by listing its values: $\langle f(0), f(1), \dots, f(n-1) \rangle$. An **infinite sequence** is a function whose domain is the set $\mathbb{N}$ of natural numbers. For example, the Fibonacci sequence, defined by recurrence (3.22), is the infinite sequence $\langle 0, 1, 1, 2, 3, 5, 8, 13, 21, \dots \rangle$.

When the domain of a function f is a Cartesian product, we often omit the extra parentheses surrounding the argument of f . For example, if we had a function $f : A_1 \times A_2 \times \cdots \times A_n \rightarrow B$, we would write $b = f(a_1, a_2, \dots, a_n)$ instead of $b = f((a_1, a_2, \dots, a_n))$. We also call each a_i an **argument** to the function f , though technically the (single) argument to f is the n -tuple $(a_1, a_2, \dots, a_n)$.

If $f : A \rightarrow B$ is a function and $b = f(a)$, then we sometimes say that b is the **image** of a under f . The image of a set $A' \subseteq A$ under f is defined by

$$f(A') = \{b \in B : b = f(a) \text{ for some } a \in A'\}.$$

The **range** of f is the image of its domain, that is, $f(A)$. For example, the range of the function $f : \mathbb{N} \rightarrow \mathbb{N}$ defined by $f(n) = 2n$ is $f(\mathbb{N}) = \{m : m = 2n \text{ for some } n \in \mathbb{N}\}$, in other words, the set of nonnegative even integers.

A function is a **surjection** if its range is its codomain. For example, the function $f(n) = \lfloor n/2 \rfloor$ is a surjective function from $\mathbb{N}$ to $\mathbb{N}$, since every element in $\mathbb{N}$ appears as the value of f for some argument. In contrast, the function $f(n) = 2n$ is not a surjective function from $\mathbb{N}$ to $\mathbb{N}$, since no argument to f can produce 3 as a value. The function $f(n) = 2n$ is, however, a surjective function from the natural numbers to the even numbers. A surjection $f : A \rightarrow B$ is sometimes described as mapping A **onto** B . When we say that f is onto, we mean that it is surjective.

A function $f : A \rightarrow B$ is an **injection** if distinct arguments to f produce distinct values, that is, if $a \neq a'$ implies $f(a) \neq f(a')$. For example, the function $f(n) = 2n$ is an injective function from $\mathbb{N}$ to $\mathbb{N}$, since each even number b is the image under f of at most one element of the domain, namely $b/2$. The function $f(n) = \lfloor n/2 \rfloor$ is not injective, since the value 1 is produced by two arguments: 2 and 3. An injection is sometimes called a **one-to-one** function.

A function $f : A \rightarrow B$ is a **bijection** if it is injective and surjective. For example, the function $f(n) = (-1)^n \lfloor n/2 \rfloor$ is a bijection from $\mathbb{N}$ to $\mathbb{Z}$:

$$\begin{aligned} 0 &\rightarrow 0, \\ 1 &\rightarrow -1, \\ 2 &\rightarrow 1, \\ 3 &\rightarrow -2, \\ 4 &\rightarrow 2, \\ &\vdots \end{aligned}$$

The function is injective, since no element of $\mathbb{Z}$ is the image of more than one element of $\mathbb{N}$. It is surjective, since every element of $\mathbb{Z}$ appears as the image of some element of $\mathbb{N}$. Hence, the function is bijective. A bijection is sometimes called a **one-to-one correspondence**, since it pairs elements in the domain and codomain. A bijection from a set A to itself is sometimes called a **permutation**.

When a function f is bijective, we define its **inverse** f^{-1} as

$$f^{-1}(b) = a \text{ if and only if } f(a) = b.$$

For example, the inverse of the function $f(n) = (-1)^n \lceil n/2 \rceil$ is

$$f^{-1}(m) = \begin{cases} 2m & \text{if } m \geq 0, \\ -2m - 1 & \text{if } m < 0. \end{cases}$$

Exercises

B.3-1

Let A and B be finite sets, and let $f : A \rightarrow B$ be a function. Show that

- a. if f is injective, then $|A| \leq |B|$;
- b. if f is surjective, then $|A| \geq |B|$.

B.3-2

Is the function $f(x) = x + 1$ bijective when the domain and the codomain are $\mathbb{N}$? Is it bijective when the domain and the codomain are $\mathbb{Z}$?

B.3-3

Give a natural definition for the inverse of a binary relation such that if a relation is in fact a bijective function, its relational inverse is its functional inverse.

B.3-4 ★

Give a bijection from $\mathbb{Z}$ to $\mathbb{Z} \times \mathbb{Z}$.

B.4 Graphs

This section presents two kinds of graphs: directed and undirected. Certain definitions in the literature differ from those given here, but for the most part, the differences are slight. Section 22.1 shows how we can represent graphs in computer memory.

A **directed graph** (or **digraph**) G is a pair (V, E) , where V is a finite set and E is a binary relation on V . The set V is called the **vertex set** of G , and its elements are called **vertices** (singular: **vertex**). The set E is called the **edge set** of G , and its elements are called **edges**. Figure B.2(a) is a pictorial representation of a directed graph on the vertex set $\{1, 2, 3, 4, 5, 6\}$. Vertices are represented by circles in the figure, and edges are represented by arrows. Note that **self-loops**—edges from a vertex to itself—are possible.

In an **undirected graph** $G = (V, E)$, the edge set E consists of *unordered* pairs of vertices, rather than ordered pairs. That is, an edge is a set $\{u, v\}$, where

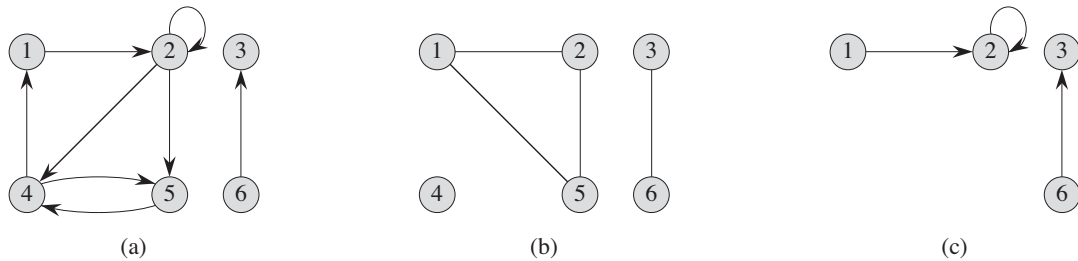


Figure B.2 Directed and undirected graphs. (a) A directed graph $G = (V, E)$, where $V = \{1, 2, 3, 4, 5, 6\}$ and $E = \{(1, 2), (2, 2), (2, 4), (2, 5), (4, 1), (4, 5), (5, 4), (6, 3)\}$. The edge $(2, 2)$ is a self-loop. (b) An undirected graph $G = (V, E)$, where $V = \{1, 2, 3, 4, 5, 6\}$ and $E = \{(1, 2), (1, 5), (2, 5), (3, 6)\}$. The vertex 4 is isolated. (c) The subgraph of the graph in part (a) induced by the vertex set $\{1, 2, 3, 6\}$.

$u, v \in V$ and $u \neq v$. By convention, we use the notation (u, v) for an edge, rather than the set notation $\{u, v\}$, and we consider (u, v) and (v, u) to be the same edge. In an undirected graph, self-loops are forbidden, and so every edge consists of two distinct vertices. Figure B.2(b) is a pictorial representation of an undirected graph on the vertex set $\{1, 2, 3, 4, 5, 6\}$.

Many definitions for directed and undirected graphs are the same, although certain terms have slightly different meanings in the two contexts. If (u, v) is an edge in a directed graph $G = (V, E)$, we say that (u, v) is **incident from** or **leaves** vertex u and is **incident to** or **enters** vertex v . For example, the edges leaving vertex 2 in Figure B.2(a) are $(2, 2)$, $(2, 4)$, and $(2, 5)$. The edges entering vertex 2 are $(1, 2)$ and $(2, 2)$. If (u, v) is an edge in an undirected graph $G = (V, E)$, we say that (u, v) is **incident on** vertices u and v . In Figure B.2(b), the edges incident on vertex 2 are $(1, 2)$ and $(2, 5)$.

If (u, v) is an edge in a graph $G = (V, E)$, we say that vertex v is **adjacent** to vertex u . When the graph is undirected, the adjacency relation is symmetric. When the graph is directed, the adjacency relation is not necessarily symmetric. If v is adjacent to u in a directed graph, we sometimes write $u \rightarrow v$. In parts (a) and (b) of Figure B.2, vertex 2 is adjacent to vertex 1, since the edge $(1, 2)$ belongs to both graphs. Vertex 1 is *not* adjacent to vertex 2 in Figure B.2(a), since the edge $(2, 1)$ does not belong to the graph.

The **degree** of a vertex in an undirected graph is the number of edges incident on it. For example, vertex 2 in Figure B.2(b) has degree 2. A vertex whose degree is 0, such as vertex 4 in Figure B.2(b), is **isolated**. In a directed graph, the **out-degree** of a vertex is the number of edges leaving it, and the **in-degree** of a vertex is the number of edges entering it. The **degree** of a vertex in a directed graph is its in-

degree plus its out-degree. Vertex 2 in Figure B.2(a) has in-degree 2, out-degree 3, and degree 5.

A **path** of **length** k from a vertex u to a vertex u' in a graph $G = (V, E)$ is a sequence $\langle v_0, v_1, v_2, \dots, v_k \rangle$ of vertices such that $u = v_0$, $u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$. The length of the path is the number of edges in the path. The path **contains** the vertices $v_0, v_1, \dots, v_k$ and the edges $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$. (There is always a 0-length path from u to u .) If there is a path p from u to u' , we say that u' is **reachable** from u via p , which we sometimes write as $u \xrightarrow{p} u'$ if G is directed. A path is **simple**⁴ if all vertices in the path are distinct. In Figure B.2(a), the path $\langle 1, 2, 5, 4 \rangle$ is a simple path of length 3. The path $\langle 2, 5, 4, 5 \rangle$ is not simple.

A **subpath** of path $p = \langle v_0, v_1, \dots, v_k \rangle$ is a contiguous subsequence of its vertices. That is, for any $0 \leq i \leq j \leq k$, the subsequence of vertices $\langle v_i, v_{i+1}, \dots, v_j \rangle$ is a subpath of p .

In a directed graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $v_0 = v_k$ and the path contains at least one edge. The cycle is **simple** if, in addition, $v_1, v_2, \dots, v_k$ are distinct. A self-loop is a cycle of length 1. Two paths $\langle v_0, v_1, v_2, \dots, v_{k-1}, v_0 \rangle$ and $\langle v'_0, v'_1, v'_2, \dots, v'_{k-1}, v'_0 \rangle$ form the same cycle if there exists an integer j such that $v'_i = v_{(i+j) \bmod k}$ for $i = 0, 1, \dots, k-1$. In Figure B.2(a), the path $\langle 1, 2, 4, 1 \rangle$ forms the same cycle as the paths $\langle 2, 4, 1, 2 \rangle$ and $\langle 4, 1, 2, 4 \rangle$. This cycle is simple, but the cycle $\langle 1, 2, 4, 5, 4, 1 \rangle$ is not. The cycle $\langle 2, 2 \rangle$ formed by the edge $(2, 2)$ is a self-loop. A directed graph with no self-loops is **simple**. In an undirected graph, a path $\langle v_0, v_1, \dots, v_k \rangle$ forms a **cycle** if $k \geq 3$ and $v_0 = v_k$; the cycle is **simple** if $v_1, v_2, \dots, v_k$ are distinct. For example, in Figure B.2(b), the path $\langle 1, 2, 5, 1 \rangle$ is a simple cycle. A graph with no cycles is **acyclic**.

An undirected graph is **connected** if every vertex is reachable from all other vertices. The **connected components** of a graph are the equivalence classes of vertices under the “is reachable from” relation. The graph in Figure B.2(b) has three connected components: $\{1, 2, 5\}$, $\{3, 6\}$, and $\{4\}$. Every vertex in $\{1, 2, 5\}$ is reachable from every other vertex in $\{1, 2, 5\}$. An undirected graph is connected if it has exactly one connected component. The edges of a connected component are those that are incident on only the vertices of the component; in other words, edge (u, v) is an edge of a connected component only if both u and v are vertices of the component.

A directed graph is **strongly connected** if every two vertices are reachable from each other. The **strongly connected components** of a directed graph are the equiv-

⁴Some authors refer to what we call a path as a “walk” and to what we call a simple path as just a “path.” We use the terms “path” and “simple path” throughout this book in a manner consistent with their definitions.

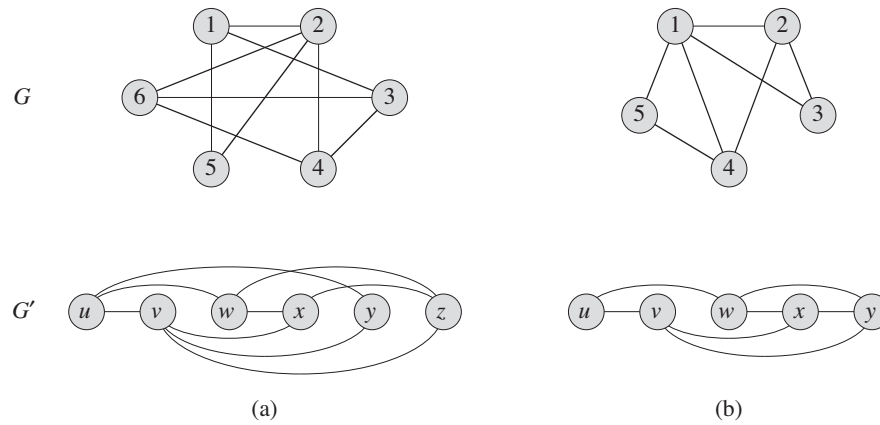


Figure B.3 (a) A pair of isomorphic graphs. The vertices of the top graph are mapped to the vertices of the bottom graph by $f(1) = u, f(2) = v, f(3) = w, f(4) = x, f(5) = y, f(6) = z$. (b) Two graphs that are not isomorphic, since the top graph has a vertex of degree 4 and the bottom graph does not.

alence classes of vertices under the “are mutually reachable” relation. A directed graph is strongly connected if it has only one strongly connected component. The graph in Figure B.2(a) has three strongly connected components: $\{1, 2, 4, 5\}$, $\{3\}$, and $\{6\}$. All pairs of vertices in $\{1, 2, 4, 5\}$ are mutually reachable. The vertices $\{3, 6\}$ do not form a strongly connected component, since vertex 6 cannot be reached from vertex 3.

Two graphs $G = (V, E)$ and $G' = (V', E')$ are **isomorphic** if there exists a bijection $f : V \rightarrow V'$ such that $(u, v) \in E$ if and only if $(f(u), f(v)) \in E'$. In other words, we can relabel the vertices of G to be vertices of G' , maintaining the corresponding edges in G and G' . Figure B.3(a) shows a pair of isomorphic graphs G and G' with respective vertex sets $V = \{1, 2, 3, 4, 5, 6\}$ and $V' = \{u, v, w, x, y, z\}$. The mapping from V to V' given by $f(1) = u, f(2) = v, f(3) = w, f(4) = x, f(5) = y, f(6) = z$ provides the required bijective function. The graphs in Figure B.3(b) are not isomorphic. Although both graphs have 5 vertices and 7 edges, the top graph has a vertex of degree 4 and the bottom graph does not.

We say that a graph $G' = (V', E')$ is a **subgraph** of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$. Given a set $V' \subseteq V$, the subgraph of G **induced** by V' is the graph $G' = (V', E')$, where

$$E' = \{(u, v) \in E : u, v \in V'\}.$$

The subgraph induced by the vertex set $\{1, 2, 3, 6\}$ in Figure B.2(a) appears in Figure B.2(c) and has the edge set $\{(1, 2), (2, 2), (6, 3)\}$.

Given an undirected graph $G = (V, E)$, the **directed version** of G is the directed graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $(u, v) \in E$. That is, we replace each undirected edge (u, v) in G by the two directed edges (u, v) and (v, u) in the directed version. Given a directed graph $G = (V, E)$, the **undirected version** of G is the undirected graph $G' = (V, E')$, where $(u, v) \in E'$ if and only if $u \neq v$ and $(u, v) \in E$. That is, the undirected version contains the edges of G “with their directions removed” and with self-loops eliminated. (Since (u, v) and (v, u) are the same edge in an undirected graph, the undirected version of a directed graph contains it only once, even if the directed graph contains both edges (u, v) and (v, u) .) In a directed graph $G = (V, E)$, a **neighbor** of a vertex u is any vertex that is adjacent to u in the undirected version of G . That is, v is a neighbor of u if $u \neq v$ and either $(u, v) \in E$ or $(v, u) \in E$. In an undirected graph, u and v are neighbors if they are adjacent.

Several kinds of graphs have special names. A **complete graph** is an undirected graph in which every pair of vertices is adjacent. A **bipartite graph** is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$ or $u \in V_2$ and $v \in V_1$. That is, all edges go between the two sets V_1 and V_2 . An acyclic, undirected graph is a **forest**, and a connected, acyclic, undirected graph is a (**free**) **tree** (see Section B.5). We often take the first letters of “directed acyclic graph” and call such a graph a **dag**.

There are two variants of graphs that you may occasionally encounter. A **multi-graph** is like an undirected graph, but it can have both multiple edges between vertices and self-loops. A **hypergraph** is like an undirected graph, but each **hyperedge**, rather than connecting two vertices, connects an arbitrary subset of vertices. Many algorithms written for ordinary directed and undirected graphs can be adapted to run on these graphlike structures.

The **contraction** of an undirected graph $G = (V, E)$ by an edge $e = (u, v)$ is a graph $G' = (V', E')$, where $V' = V - \{u, v\} \cup \{x\}$ and x is a new vertex. The set of edges E' is formed from E by deleting the edge (u, v) and, for each vertex w incident on u or v , deleting whichever of (u, w) and (v, w) is in E and adding the new edge (x, w) . In effect, u and v are “contracted” into a single vertex.

Exercises

B.4-1

Attendees of a faculty party shake hands to greet each other, and each professor remembers how many times he or she shook hands. At the end of the party, the department head adds up the number of times that each professor shook hands.

Show that the result is even by proving the *handshaking lemma*: if $G = (V, E)$ is an undirected graph, then

$$\sum_{v \in V} \text{degree}(v) = 2|E|.$$

B.4-2

Show that if a directed or undirected graph contains a path between two vertices u and v , then it contains a simple path between u and v . Show that if a directed graph contains a cycle, then it contains a simple cycle.

B.4-3

Show that any connected, undirected graph $G = (V, E)$ satisfies $|E| \geq |V| - 1$.

B.4-4

Verify that in an undirected graph, the “is reachable from” relation is an equivalence relation on the vertices of the graph. Which of the three properties of an equivalence relation hold in general for the “is reachable from” relation on the vertices of a directed graph?

B.4-5

What is the undirected version of the directed graph in Figure B.2(a)? What is the directed version of the undirected graph in Figure B.2(b)?

B.4-6 ★

Show that we can represent a hypergraph by a bipartite graph if we let incidence in the hypergraph correspond to adjacency in the bipartite graph. (*Hint*: Let one set of vertices in the bipartite graph correspond to vertices of the hypergraph, and let the other set of vertices of the bipartite graph correspond to hyperedges.)

B.5 Trees

As with graphs, there are many related, but slightly different, notions of trees. This section presents definitions and mathematical properties of several kinds of trees. Sections 10.4 and 22.1 describe how we can represent trees in computer memory.

B.5.1 Free trees

As defined in Section B.4, a *free tree* is a connected, acyclic, undirected graph. We often omit the adjective “free” when we say that a graph is a tree. If an undirected graph is acyclic but possibly disconnected, it is a *forest*. Many algorithms that work

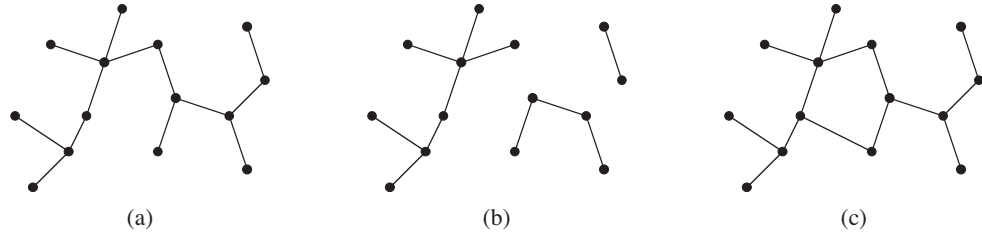


Figure B.4 (a) A free tree. (b) A forest. (c) A graph that contains a cycle and is therefore neither a tree nor a forest.

for trees also work for forests. Figure B.4(a) shows a free tree, and Figure B.4(b) shows a forest. The forest in Figure B.4(b) is not a tree because it is not connected. The graph in Figure B.4(c) is connected but neither a tree nor a forest, because it contains a cycle.

The following theorem captures many important facts about free trees.

Theorem B.2 (Properties of free trees)

Let $G = (V, E)$ be an undirected graph. The following statements are equivalent.

1. G is a free tree.
2. Any two vertices in G are connected by a unique simple path.
3. G is connected, but if any edge is removed from E , the resulting graph is disconnected.
4. G is connected, and $|E| = |V| - 1$.
5. G is acyclic, and $|E| = |V| - 1$.
6. G is acyclic, but if any edge is added to E , the resulting graph contains a cycle.

Proof (1) $\Rightarrow$ (2): Since a tree is connected, any two vertices in G are connected by at least one simple path. Suppose, for the sake of contradiction, that vertices u and v are connected by two distinct simple paths p_1 and p_2 , as shown in Figure B.5. Let w be the vertex at which the paths first diverge; that is, w is the first vertex on both p_1 and p_2 whose successor on p_1 is x and whose successor on p_2 is y , where $x \neq y$. Let z be the first vertex at which the paths reconverge; that is, z is the first vertex following w on p_1 that is also on p_2 . Let p' be the subpath of p_1 from w through x to z , and let p'' be the subpath of p_2 from w through y to z . Paths p' and p'' share no vertices except their endpoints. Thus, the path obtained by concatenating p' and the reverse of p'' is a cycle, which contradicts our assumption

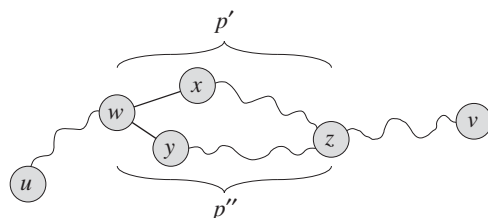


Figure B.5 A step in the proof of Theorem B.2: if (1) G is a free tree, then (2) any two vertices in G are connected by a unique simple path. Assume for the sake of contradiction that vertices u and v are connected by two distinct simple paths p_1 and p_2 . These paths first diverge at vertex w , and they first re-converge at vertex z . The path p' concatenated with the reverse of the path p'' forms a cycle, which yields the contradiction.

that G is a tree. Thus, if G is a tree, there can be at most one simple path between two vertices.

(2) $\Rightarrow$ (3): If any two vertices in G are connected by a unique simple path, then G is connected. Let (u, v) be any edge in E . This edge is a path from u to v , and so it must be the unique path from u to v . If we remove (u, v) from G , there is no path from u to v , and hence its removal disconnects G .

(3) $\Rightarrow$ (4): By assumption, the graph G is connected, and by Exercise B.4-3, we have $|E| \geq |V| - 1$. We shall prove $|E| \leq |V| - 1$ by induction. A connected graph with $n = 1$ or $n = 2$ vertices has $n - 1$ edges. Suppose that G has $n \geq 3$ vertices and that all graphs satisfying (3) with fewer than n vertices also satisfy $|E| \leq |V| - 1$. Removing an arbitrary edge from G separates the graph into $k \geq 2$ connected components (actually $k = 2$). Each component satisfies (3), or else G would not satisfy (3). If we view each connected component V_i , with edge set E_i , as its own free tree, then because each component has fewer than $|V|$ vertices, by the inductive hypothesis we have $|E_i| \leq |V_i| - 1$. Thus, the number of edges in all components combined is at most $|V| - k \leq |V| - 2$. Adding in the removed edge yields $|E| \leq |V| - 1$.

(4) $\Rightarrow$ (5): Suppose that G is connected and that $|E| = |V| - 1$. We must show that G is acyclic. Suppose that G has a cycle containing k vertices $v_1, v_2, \dots, v_k$, and without loss of generality assume that this cycle is simple. Let $G_k = (V_k, E_k)$ be the subgraph of G consisting of the cycle. Note that $|V_k| = |E_k| = k$. If $k < |V|$, there must be a vertex $v_{k+1} \in V - V_k$ that is adjacent to some vertex $v_i \in V_k$, since G is connected. Define $G_{k+1} = (V_{k+1}, E_{k+1})$ to be the subgraph of G with $V_{k+1} = V_k \cup \{v_{k+1}\}$ and $E_{k+1} = E_k \cup \{(v_i, v_{k+1})\}$. Note that $|V_{k+1}| = |E_{k+1}| = k + 1$. If $k + 1 < |V|$, we can continue, defining G_{k+2} in the same manner, and so forth, until we obtain $G_n = (V_n, E_n)$, where $n = |V|$,

$V_n = V$, and $|E_n| = |V_n| = |V|$. Since G_n is a subgraph of G , we have $E_n \subseteq E$, and hence $|E| \geq |V|$, which contradicts the assumption that $|E| = |V| - 1$. Thus, G is acyclic.

(5) $\Rightarrow$ (6): Suppose that G is acyclic and that $|E| = |V| - 1$. Let k be the number of connected components of G . Each connected component is a free tree by definition, and since (1) implies (5), the sum of all edges in all connected components of G is $|V| - k$. Consequently, we must have $k = 1$, and G is in fact a tree. Since (1) implies (2), any two vertices in G are connected by a unique simple path. Thus, adding any edge to G creates a cycle.

(6) $\Rightarrow$ (1): Suppose that G is acyclic but that adding any edge to E creates a cycle. We must show that G is connected. Let u and v be arbitrary vertices in G . If u and v are not already adjacent, adding the edge (u, v) creates a cycle in which all edges but (u, v) belong to G . Thus, the cycle minus edge (u, v) must contain a path from u to v , and since u and v were chosen arbitrarily, G is connected. ■

B.5.2 Rooted and ordered trees

A **rooted tree** is a free tree in which one of the vertices is distinguished from the others. We call the distinguished vertex the **root** of the tree. We often refer to a vertex of a rooted tree as a **node**⁵ of the tree. Figure B.6(a) shows a rooted tree on a set of 12 nodes with root 7.

Consider a node x in a rooted tree T with root r . We call any node y on the unique simple path from r to x an **ancestor** of x . If y is an ancestor of x , then x is a **descendant** of y . (Every node is both an ancestor and a descendant of itself.) If y is an ancestor of x and $x \neq y$, then y is a **proper ancestor** of x and x is a **proper descendant** of y . The **subtree rooted at x** is the tree induced by descendants of x , rooted at x . For example, the subtree rooted at node 8 in Figure B.6(a) contains nodes 8, 6, 5, and 9.

If the last edge on the simple path from the root r of a tree T to a node x is (y, x) , then y is the **parent** of x , and x is a **child** of y . The root is the only node in T with no parent. If two nodes have the same parent, they are **siblings**. A node with no children is a **leaf** or **external node**. A nonleaf node is an **internal node**.

⁵The term “node” is often used in the graph theory literature as a synonym for “vertex.” We reserve the term “node” to mean a vertex of a rooted tree.

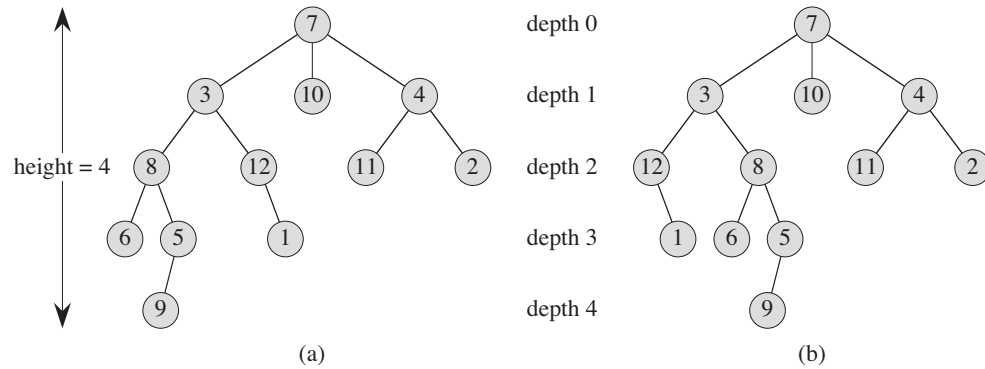


Figure B.6 Rooted and ordered trees. **(a)** A rooted tree with height 4. The tree is drawn in a standard way: the root (node 7) is at the top, its children (nodes with depth 1) are beneath it, their children (nodes with depth 2) are beneath them, and so forth. If the tree is ordered, the relative left-to-right order of the children of a node matters; otherwise it doesn't. **(b)** Another rooted tree. As a rooted tree, it is identical to the tree in (a), but as an ordered tree it is different, since the children of node 3 appear in a different order.

The number of children of a node x in a rooted tree T equals the *degree* of x .⁶ The length of the simple path from the root r to a node x is the *depth* of x in T . A *level* of a tree consists of all nodes at the same depth. The *height* of a node in a tree is the number of edges on the longest simple downward path from the node to a leaf, and the height of a tree is the height of its root. The height of a tree is also equal to the largest depth of any node in the tree.

An *ordered tree* is a rooted tree in which the children of each node are ordered. That is, if a node has k children, then there is a first child, a second child, ..., and a k th child. The two trees in Figure B.6 are different when considered to be ordered trees, but the same when considered to be just rooted trees.

B.5.3 Binary and positional trees

We define binary trees recursively. A *binary tree* T is a structure defined on a finite set of nodes that either

- contains no nodes, or

⁶Notice that the degree of a node depends on whether we consider T to be a rooted tree or a free tree. The degree of a vertex in a free tree is, as in any undirected graph, the number of adjacent vertices. In a rooted tree, however, the degree is the number of children—the parent of a node does not count toward its degree.

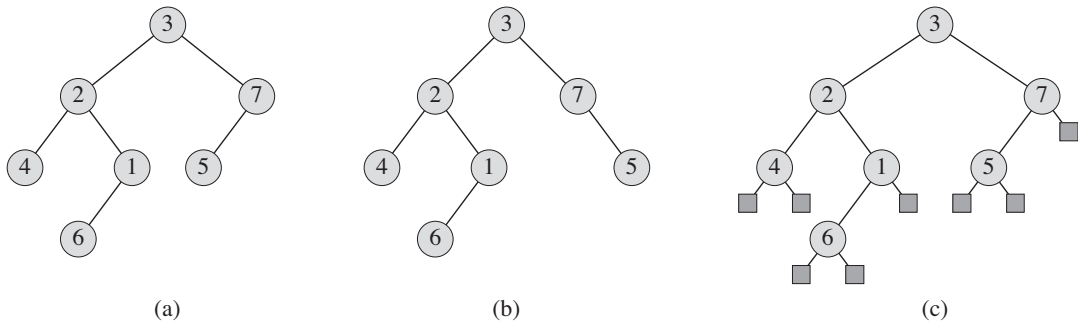


Figure B.7 Binary trees. **(a)** A binary tree drawn in a standard way. The left child of a node is drawn beneath the node and to the left. The right child is drawn beneath and to the right. **(b)** A binary tree different from the one in (a). In (a), the left child of node 7 is 5 and the right child is absent. In (b), the left child of node 7 is absent and the right child is 5. As ordered trees, these trees are the same, but as binary trees, they are distinct. **(c)** The binary tree in (a) represented by the internal nodes of a full binary tree: an ordered tree in which each internal node has degree 2. The leaves in the tree are shown as squares.

- is composed of three disjoint sets of nodes: a **root** node, a binary tree called its **left subtree**, and a binary tree called its **right subtree**.

The binary tree that contains no nodes is called the **empty tree** or **null tree**, sometimes denoted NIL. If the left subtree is nonempty, its root is called the **left child** of the root of the entire tree. Likewise, the root of a nonnull right subtree is the **right child** of the root of the entire tree. If a subtree is the null tree NIL, we say that the child is **absent** or **missing**. Figure B.7(a) shows a binary tree.

A binary tree is not simply an ordered tree in which each node has degree at most 2. For example, in a binary tree, if a node has just one child, the position of the child—whether it is the **left child** or the **right child**—matters. In an ordered tree, there is no distinguishing a sole child as being either left or right. Figure B.7(b) shows a binary tree that differs from the tree in Figure B.7(a) because of the position of one node. Considered as ordered trees, however, the two trees are identical.

We can represent the positioning information in a binary tree by the internal nodes of an ordered tree, as shown in Figure B.7(c). The idea is to replace each missing child in the binary tree with a node having no children. These leaf nodes are drawn as squares in the figure. The tree that results is a **full binary tree**: each node is either a leaf or has degree exactly 2. There are no degree-1 nodes. Consequently, the order of the children of a node preserves the position information.

We can extend the positioning information that distinguishes binary trees from ordered trees to trees with more than 2 children per node. In a **positional tree**, the

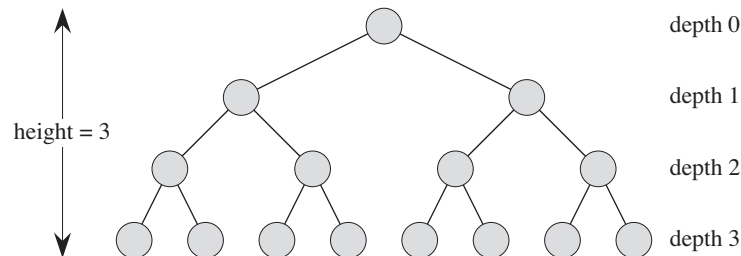


Figure B.8 A complete binary tree of height 3 with 8 leaves and 7 internal nodes.

children of a node are labeled with distinct positive integers. The i th child of a node is **absent** if no child is labeled with integer i . A **k -ary tree** is a positional tree in which for every node, all children with labels greater than k are missing. Thus, a binary tree is a k -ary tree with $k = 2$.

A **complete k -ary tree** is a k -ary tree in which all leaves have the same depth and all internal nodes have degree k . Figure B.8 shows a complete binary tree of height 3. How many leaves does a complete k -ary tree of height h have? The root has k children at depth 1, each of which has k children at depth 2, etc. Thus, the number of leaves at depth h is k^h . Consequently, the height of a complete k -ary tree with n leaves is $\log_k n$. The number of internal nodes of a complete k -ary tree of height h is

$$\begin{aligned} 1 + k + k^2 + \cdots + k^{h-1} &= \sum_{i=0}^{h-1} k^i \\ &= \frac{k^h - 1}{k - 1} \end{aligned}$$

by equation (A.5). Thus, a complete binary tree has $2^h - 1$ internal nodes.

Exercises

B.5-1

Draw all the free trees composed of the three vertices x , y , and z . Draw all the rooted trees with nodes x , y , and z with x as the root. Draw all the ordered trees with nodes x , y , and z with x as the root. Draw all the binary trees with nodes x , y , and z with x as the root.

B.5-2

Let $G = (V, E)$ be a directed acyclic graph in which there is a vertex $v_0 \in V$ such that there exists a unique path from v_0 to every vertex $v \in V$. Prove that the undirected version of G forms a tree.

B.5-3

Show by induction that the number of degree-2 nodes in any nonempty binary tree is 1 fewer than the number of leaves. Conclude that the number of internal nodes in a full binary tree is 1 fewer than the number of leaves.

B.5-4

Use induction to show that a nonempty binary tree with n nodes has height at least $\lfloor \lg n \rfloor$.

B.5-5 ★

The *internal path length* of a full binary tree is the sum, taken over all internal nodes of the tree, of the depth of each node. Likewise, the *external path length* is the sum, taken over all leaves of the tree, of the depth of each leaf. Consider a full binary tree with n internal nodes, internal path length i , and external path length e . Prove that $e = i + 2n$.

B.5-6 ★

Let us associate a “weight” $w(x) = 2^{-d}$ with each leaf x of depth d in a binary tree T , and let L be the set of leaves of T . Prove that $\sum_{x \in L} w(x) \leq 1$. (This is known as the *Kraft inequality*.)

B.5-7 ★

Show that if $L \geq 2$, then every binary tree with L leaves contains a subtree having between $L/3$ and $2L/3$ leaves, inclusive.

Problems
B-1 Graph coloring

Given an undirected graph $G = (V, E)$, a *k-coloring* of G is a function $c : V \rightarrow \{0, 1, \dots, k-1\}$ such that $c(u) \neq c(v)$ for every edge $(u, v) \in E$. In other words, the numbers $0, 1, \dots, k-1$ represent the k colors, and adjacent vertices must have different colors.

a. Show that any tree is 2-colorable.

- b.** Show that the following are equivalent:
1. G is bipartite.
 2. G is 2-colorable.
 3. G has no cycles of odd length.
- c.** Let d be the maximum degree of any vertex in a graph G . Prove that we can color G with $d + 1$ colors.
- d.** Show that if G has $O(|V|)$ edges, then we can color G with $O(\sqrt{|V|})$ colors.

B-2 Friendly graphs

Reword each of the following statements as a theorem about undirected graphs, and then prove it. Assume that friendship is symmetric but not reflexive.

- a.** Any group of at least two people contains at least two people with the same number of friends in the group.
- b.** Every group of six people contains either at least three mutual friends or at least three mutual strangers.
- c.** Any group of people can be partitioned into two subgroups such that at least half the friends of each person belong to the subgroup of which that person is *not* a member.
- d.** If everyone in a group is the friend of at least half the people in the group, then the group can be seated around a table in such a way that everyone is seated between two friends.

B-3 Bisecting trees

Many divide-and-conquer algorithms that operate on graphs require that the graph be bisected into two nearly equal-sized subgraphs, which are induced by a partition of the vertices. This problem investigates bisections of trees formed by removing a small number of edges. We require that whenever two vertices end up in the same subtree after removing edges, then they must be in the same partition.

- a.** Show that we can partition the vertices of any n -vertex binary tree into two sets A and B , such that $|A| \leq 3n/4$ and $|B| \leq 3n/4$, by removing a single edge.
- b.** Show that the constant $3/4$ in part (a) is optimal in the worst case by giving an example of a simple binary tree whose most evenly balanced partition upon removal of a single edge has $|A| = 3n/4$.

- c. Show that by removing at most $O(\lg n)$ edges, we can partition the vertices of any n -vertex binary tree into two sets A and B such that $|A| = \lfloor n/2 \rfloor$ and $|B| = \lceil n/2 \rceil$.

Appendix notes

G. Boole pioneered the development of symbolic logic, and he introduced many of the basic set notations in a book published in 1854. Modern set theory was created by G. Cantor during the period 1874–1895. Cantor focused primarily on sets of infinite cardinality. The term “function” is attributed to G. W. Leibniz, who used it to refer to several kinds of mathematical formulas. His limited definition has been generalized many times. Graph theory originated in 1736, when L. Euler proved that it was impossible to cross each of the seven bridges in the city of Königsberg exactly once and return to the starting point.

The book by Harary [160] provides a useful compendium of many definitions and results from graph theory.

C Counting and Probability

This appendix reviews elementary combinatorics and probability theory. If you have a good background in these areas, you may want to skim the beginning of this appendix lightly and concentrate on the later sections. Most of this book's chapters do not require probability, but for some chapters it is essential.

Section C.1 reviews elementary results in counting theory, including standard formulas for counting permutations and combinations. The axioms of probability and basic facts concerning probability distributions form Section C.2. Random variables are introduced in Section C.3, along with the properties of expectation and variance. Section C.4 investigates the geometric and binomial distributions that arise from studying Bernoulli trials. The study of the binomial distribution continues in Section C.5, an advanced discussion of the “tails” of the distribution.

C.1 Counting

Counting theory tries to answer the question “How many?” without actually enumerating all the choices. For example, we might ask, “How many different n -bit numbers are there?” or “How many orderings of n distinct elements are there?” In this section, we review the elements of counting theory. Since some of the material assumes a basic understanding of sets, you might wish to start by reviewing the material in Section B.1.

Rules of sum and product

We can sometimes express a set of items that we wish to count as a union of disjoint sets or as a Cartesian product of sets.

The **rule of sum** says that the number of ways to choose one element from one of two *disjoint* sets is the sum of the cardinalities of the sets. That is, if A and B are two finite sets with no members in common, then $|A \cup B| = |A| + |B|$, which

follows from equation (B.3). For example, each position on a car's license plate is a letter or a digit. The number of possibilities for each position is therefore $26 + 10 = 36$, since there are 26 choices if it is a letter and 10 choices if it is a digit.

The **rule of product** says that the number of ways to choose an ordered pair is the number of ways to choose the first element times the number of ways to choose the second element. That is, if A and B are two finite sets, then $|A \times B| = |A| \cdot |B|$, which is simply equation (B.4). For example, if an ice-cream parlor offers 28 flavors of ice cream and 4 toppings, the number of possible sundaes with one scoop of ice cream and one topping is $28 \cdot 4 = 112$.

Strings

A **string** over a finite set S is a sequence of elements of S . For example, there are 8 binary strings of length 3:

000, 001, 010, 011, 100, 101, 110, 111 .

We sometimes call a string of length k a **k -string**. A **substring** s' of a string s is an ordered sequence of consecutive elements of s . A **k -substring** of a string is a substring of length k . For example, 010 is a 3-substring of 01101001 (the 3-substring that begins in position 4), but 111 is not a substring of 01101001.

We can view a k -string over a set S as an element of the Cartesian product S^k of k -tuples; thus, there are $|S|^k$ strings of length k . For example, the number of binary k -strings is 2^k . Intuitively, to construct a k -string over an n -set, we have n ways to pick the first element; for each of these choices, we have n ways to pick the second element; and so forth k times. This construction leads to the k -fold product $n \cdot n \cdots n = n^k$ as the number of k -strings.

Permutations

A **permutation** of a finite set S is an ordered sequence of all the elements of S , with each element appearing exactly once. For example, if $S = \{a, b, c\}$, then S has 6 permutations:

$abc, acb, bac, bca, cab, cba$.

There are $n!$ permutations of a set of n elements, since we can choose the first element of the sequence in n ways, the second in $n - 1$ ways, the third in $n - 2$ ways, and so on.

A **k -permutation** of S is an ordered sequence of k elements of S , with no element appearing more than once in the sequence. (Thus, an ordinary permutation is an n -permutation of an n -set.) The twelve 2-permutations of the set $\{a, b, c, d\}$ are

$ab, ac, ad, ba, bc, bd, ca, cb, cd, da, db, dc$.

The number of k -permutations of an n -set is

$$n(n-1)(n-2)\cdots(n-k+1) = \frac{n!}{(n-k)!} , \quad (\text{C.1})$$

since we have n ways to choose the first element, $n-1$ ways to choose the second element, and so on, until we have selected k elements, the last being a selection from the remaining $n-k+1$ elements.

Combinations

A **k -combination** of an n -set S is simply a k -subset of S . For example, the 4-set $\{a, b, c, d\}$ has six 2-combinations:

ab, ac, ad, bc, bd, cd .

(Here we use the shorthand of denoting the 2-subset $\{a, b\}$ by ab , and so on.) We can construct a k -combination of an n -set by choosing k distinct (different) elements from the n -set. The order in which we select the elements does not matter.

We can express the number of k -combinations of an n -set in terms of the number of k -permutations of an n -set. Every k -combination has exactly $k!$ permutations of its elements, each of which is a distinct k -permutation of the n -set. Thus, the number of k -combinations of an n -set is the number of k -permutations divided by $k!$; from equation (C.1), this quantity is

$$\frac{n!}{k!(n-k)!} . \quad (\text{C.2})$$

For $k=0$, this formula tells us that the number of ways to choose 0 elements from an n -set is 1 (not 0), since $0! = 1$.

Binomial coefficients

The notation $\binom{n}{k}$ (read “ n choose k ”) denotes the number of k -combinations of an n -set. From equation (C.2), we have

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} .$$

This formula is symmetric in k and $n-k$:

$$\binom{n}{k} = \binom{n}{n-k} . \quad (\text{C.3})$$

These numbers are also known as **binomial coefficients**, due to their appearance in the **binomial expansion**:

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}. \quad (\text{C.4})$$

A special case of the binomial expansion occurs when $x = y = 1$:

$$2^n = \sum_{k=0}^n \binom{n}{k}.$$

This formula corresponds to counting the 2^n binary n -strings by the number of 1s they contain: $\binom{n}{k}$ binary n -strings contain exactly k 1s, since we have $\binom{n}{k}$ ways to choose k out of the n positions in which to place the 1s.

Many identities involve binomial coefficients. The exercises at the end of this section give you the opportunity to prove a few.

Binomial bounds

We sometimes need to bound the size of a binomial coefficient. For $1 \leq k \leq n$, we have the lower bound

$$\begin{aligned} \binom{n}{k} &= \frac{n(n-1) \cdots (n-k+1)}{k(k-1) \cdots 1} \\ &= \left(\frac{n}{k}\right) \left(\frac{n-1}{k-1}\right) \cdots \left(\frac{n-k+1}{1}\right) \\ &\geq \left(\frac{n}{k}\right)^k. \end{aligned}$$

Taking advantage of the inequality $k! \geq (k/e)^k$ derived from Stirling's approximation (3.18), we obtain the upper bounds

$$\begin{aligned} \binom{n}{k} &= \frac{n(n-1) \cdots (n-k+1)}{k(k-1) \cdots 1} \\ &\leq \frac{n^k}{k!} \\ &\leq \left(\frac{en}{k}\right)^k. \end{aligned} \quad (\text{C.5})$$

For all integers k such that $0 \leq k \leq n$, we can use induction (see Exercise C.1-12) to prove the bound

$$\binom{n}{k} \leq \frac{n^n}{k^k (n-k)^{n-k}}, \quad (\text{C.6})$$

where for convenience we assume that $0^0 = 1$. For $k = \lambda n$, where $0 \leq \lambda \leq 1$, we can rewrite this bound as

$$\begin{aligned} \binom{n}{\lambda n} &\leq \frac{n^n}{(\lambda n)^{\lambda n} ((1-\lambda)n)^{(1-\lambda)n}} \\ &= \left(\left(\frac{1}{\lambda} \right)^\lambda \left(\frac{1}{1-\lambda} \right)^{1-\lambda} \right)^n \\ &= 2^{n H(\lambda)}, \end{aligned}$$

where

$$H(\lambda) = -\lambda \lg \lambda - (1-\lambda) \lg(1-\lambda) \quad (\text{C.7})$$

is the **(binary) entropy function** and where, for convenience, we assume that $0 \lg 0 = 0$, so that $H(0) = H(1) = 0$.

Exercises

C.1-1

How many k -substrings does an n -string have? (Consider identical k -substrings at different positions to be different.) How many substrings does an n -string have in total?

C.1-2

An n -input, m -output **boolean function** is a function from $\{\text{TRUE}, \text{FALSE}\}^n$ to $\{\text{TRUE}, \text{FALSE}\}^m$. How many n -input, 1-output boolean functions are there? How many n -input, m -output boolean functions are there?

C.1-3

In how many ways can n professors sit around a circular conference table? Consider two seatings to be the same if one can be rotated to form the other.

C.1-4

In how many ways can we choose three distinct numbers from the set $\{1, 2, \dots, 99\}$ so that their sum is even?

C.1-5

Prove the identity

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1} \quad (\text{C.8})$$

for $0 < k \leq n$.

C.1-6

Prove the identity

$$\binom{n}{k} = \frac{n}{n-k} \binom{n-1}{k}$$

for $0 \leq k < n$.

C.1-7

To choose k objects from n , you can make one of the objects distinguished and consider whether the distinguished object is chosen. Use this approach to prove that

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

C.1-8

Using the result of Exercise C.1-7, make a table for $n = 0, 1, \dots, 6$ and $0 \leq k \leq n$ of the binomial coefficients $\binom{n}{k}$ with $\binom{0}{0}$ at the top, $\binom{1}{0}$ and $\binom{1}{1}$ on the next line, and so forth. Such a table of binomial coefficients is called *Pascal's triangle*.

C.1-9

Prove that

$$\sum_{i=1}^n i = \binom{n+1}{2}.$$

C.1-10

Show that for any integers $n \geq 0$ and $0 \leq k \leq n$, the expression $\binom{n}{k}$ achieves its maximum value when $k = \lfloor n/2 \rfloor$ or $k = \lceil n/2 \rceil$.

C.1-11 ★

Argue that for any integers $n \geq 0$, $j \geq 0$, $k \geq 0$, and $j + k \leq n$,

$$\binom{n}{j+k} \leq \binom{n}{j} \binom{n-j}{k}. \quad (\text{C.9})$$

Provide both an algebraic proof and an argument based on a method for choosing $j + k$ items out of n . Give an example in which equality does not hold.

C.1-12 ★

Use induction on all integers k such that $0 \leq k \leq n/2$ to prove inequality (C.6), and use equation (C.3) to extend it to all integers k such that $0 \leq k \leq n$.

C.1-13 ★

Use Stirling's approximation to prove that

$$\binom{2n}{n} = \frac{2^{2n}}{\sqrt{\pi n}} (1 + O(1/n)) . \quad (\text{C.10})$$

C.1-14 ★

By differentiating the entropy function $H(\lambda)$, show that it achieves its maximum value at $\lambda = 1/2$. What is $H(1/2)$?

C.1-15 ★

Show that for any integer $n \geq 0$,

$$\sum_{k=0}^n \binom{n}{k} k = n 2^{n-1} . \quad (\text{C.11})$$

C.2 Probability

Probability is an essential tool for the design and analysis of probabilistic and randomized algorithms. This section reviews basic probability theory.

We define probability in terms of a *sample space* S , which is a set whose elements are called *elementary events*. We can think of each elementary event as a possible outcome of an experiment. For the experiment of flipping two distinguishable coins, with each individual flip resulting in a head (H) or a tail (T), we can view the sample space as consisting of the set of all possible 2-strings over $\{H, T\}$:

$$S = \{HH, HT, TH, TT\} .$$

An **event** is a subset¹ of the sample space S . For example, in the experiment of flipping two coins, the event of obtaining one head and one tail is $\{HT, TH\}$. The event S is called the **certain event**, and the event $\emptyset$ is called the **null event**. We say that two events A and B are **mutually exclusive** if $A \cap B = \emptyset$. We sometimes treat an elementary event $s \in S$ as the event $\{s\}$. By definition, all elementary events are mutually exclusive.

Axioms of probability

A **probability distribution** $\Pr\{\}$ on a sample space S is a mapping from events of S to real numbers satisfying the following **probability axioms**:

1. $\Pr\{A\} \geq 0$ for any event A .
2. $\Pr\{S\} = 1$.
3. $\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\}$ for any two mutually exclusive events A and B . More generally, for any (finite or countably infinite) sequence of events $A_1, A_2, \dots$ that are pairwise mutually exclusive,

$$\Pr\left\{\bigcup_i A_i\right\} = \sum_i \Pr\{A_i\}.$$

We call $\Pr\{A\}$ the **probability** of the event A . We note here that axiom 2 is a normalization requirement: there is really nothing fundamental about choosing 1 as the probability of the certain event, except that it is natural and convenient.

Several results follow immediately from these axioms and basic set theory (see Section B.1). The null event $\emptyset$ has probability $\Pr\{\emptyset\} = 0$. If $A \subseteq B$, then $\Pr\{A\} \leq \Pr\{B\}$. Using $\overline{A}$ to denote the event $S - A$ (the **complement** of A), we have $\Pr\{\overline{A}\} = 1 - \Pr\{A\}$. For any two events A and B ,

$$\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\} - \Pr\{A \cap B\} \tag{C.12}$$

$$\leq \Pr\{A\} + \Pr\{B\}. \tag{C.13}$$

¹For a general probability distribution, there may be some subsets of the sample space S that are not considered to be events. This situation usually arises when the sample space is uncountably infinite. The main requirement for what subsets are events is that the set of events of a sample space be closed under the operations of taking the complement of an event, forming the union of a finite or countable number of events, and taking the intersection of a finite or countable number of events. Most of the probability distributions we shall see are over finite or countable sample spaces, and we shall generally consider all subsets of a sample space to be events. A notable exception is the continuous uniform probability distribution, which we shall see shortly.

In our coin-flipping example, suppose that each of the four elementary events has probability $1/4$. Then the probability of getting at least one head is

$$\begin{aligned}\Pr\{HH, HT, TH\} &= \Pr\{HH\} + \Pr\{HT\} + \Pr\{TH\} \\ &= 3/4.\end{aligned}$$

Alternatively, since the probability of getting strictly less than one head is $\Pr\{TT\} = 1/4$, the probability of getting at least one head is $1 - 1/4 = 3/4$.

Discrete probability distributions

A probability distribution is **discrete** if it is defined over a finite or countably infinite sample space. Let S be the sample space. Then for any event A ,

$$\Pr\{A\} = \sum_{s \in A} \Pr\{s\},$$

since elementary events, specifically those in A , are mutually exclusive. If S is finite and every elementary event $s \in S$ has probability

$$\Pr\{s\} = 1/|S|,$$

then we have the **uniform probability distribution** on S . In such a case the experiment is often described as “picking an element of S at random.”

As an example, consider the process of flipping a **fair coin**, one for which the probability of obtaining a head is the same as the probability of obtaining a tail, that is, $1/2$. If we flip the coin n times, we have the uniform probability distribution defined on the sample space $S = \{H, T\}^n$, a set of size 2^n . We can represent each elementary event in S as a string of length n over $\{H, T\}$, each string occurring with probability $1/2^n$. The event

$$A = \{\text{exactly } k \text{ heads and exactly } n - k \text{ tails occur}\}$$

is a subset of S of size $|A| = \binom{n}{k}$, since $\binom{n}{k}$ strings of length n over $\{H, T\}$ contain exactly k H's. The probability of event A is thus $\Pr\{A\} = \binom{n}{k}/2^n$.

Continuous uniform probability distribution

The continuous uniform probability distribution is an example of a probability distribution in which not all subsets of the sample space are considered to be events. The continuous uniform probability distribution is defined over a closed interval $[a, b]$ of the reals, where $a < b$. Our intuition is that each point in the interval $[a, b]$ should be “equally likely.” There are an uncountable number of points, however, so if we give all points the same finite, positive probability, we cannot simultaneously satisfy axioms 2 and 3. For this reason, we would like to associate a

probability only with *some* of the subsets of S , in such a way that the axioms are satisfied for these events.

For any closed interval $[c, d]$, where $a \leq c \leq d \leq b$, the **continuous uniform probability distribution** defines the probability of the event $[c, d]$ to be

$$\Pr\{[c, d]\} = \frac{d - c}{b - a}.$$

Note that for any point $x = [x, x]$, the probability of x is 0. If we remove the endpoints of an interval $[c, d]$, we obtain the open interval (c, d) . Since $[c, d] = [c, c] \cup (c, d) \cup [d, d]$, axiom 3 gives us $\Pr\{[c, d]\} = \Pr\{(c, d)\}$. Generally, the set of events for the continuous uniform probability distribution contains any subset of the sample space $[a, b]$ that can be obtained by a finite or countable union of open and closed intervals, as well as certain more complicated sets.

Conditional probability and independence

Sometimes we have some prior partial knowledge about the outcome of an experiment. For example, suppose that a friend has flipped two fair coins and has told you that at least one of the coins showed a head. What is the probability that both coins are heads? The information given eliminates the possibility of two tails. The three remaining elementary events are equally likely, so we infer that each occurs with probability $1/3$. Since only one of these elementary events shows two heads, the answer to our question is $1/3$.

Conditional probability formalizes the notion of having prior partial knowledge of the outcome of an experiment. The **conditional probability** of an event A given that another event B occurs is defined to be

$$\Pr\{A \mid B\} = \frac{\Pr\{A \cap B\}}{\Pr\{B\}} \quad (\text{C.14})$$

whenever $\Pr\{B\} \neq 0$. (We read “ $\Pr\{A \mid B\}$ ” as “the probability of A given B .”) Intuitively, since we are given that event B occurs, the event that A also occurs is $A \cap B$. That is, $A \cap B$ is the set of outcomes in which both A and B occur. Because the outcome is one of the elementary events in B , we normalize the probabilities of all the elementary events in B by dividing them by $\Pr\{B\}$, so that they sum to 1. The conditional probability of A given B is, therefore, the ratio of the probability of event $A \cap B$ to the probability of event B . In the example above, A is the event that both coins are heads, and B is the event that at least one coin is a head. Thus, $\Pr\{A \mid B\} = (1/4)/(3/4) = 1/3$.

Two events are **independent** if

$$\Pr\{A \cap B\} = \Pr\{A\} \Pr\{B\}, \quad (\text{C.15})$$

which is equivalent, if $\Pr\{B\} \neq 0$, to the condition

$$\Pr\{A \mid B\} = \Pr\{A\} .$$

For example, suppose that we flip two fair coins and that the outcomes are independent. Then the probability of two heads is $(1/2)(1/2) = 1/4$. Now suppose that one event is that the first coin comes up heads and the other event is that the coins come up differently. Each of these events occurs with probability $1/2$, and the probability that both events occur is $1/4$; thus, according to the definition of independence, the events are independent—even though you might think that both events depend on the first coin. Finally, suppose that the coins are welded together so that they both fall heads or both fall tails and that the two possibilities are equally likely. Then the probability that each coin comes up heads is $1/2$, but the probability that they both come up heads is $1/2 \neq (1/2)(1/2)$. Consequently, the event that one comes up heads and the event that the other comes up heads are not independent.

A collection $A_1, A_2, \dots, A_n$ of events is said to be *pairwise independent* if

$$\Pr\{A_i \cap A_j\} = \Pr\{A_i\} \Pr\{A_j\}$$

for all $1 \leq i < j \leq n$. We say that the events of the collection are (*mutually*) *independent* if every k -subset $A_{i_1}, A_{i_2}, \dots, A_{i_k}$ of the collection, where $2 \leq k \leq n$ and $1 \leq i_1 < i_2 < \dots < i_k \leq n$, satisfies

$$\Pr\{A_{i_1} \cap A_{i_2} \cap \dots \cap A_{i_k}\} = \Pr\{A_{i_1}\} \Pr\{A_{i_2}\} \dots \Pr\{A_{i_k}\} .$$

For example, suppose we flip two fair coins. Let A_1 be the event that the first coin is heads, let A_2 be the event that the second coin is heads, and let A_3 be the event that the two coins are different. We have

$$\begin{aligned} \Pr\{A_1\} &= 1/2 , \\ \Pr\{A_2\} &= 1/2 , \\ \Pr\{A_3\} &= 1/2 , \\ \Pr\{A_1 \cap A_2\} &= 1/4 , \\ \Pr\{A_1 \cap A_3\} &= 1/4 , \\ \Pr\{A_2 \cap A_3\} &= 1/4 , \\ \Pr\{A_1 \cap A_2 \cap A_3\} &= 0 . \end{aligned}$$

Since for $1 \leq i < j \leq 3$, we have $\Pr\{A_i \cap A_j\} = \Pr\{A_i\} \Pr\{A_j\} = 1/4$, the events A_1, A_2 , and A_3 are pairwise independent. The events are not mutually independent, however, because $\Pr\{A_1 \cap A_2 \cap A_3\} = 0$ and $\Pr\{A_1\} \Pr\{A_2\} \Pr\{A_3\} = 1/8 \neq 0$.

Bayes's theorem

From the definition of conditional probability (C.14) and the commutative law $A \cap B = B \cap A$, it follows that for two events A and B , each with nonzero probability,

$$\begin{aligned}\Pr\{A \cap B\} &= \Pr\{B\} \Pr\{A \mid B\} \\ &= \Pr\{A\} \Pr\{B \mid A\}.\end{aligned}\tag{C.16}$$

Solving for $\Pr\{A \mid B\}$, we obtain

$$\Pr\{A \mid B\} = \frac{\Pr\{A\} \Pr\{B \mid A\}}{\Pr\{B\}},\tag{C.17}$$

which is known as **Bayes's theorem**. The denominator $\Pr\{B\}$ is a normalizing constant, which we can reformulate as follows. Since $B = (B \cap A) \cup (B \cap \bar{A})$, and since $B \cap A$ and $B \cap \bar{A}$ are mutually exclusive events,

$$\begin{aligned}\Pr\{B\} &= \Pr\{B \cap A\} + \Pr\{B \cap \bar{A}\} \\ &= \Pr\{A\} \Pr\{B \mid A\} + \Pr\{\bar{A}\} \Pr\{B \mid \bar{A}\}.\end{aligned}$$

Substituting into equation (C.17), we obtain an equivalent form of Bayes's theorem:

$$\Pr\{A \mid B\} = \frac{\Pr\{A\} \Pr\{B \mid A\}}{\Pr\{A\} \Pr\{B \mid A\} + \Pr\{\bar{A}\} \Pr\{B \mid \bar{A}\}}.\tag{C.18}$$

Bayes's theorem can simplify the computing of conditional probabilities. For example, suppose that we have a fair coin and a biased coin that always comes up heads. We run an experiment consisting of three independent events: we choose one of the two coins at random, we flip that coin once, and then we flip it again. Suppose that the coin we have chosen comes up heads both times. What is the probability that it is biased?

We solve this problem using Bayes's theorem. Let A be the event that we choose the biased coin, and let B be the event that the chosen coin comes up heads both times. We wish to determine $\Pr\{A \mid B\}$. We have $\Pr\{A\} = 1/2$, $\Pr\{B \mid A\} = 1$, $\Pr\{\bar{A}\} = 1/2$, and $\Pr\{B \mid \bar{A}\} = 1/4$; hence,

$$\begin{aligned}\Pr\{A \mid B\} &= \frac{(1/2) \cdot 1}{(1/2) \cdot 1 + (1/2) \cdot (1/4)} \\ &= 4/5.\end{aligned}$$

Exercises**C.2-1**

Professor Rosencrantz flips a fair coin once. Professor Guildenstern flips a fair coin twice. What is the probability that Professor Rosencrantz obtains more heads than Professor Guildenstern?

C.2-2

Prove **Boole's inequality**: For any finite or countably infinite sequence of events $A_1, A_2, \dots$,

$$\Pr\{A_1 \cup A_2 \cup \dots\} \leq \Pr\{A_1\} + \Pr\{A_2\} + \dots \quad (\text{C.19})$$

C.2-3

Suppose we shuffle a deck of 10 cards, each bearing a distinct number from 1 to 10, to mix the cards thoroughly. We then remove three cards, one at a time, from the deck. What is the probability that we select the three cards in sorted (increasing) order?

C.2-4

Prove that

$$\Pr\{A \mid B\} + \Pr\{\bar{A} \mid B\} = 1.$$

C.2-5

Prove that for any collection of events $A_1, A_2, \dots, A_n$,

$$\Pr\{A_1 \cap A_2 \cap \dots \cap A_n\} = \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \cdots \Pr\{A_n \mid A_1 \cap A_2 \cap \dots \cap A_{n-1}\}.$$

C.2-6 ★

Describe a procedure that takes as input two integers a and b such that $0 < a < b$ and, using fair coin flips, produces as output heads with probability a/b and tails with probability $(b - a)/b$. Give a bound on the expected number of coin flips, which should be $O(1)$. (*Hint*: Represent a/b in binary.)

C.2-7 ★

Show how to construct a set of n events that are pairwise independent but such that no subset of $k > 2$ of them is mutually independent.

C.2-8 ★

Two events A and B are **conditionally independent**, given C , if

$$\Pr\{A \cap B \mid C\} = \Pr\{A \mid C\} \cdot \Pr\{B \mid C\}.$$

Give a simple but nontrivial example of two events that are not independent but are conditionally independent given a third event.

C.2-9 ★

You are a contestant in a game show in which a prize is hidden behind one of three curtains. You will win the prize if you select the correct curtain. After you

have picked one curtain but before the curtain is lifted, the emcee lifts one of the other curtains, knowing that it will reveal an empty stage, and asks if you would like to switch from your current selection to the remaining curtain. How would your chances change if you switch? (This question is the celebrated **Monty Hall problem**, named after a game-show host who often presented contestants with just this dilemma.)

C.2-10 ★

A prison warden has randomly picked one prisoner among three to go free. The other two will be executed. The guard knows which one will go free but is forbidden to give any prisoner information regarding his status. Let us call the prisoners X , Y , and Z . Prisoner X asks the guard privately which of Y or Z will be executed, arguing that since he already knows that at least one of them must die, the guard won't be revealing any information about his own status. The guard tells X that Y is to be executed. Prisoner X feels happier now, since he figures that either he or prisoner Z will go free, which means that his probability of going free is now $1/2$. Is he right, or are his chances still $1/3$? Explain.

C.3 Discrete random variables

A (*discrete*) *random variable* X is a function from a finite or countably infinite sample space S to the real numbers. It associates a real number with each possible outcome of an experiment, which allows us to work with the probability distribution induced on the resulting set of numbers. Random variables can also be defined for uncountably infinite sample spaces, but they raise technical issues that are unnecessary to address for our purposes. Henceforth, we shall assume that random variables are discrete.

For a random variable X and a real number x , we define the event $X = x$ to be $\{s \in S : X(s) = x\}$; thus,

$$\Pr\{X = x\} = \sum_{s \in S: X(s)=x} \Pr\{s\}.$$

The function

$$f(x) = \Pr\{X = x\}$$

is the **probability density function** of the random variable X . From the probability axioms, $\Pr\{X = x\} \geq 0$ and $\sum_x \Pr\{X = x\} = 1$.

As an example, consider the experiment of rolling a pair of ordinary, 6-sided dice. There are 36 possible elementary events in the sample space. We assume

that the probability distribution is uniform, so that each elementary event $s \in S$ is equally likely: $\Pr\{s\} = 1/36$. Define the random variable X to be the *maximum* of the two values showing on the dice. We have $\Pr\{X = 3\} = 5/36$, since X assigns a value of 3 to 5 of the 36 possible elementary events, namely, (1, 3), (2, 3), (3, 3), (3, 2), and (3, 1).

We often define several random variables on the same sample space. If X and Y are random variables, the function

$$f(x, y) = \Pr\{X = x \text{ and } Y = y\}$$

is the **joint probability density function** of X and Y . For a fixed value y ,

$$\Pr\{Y = y\} = \sum_x \Pr\{X = x \text{ and } Y = y\} ,$$

and similarly, for a fixed value x ,

$$\Pr\{X = x\} = \sum_y \Pr\{X = x \text{ and } Y = y\} .$$

Using the definition (C.14) of conditional probability, we have

$$\Pr\{X = x \mid Y = y\} = \frac{\Pr\{X = x \text{ and } Y = y\}}{\Pr\{Y = y\}} .$$

We define two random variables X and Y to be **independent** if for all x and y , the events $X = x$ and $Y = y$ are independent or, equivalently, if for all x and y , we have $\Pr\{X = x \text{ and } Y = y\} = \Pr\{X = x\} \Pr\{Y = y\}$.

Given a set of random variables defined over the same sample space, we can define new random variables as sums, products, or other functions of the original variables.

Expected value of a random variable

The simplest and most useful summary of the distribution of a random variable is the “average” of the values it takes on. The **expected value** (or, synonymously, **expectation** or **mean**) of a discrete random variable X is

$$E[X] = \sum_x x \cdot \Pr\{X = x\} , \tag{C.20}$$

which is well defined if the sum is finite or converges absolutely. Sometimes the expectation of X is denoted by μ_X or, when the random variable is apparent from context, simply by μ .

Consider a game in which you flip two fair coins. You earn \$3 for each head but lose \$2 for each tail. The expected value of the random variable X representing

your earnings is

$$\begin{aligned} E[X] &= 6 \cdot \Pr\{2 \text{ H's}\} + 1 \cdot \Pr\{1 \text{ H, } 1 \text{ T}\} - 4 \cdot \Pr\{2 \text{ T's}\} \\ &= 6(1/4) + 1(1/2) - 4(1/4) \\ &= 1. \end{aligned}$$

The expectation of the sum of two random variables is the sum of their expectations, that is,

$$E[X + Y] = E[X] + E[Y], \quad (\text{C.21})$$

whenever $E[X]$ and $E[Y]$ are defined. We call this property **linearity of expectation**, and it holds even if X and Y are not independent. It also extends to finite and absolutely convergent summations of expectations. Linearity of expectation is the key property that enables us to perform probabilistic analyses by using indicator random variables (see Section 5.2).

If X is any random variable, any function $g(x)$ defines a new random variable $g(X)$. If the expectation of $g(X)$ is defined, then

$$E[g(X)] = \sum_x g(x) \cdot \Pr\{X = x\}.$$

Letting $g(x) = ax$, we have for any constant a ,

$$E[aX] = aE[X]. \quad (\text{C.22})$$

Consequently, expectations are linear: for any two random variables X and Y and any constant a ,

$$E[aX + Y] = aE[X] + E[Y]. \quad (\text{C.23})$$

When two random variables X and Y are independent and each has a defined expectation,

$$\begin{aligned} E[XY] &= \sum_x \sum_y xy \cdot \Pr\{X = x \text{ and } Y = y\} \\ &= \sum_x \sum_y xy \cdot \Pr\{X = x\} \Pr\{Y = y\} \\ &= \left(\sum_x x \cdot \Pr\{X = x\} \right) \left(\sum_y y \cdot \Pr\{Y = y\} \right) \\ &= E[X]E[Y]. \end{aligned}$$

In general, when n random variables $X_1, X_2, \dots, X_n$ are mutually independent,

$$E[X_1 X_2 \cdots X_n] = E[X_1]E[X_2] \cdots E[X_n]. \quad (\text{C.24})$$

When a random variable X takes on values from the set of natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$, we have a nice formula for its expectation:

$$\begin{aligned}
 E[X] &= \sum_{i=0}^{\infty} i \cdot \Pr\{X = i\} \\
 &= \sum_{i=0}^{\infty} i (\Pr\{X \geq i\} - \Pr\{X \geq i+1\}) \\
 &= \sum_{i=1}^{\infty} \Pr\{X \geq i\} ,
 \end{aligned} \tag{C.25}$$

since each term $\Pr\{X \geq i\}$ is added in i times and subtracted out $i-1$ times (except $\Pr\{X \geq 0\}$, which is added in 0 times and not subtracted out at all).

When we apply a convex function $f(x)$ to a random variable X , **Jensen's inequality** gives us

$$E[f(X)] \geq f(E[X]) , \tag{C.26}$$

provided that the expectations exist and are finite. (A function $f(x)$ is **convex** if for all x and y and for all $0 \leq \lambda \leq 1$, we have $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$.)

Variance and standard deviation

The expected value of a random variable does not tell us how “spread out” the variable’s values are. For example, if we have random variables X and Y for which $\Pr\{X = 1/4\} = \Pr\{X = 3/4\} = 1/2$ and $\Pr\{Y = 0\} = \Pr\{Y = 1\} = 1/2$, then both $E[X]$ and $E[Y]$ are $1/2$, yet the actual values taken on by Y are farther from the mean than the actual values taken on by X .

The notion of variance mathematically expresses how far from the mean a random variable’s values are likely to be. The **variance** of a random variable X with mean $E[X]$ is

$$\begin{aligned}
 \text{Var}[X] &= E[(X - E[X])^2] \\
 &= E[X^2 - 2XE[X] + E^2[X]] \\
 &= E[X^2] - 2E[XE[X]] + E^2[X] \\
 &= E[X^2] - 2E^2[X] + E^2[X] \\
 &= E[X^2] - E^2[X] .
 \end{aligned} \tag{C.27}$$

To justify the equality $E[E^2[X]] = E^2[X]$, note that because $E[X]$ is a real number and not a random variable, so is $E^2[X]$. The equality $E[XE[X]] = E^2[X]$

follows from equation (C.22), with $a = E[X]$. Rewriting equation (C.27) yields an expression for the expectation of the square of a random variable:

$$E[X^2] = \text{Var}[X] + E^2[X] . \quad (\text{C.28})$$

The variance of a random variable X and the variance of aX are related (see Exercise C.3-10):

$$\text{Var}[aX] = a^2 \text{Var}[X] .$$

When X and Y are independent random variables,

$$\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y] .$$

In general, if n random variables $X_1, X_2, \dots, X_n$ are pairwise independent, then

$$\text{Var}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \text{Var}[X_i] . \quad (\text{C.29})$$

The **standard deviation** of a random variable X is the nonnegative square root of the variance of X . The standard deviation of a random variable X is sometimes denoted σ_X or simply σ when the random variable X is understood from context. With this notation, the variance of X is denoted σ^2 .

Exercises

C.3-1

Suppose we roll two ordinary, 6-sided dice. What is the expectation of the sum of the two values showing? What is the expectation of the maximum of the two values showing?

C.3-2

An array $A[1..n]$ contains n distinct numbers that are randomly ordered, with each permutation of the n numbers being equally likely. What is the expectation of the index of the maximum element in the array? What is the expectation of the index of the minimum element in the array?

C.3-3

A carnival game consists of three dice in a cage. A player can bet a dollar on any of the numbers 1 through 6. The cage is shaken, and the payoff is as follows. If the player's number doesn't appear on any of the dice, he loses his dollar. Otherwise, if his number appears on exactly k of the three dice, for $k = 1, 2, 3$, he keeps his dollar and wins k more dollars. What is his expected gain from playing the carnival game once?

C.3-4

Argue that if X and Y are nonnegative random variables, then

$$E[\max(X, Y)] \leq E[X] + E[Y] .$$

C.3-5 ★

Let X and Y be independent random variables. Prove that $f(X)$ and $g(Y)$ are independent for any choice of functions f and g .

C.3-6 ★

Let X be a nonnegative random variable, and suppose that $E[X]$ is well defined. Prove **Markov's inequality**:

$$\Pr\{X \geq t\} \leq E[X] / t \tag{C.30}$$

for all $t > 0$.

C.3-7 ★

Let S be a sample space, and let X and X' be random variables such that $X(s) \geq X'(s)$ for all $s \in S$. Prove that for any real constant t ,

$$\Pr\{X \geq t\} \geq \Pr\{X' \geq t\} .$$

C.3-8

Which is larger: the expectation of the square of a random variable, or the square of its expectation?

C.3-9

Show that for any random variable X that takes on only the values 0 and 1, we have $\text{Var}[X] = E[X]E[1 - X]$.

C.3-10

Prove that $\text{Var}[aX] = a^2\text{Var}[X]$ from the definition (C.27) of variance.

C.4 The geometric and binomial distributions

We can think of a coin flip as an instance of a **Bernoulli trial**, which is an experiment with only two possible outcomes: **success**, which occurs with probability p , and **failure**, which occurs with probability $q = 1 - p$. When we speak of **Bernoulli trials** collectively, we mean that the trials are mutually independent and, unless we specifically say otherwise, that each has the same probability p for success. Two

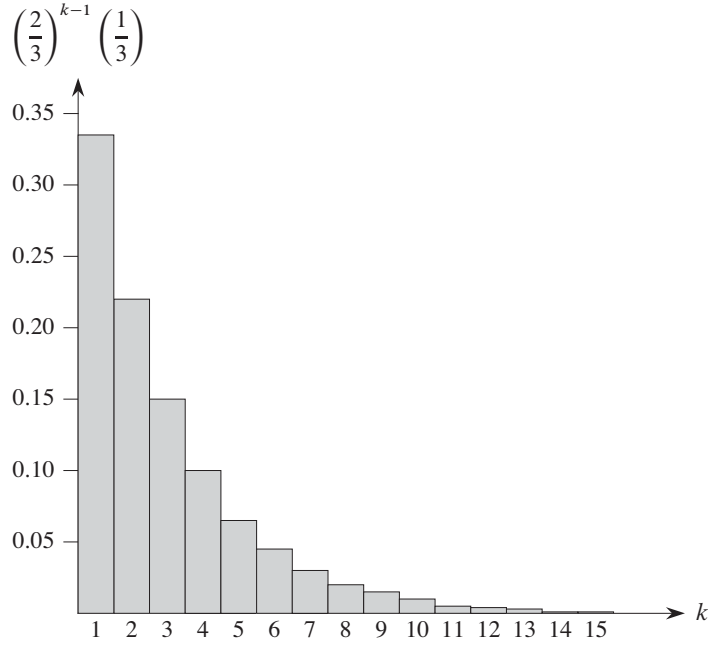


Figure C.1 A geometric distribution with probability $p = 1/3$ of success and a probability $q = 1 - p$ of failure. The expectation of the distribution is $1/p = 3$.

important distributions arise from Bernoulli trials: the geometric distribution and the binomial distribution.

The geometric distribution

Suppose we have a sequence of Bernoulli trials, each with a probability p of success and a probability $q = 1 - p$ of failure. How many trials occur before we obtain a success? Let us define the random variable X be the number of trials needed to obtain a success. Then X has values in the range $\{1, 2, \dots\}$, and for $k \geq 1$,

$$\Pr\{X = k\} = q^{k-1} p, \quad (\text{C.31})$$

since we have $k - 1$ failures before the one success. A probability distribution satisfying equation (C.31) is said to be a **geometric distribution**. Figure C.1 illustrates such a distribution.

Assuming that $q < 1$, we can calculate the expectation of a geometric distribution using identity (A.8):

$$\begin{aligned}
 E[X] &= \sum_{k=1}^{\infty} k q^{k-1} p \\
 &= \frac{p}{q} \sum_{k=0}^{\infty} k q^k \\
 &= \frac{p}{q} \cdot \frac{q}{(1-q)^2} \\
 &= \frac{p}{q} \cdot \frac{q}{p^2} \\
 &= 1/p.
 \end{aligned} \tag{C.32}$$

Thus, on average, it takes $1/p$ trials before we obtain a success, an intuitive result. The variance, which can be calculated similarly, but using Exercise A.1-3, is

$$\text{Var}[X] = q/p^2. \tag{C.33}$$

As an example, suppose we repeatedly roll two dice until we obtain either a seven or an eleven. Of the 36 possible outcomes, 6 yield a seven and 2 yield an eleven. Thus, the probability of success is $p = 8/36 = 2/9$, and we must roll $1/p = 9/2 = 4.5$ times on average to obtain a seven or eleven.

The binomial distribution

How many successes occur during n Bernoulli trials, where a success occurs with probability p and a failure with probability $q = 1 - p$? Define the random variable X to be the number of successes in n trials. Then X has values in the range $\{0, 1, \dots, n\}$, and for $k = 0, 1, \dots, n$,

$$\Pr\{X = k\} = \binom{n}{k} p^k q^{n-k}, \tag{C.34}$$

since there are $\binom{n}{k}$ ways to pick which k of the n trials are successes, and the probability that each occurs is $p^k q^{n-k}$. A probability distribution satisfying equation (C.34) is said to be a **binomial distribution**. For convenience, we define the family of binomial distributions using the notation

$$b(k; n, p) = \binom{n}{k} p^k (1-p)^{n-k}. \tag{C.35}$$

Figure C.2 illustrates a binomial distribution. The name “binomial” comes from the right-hand side of equation (C.34) being the k th term of the expansion of $(p + q)^n$. Consequently, since $p + q = 1$,

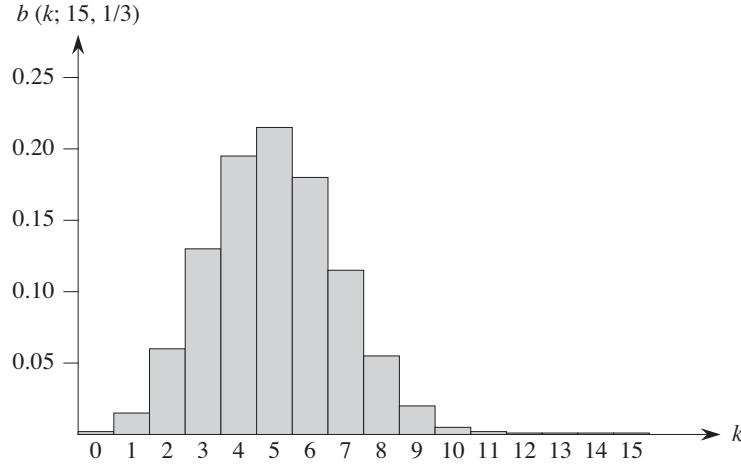


Figure C.2 The binomial distribution $b(k; 15, 1/3)$ resulting from $n = 15$ Bernoulli trials, each with probability $p = 1/3$ of success. The expectation of the distribution is $np = 5$.

$$\sum_{k=0}^n b(k; n, p) = 1, \quad (\text{C.36})$$

as axiom 2 of the probability axioms requires.

We can compute the expectation of a random variable having a binomial distribution from equations (C.8) and (C.36). Let X be a random variable that follows the binomial distribution $b(k; n, p)$, and let $q = 1 - p$. By the definition of expectation, we have

$$\begin{aligned} E[X] &= \sum_{k=0}^n k \cdot \Pr\{X = k\} \\ &= \sum_{k=0}^n k \cdot b(k; n, p) \\ &= \sum_{k=1}^n k \binom{n}{k} p^k q^{n-k} \\ &= np \sum_{k=1}^n \binom{n-1}{k-1} p^{k-1} q^{n-k} \quad (\text{by equation (C.8)}) \\ &= np \sum_{k=0}^{n-1} \binom{n-1}{k} p^k q^{(n-1)-k} \end{aligned}$$

$$\begin{aligned}
&= np \sum_{k=0}^{n-1} b(k; n-1, p) \\
&= np \quad (\text{by equation (C.36)}) .
\end{aligned} \tag{C.37}$$

By using the linearity of expectation, we can obtain the same result with substantially less algebra. Let X_i be the random variable describing the number of successes in the i th trial. Then $E[X_i] = p \cdot 1 + q \cdot 0 = p$, and by linearity of expectation (equation (C.21)), the expected number of successes for n trials is

$$\begin{aligned}
E[X] &= E\left[\sum_{i=1}^n X_i\right] \\
&= \sum_{i=1}^n E[X_i] \\
&= \sum_{i=1}^n p \\
&= np .
\end{aligned} \tag{C.38}$$

We can use the same approach to calculate the variance of the distribution. Using equation (C.27), we have $\text{Var}[X_i] = E[X_i^2] - E^2[X_i]$. Since X_i only takes on the values 0 and 1, we have $X_i^2 = X_i$, which implies $E[X_i^2] = E[X_i] = p$. Hence,

$$\text{Var}[X_i] = p - p^2 = p(1 - p) = pq . \tag{C.39}$$

To compute the variance of X , we take advantage of the independence of the n trials; thus, by equation (C.29),

$$\begin{aligned}
\text{Var}[X] &= \text{Var}\left[\sum_{i=1}^n X_i\right] \\
&= \sum_{i=1}^n \text{Var}[X_i] \\
&= \sum_{i=1}^n pq \\
&= npq .
\end{aligned} \tag{C.40}$$

As Figure C.2 shows, the binomial distribution $b(k; n, p)$ increases with k until it reaches the mean np , and then it decreases. We can prove that the distribution always behaves in this manner by looking at the ratio of successive terms:

$$\begin{aligned}
\frac{b(k; n, p)}{b(k-1; n, p)} &= \frac{\binom{n}{k} p^k q^{n-k}}{\binom{n}{k-1} p^{k-1} q^{n-k+1}} \\
&= \frac{n!(k-1)!(n-k+1)!p}{k!(n-k)!n!q} \\
&= \frac{(n-k+1)p}{kq} \\
&= 1 + \frac{(n+1)p-k}{kq}.
\end{aligned} \tag{C.41}$$

This ratio is greater than 1 precisely when $(n+1)p - k$ is positive. Consequently, $b(k; n, p) > b(k-1; n, p)$ for $k < (n+1)p$ (the distribution increases), and $b(k; n, p) < b(k-1; n, p)$ for $k > (n+1)p$ (the distribution decreases). If $k = (n+1)p$ is an integer, then $b(k; n, p) = b(k-1; n, p)$, and so the distribution then has two maxima: at $k = (n+1)p$ and at $k-1 = (n+1)p-1 = np - q$. Otherwise, it attains a maximum at the unique integer k that lies in the range $np - q < k < (n+1)p$.

The following lemma provides an upper bound on the binomial distribution.

Lemma C.1

Let $n \geq 0$, let $0 < p < 1$, let $q = 1 - p$, and let $0 \leq k \leq n$. Then

$$b(k; n, p) \leq \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}.$$

Proof Using equation (C.6), we have

$$\begin{aligned}
b(k; n, p) &= \binom{n}{k} p^k q^{n-k} \\
&\leq \left(\frac{n}{k}\right)^k \left(\frac{n}{n-k}\right)^{n-k} p^k q^{n-k} \\
&= \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}.
\end{aligned} \quad \blacksquare$$

Exercises

C.4-1

Verify axiom 2 of the probability axioms for the geometric distribution.

C.4-2

How many times on average must we flip 6 fair coins before we obtain 3 heads and 3 tails?

C.4-3

Show that $b(k; n, p) = b(n - k; n, q)$, where $q = 1 - p$.

C.4-4

Show that value of the maximum of the binomial distribution $b(k; n, p)$ is approximately $1/\sqrt{2\pi npq}$, where $q = 1 - p$.

C.4-5 ★

Show that the probability of no successes in n Bernoulli trials, each with probability $p = 1/n$, is approximately $1/e$. Show that the probability of exactly one success is also approximately $1/e$.

C.4-6 ★

Professor Rosencrantz flips a fair coin n times, and so does Professor Guildenstern. Show that the probability that they get the same number of heads is $\binom{2n}{n}/4^n$. (*Hint:* For Professor Rosencrantz, call a head a success; for Professor Guildenstern, call a tail a success.) Use your argument to verify the identity

$$\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}.$$

C.4-7 ★

Show that for $0 \leq k \leq n$,

$$b(k; n, 1/2) \leq 2^{n H(k/n) - n},$$

where $H(x)$ is the entropy function (C.7).

C.4-8 ★

Consider n Bernoulli trials, where for $i = 1, 2, \dots, n$, the i th trial has probability p_i of success, and let X be the random variable denoting the total number of successes. Let $p \geq p_i$ for all $i = 1, 2, \dots, n$. Prove that for $1 \leq k \leq n$,

$$\Pr\{X < k\} \geq \sum_{i=0}^{k-1} b(i; n, p).$$

C.4-9 ★

Let X be the random variable for the total number of successes in a set A of n Bernoulli trials, where the i th trial has a probability p_i of success, and let X' be the random variable for the total number of successes in a second set A' of n Bernoulli trials, where the i th trial has a probability $p'_i \geq p_i$ of success. Prove that for $0 \leq k \leq n$,

$$\Pr\{X' \geq k\} \geq \Pr\{X \geq k\}.$$

(Hint: Show how to obtain the Bernoulli trials in A' by an experiment involving the trials of A , and use the result of Exercise C.3-7.)

★ C.5 The tails of the binomial distribution

The probability of having at least, or at most, k successes in n Bernoulli trials, each with probability p of success, is often of more interest than the probability of having exactly k successes. In this section, we investigate the *tails* of the binomial distribution: the two regions of the distribution $b(k; n, p)$ that are far from the mean np . We shall prove several important bounds on (the sum of all terms in) a tail.

We first provide a bound on the right tail of the distribution $b(k; n, p)$. We can determine bounds on the left tail by inverting the roles of successes and failures.

Theorem C.2

Consider a sequence of n Bernoulli trials, where success occurs with probability p . Let X be the random variable denoting the total number of successes. Then for $0 \leq k \leq n$, the probability of at least k successes is

$$\begin{aligned} \Pr\{X \geq k\} &= \sum_{i=k}^n b(i; n, p) \\ &\leq \binom{n}{k} p^k. \end{aligned}$$

Proof For $S \subseteq \{1, 2, \dots, n\}$, we let A_S denote the event that the i th trial is a success for every $i \in S$. Clearly $\Pr\{A_S\} = p^{|S|}$ if $|S| = k$. We have

$$\begin{aligned} \Pr\{X \geq k\} &= \Pr\{\text{there exists } S \subseteq \{1, 2, \dots, n\} : |S| = k \text{ and } A_S\} \\ &= \Pr\left\{\bigcup_{S \subseteq \{1, 2, \dots, n\} : |S|=k} A_S\right\} \\ &\leq \sum_{S \subseteq \{1, 2, \dots, n\} : |S|=k} \Pr\{A_S\} \quad (\text{by inequality (C.19)}) \\ &= \binom{n}{k} p^k. \end{aligned}$$

■

The following corollary restates the theorem for the left tail of the binomial distribution. In general, we shall leave it to you to adapt the proofs from one tail to the other.

Corollary C.3

Consider a sequence of n Bernoulli trials, where success occurs with probability p . If X is the random variable denoting the total number of successes, then for $0 \leq k \leq n$, the probability of at most k successes is

$$\begin{aligned} \Pr\{X \leq k\} &= \sum_{i=0}^k b(i; n, p) \\ &\leq \binom{n}{n-k} (1-p)^{n-k} \\ &= \binom{n}{k} (1-p)^{n-k}. \quad \blacksquare \end{aligned}$$

Our next bound concerns the left tail of the binomial distribution. Its corollary shows that, far from the mean, the left tail diminishes exponentially.

Theorem C.4

Consider a sequence of n Bernoulli trials, where success occurs with probability p and failure with probability $q = 1 - p$. Let X be the random variable denoting the total number of successes. Then for $0 < k < np$, the probability of fewer than k successes is

$$\begin{aligned} \Pr\{X < k\} &= \sum_{i=0}^{k-1} b(i; n, p) \\ &< \frac{kq}{np - k} b(k; n, p). \end{aligned}$$

Proof We bound the series $\sum_{i=0}^{k-1} b(i; n, p)$ by a geometric series using the technique from Section A.2, page 1151. For $i = 1, 2, \dots, k$, we have from equation (C.41),

$$\begin{aligned} \frac{b(i-1; n, p)}{b(i; n, p)} &= \frac{iq}{(n-i+1)p} \\ &< \frac{iq}{(n-i)p} \\ &\leq \frac{kq}{(n-k)p}. \end{aligned}$$

If we let

$$\begin{aligned}
 x &= \frac{kq}{(n-k)p} \\
 &< \frac{kq}{(n-np)p} \\
 &= \frac{kq}{nqp} \\
 &= \frac{k}{np} \\
 &< 1,
 \end{aligned}$$

it follows that

$$b(i-1; n, p) < x b(i; n, p)$$

for $0 < i \leq k$. Iteratively applying this inequality $k-i$ times, we obtain

$$b(i; n, p) < x^{k-i} b(k; n, p)$$

for $0 \leq i < k$, and hence

$$\begin{aligned}
 \sum_{i=0}^{k-1} b(i; n, p) &< \sum_{i=0}^{k-1} x^{k-i} b(k; n, p) \\
 &< b(k; n, p) \sum_{i=0}^{\infty} x^i \\
 &= \frac{x}{1-x} b(k; n, p) \\
 &= \frac{kq}{np-k} b(k; n, p). \quad \blacksquare
 \end{aligned}$$

Corollary C.5

Consider a sequence of n Bernoulli trials, where success occurs with probability p and failure with probability $q = 1 - p$. Then for $0 < k \leq np/2$, the probability of fewer than k successes is less than one half of the probability of fewer than $k+1$ successes.

Proof Because $k \leq np/2$, we have

$$\frac{kq}{np-k} \leq \frac{(np/2)q}{np-(np/2)}$$

$$\begin{aligned}
&= \frac{(np/2)q}{np/2} \\
&\leq 1,
\end{aligned} \tag{C.42}$$

since $q \leq 1$. Letting X be the random variable denoting the number of successes, Theorem C.4 and inequality (C.42) imply that the probability of fewer than k successes is

$$\Pr\{X < k\} = \sum_{i=0}^{k-1} b(i; n, p) < b(k; n, p).$$

Thus we have

$$\begin{aligned}
\frac{\Pr\{X < k\}}{\Pr\{X < k+1\}} &= \frac{\sum_{i=0}^{k-1} b(i; n, p)}{\sum_{i=0}^k b(i; n, p)} \\
&= \frac{\sum_{i=0}^{k-1} b(i; n, p)}{\sum_{i=0}^{k-1} b(i; n, p) + b(k; n, p)} \\
&< 1/2,
\end{aligned}$$

since $\sum_{i=0}^{k-1} b(i; n, p) < b(k; n, p)$. ■

Bounds on the right tail follow similarly. Exercise C.5-2 asks you to prove them.

Corollary C.6

Consider a sequence of n Bernoulli trials, where success occurs with probability p . Let X be the random variable denoting the total number of successes. Then for $np < k < n$, the probability of more than k successes is

$$\begin{aligned}
\Pr\{X > k\} &= \sum_{i=k+1}^n b(i; n, p) \\
&< \frac{(n-k)p}{k-np} b(k; n, p).
\end{aligned} \quad \blacksquare$$

Corollary C.7

Consider a sequence of n Bernoulli trials, where success occurs with probability p and failure with probability $q = 1 - p$. Then for $(np + n)/2 < k < n$, the probability of more than k successes is less than one half of the probability of more than $k - 1$ successes. ■

The next theorem considers n Bernoulli trials, each with a probability p_i of success, for $i = 1, 2, \dots, n$. As the subsequent corollary shows, we can use the

theorem to provide a bound on the right tail of the binomial distribution by setting $p_i = p$ for each trial.

Theorem C.8

Consider a sequence of n Bernoulli trials, where in the i th trial, for $i = 1, 2, \dots, n$, success occurs with probability p_i and failure occurs with probability $q_i = 1 - p_i$. Let X be the random variable describing the total number of successes, and let $\mu = E[X]$. Then for $r > \mu$,

$$\Pr\{X - \mu \geq r\} \leq \left(\frac{\mu e}{r}\right)^r.$$

Proof Since for any $\alpha > 0$, the function $e^{\alpha x}$ is strictly increasing in x ,

$$\Pr\{X - \mu \geq r\} = \Pr\{e^{\alpha(X-\mu)} \geq e^{\alpha r}\}, \quad (\text{C.43})$$

where we will determine α later. Using Markov's inequality (C.30), we obtain

$$\Pr\{e^{\alpha(X-\mu)} \geq e^{\alpha r}\} \leq E[e^{\alpha(X-\mu)}] e^{-\alpha r}. \quad (\text{C.44})$$

The bulk of the proof consists of bounding $E[e^{\alpha(X-\mu)}]$ and substituting a suitable value for α in inequality (C.44). First, we evaluate $E[e^{\alpha(X-\mu)}]$. Using the technique of indicator random variables (see Section 5.2), let $X_i = I\{\text{the } i\text{th Bernoulli trial is a success}\}$ for $i = 1, 2, \dots, n$; that is, X_i is the random variable that is 1 if the i th Bernoulli trial is a success and 0 if it is a failure. Thus,

$$X = \sum_{i=1}^n X_i,$$

and by linearity of expectation,

$$\mu = E[X] = E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n p_i,$$

which implies

$$X - \mu = \sum_{i=1}^n (X_i - p_i).$$

To evaluate $E[e^{\alpha(X-\mu)}]$, we substitute for $X - \mu$, obtaining

$$\begin{aligned} E[e^{\alpha(X-\mu)}] &= E[e^{\alpha \sum_{i=1}^n (X_i - p_i)}] \\ &= E\left[\prod_{i=1}^n e^{\alpha(X_i - p_i)}\right] \\ &= \prod_{i=1}^n E[e^{\alpha(X_i - p_i)}], \end{aligned}$$

which follows from (C.24), since the mutual independence of the random variables X_i implies the mutual independence of the random variables $e^{\alpha(X_i - p_i)}$ (see Exercise C.3-5). By the definition of expectation,

$$\begin{aligned} \mathbb{E}[e^{\alpha(X_i - p_i)}] &= e^{\alpha(1-p_i)} p_i + e^{\alpha(0-p_i)} q_i \\ &= p_i e^{\alpha q_i} + q_i e^{-\alpha p_i} \\ &\leq p_i e^{\alpha} + 1 \\ &\leq \exp(p_i e^{\alpha}), \end{aligned} \tag{C.45}$$

where $\exp(x)$ denotes the exponential function: $\exp(x) = e^x$. (Inequality (C.45) follows from the inequalities $\alpha > 0$, $q_i \leq 1$, $e^{\alpha q_i} \leq e^{\alpha}$, and $e^{-\alpha p_i} \leq 1$, and the last line follows from inequality (3.12).) Consequently,

$$\begin{aligned} \mathbb{E}[e^{\alpha(X - \mu)}] &= \prod_{i=1}^n \mathbb{E}[e^{\alpha(X_i - p_i)}] \\ &\leq \prod_{i=1}^n \exp(p_i e^{\alpha}) \\ &= \exp\left(\sum_{i=1}^n p_i e^{\alpha}\right) \\ &= \exp(\mu e^{\alpha}), \end{aligned} \tag{C.46}$$

since $\mu = \sum_{i=1}^n p_i$. Therefore, from equation (C.43) and inequalities (C.44) and (C.46), it follows that

$$\Pr\{X - \mu \geq r\} \leq \exp(\mu e^{\alpha} - \alpha r). \tag{C.47}$$

Choosing $\alpha = \ln(r/\mu)$ (see Exercise C.5-7), we obtain

$$\begin{aligned} \Pr\{X - \mu \geq r\} &\leq \exp(\mu e^{\ln(r/\mu)} - r \ln(r/\mu)) \\ &= \exp(r - r \ln(r/\mu)) \\ &= \frac{e^r}{(r/\mu)^r} \\ &= \left(\frac{\mu e}{r}\right)^r. \end{aligned} \quad \blacksquare$$

When applied to Bernoulli trials in which each trial has the same probability of success, Theorem C.8 yields the following corollary bounding the right tail of a binomial distribution.

Corollary C.9

Consider a sequence of n Bernoulli trials, where in each trial success occurs with probability p and failure occurs with probability $q = 1 - p$. Then for $r > np$,

$$\begin{aligned} \Pr\{X - np \geq r\} &= \sum_{k=\lceil np+r \rceil}^n b(k; n, p) \\ &\leq \left(\frac{npe}{r}\right)^r. \end{aligned}$$

Proof By equation (C.37), we have $\mu = E[X] = np$. ■

Exercises**C.5-1 ★**

Which is less likely: obtaining no heads when you flip a fair coin n times, or obtaining fewer than n heads when you flip the coin $4n$ times?

C.5-2 ★

Prove Corollaries C.6 and C.7.

C.5-3 ★

Show that

$$\sum_{i=0}^{k-1} \binom{n}{i} a^i < (a+1)^n \frac{k}{na - k(a+1)} b(k; n, a/(a+1))$$

for all $a > 0$ and all k such that $0 < k < na/(a+1)$.

C.5-4 ★

Prove that if $0 < k < np$, where $0 < p < 1$ and $q = 1 - p$, then

$$\sum_{i=0}^{k-1} p^i q^{n-i} < \frac{kq}{np - k} \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}.$$

C.5-5 ★

Show that the conditions of Theorem C.8 imply that

$$\Pr\{\mu - X \geq r\} \leq \left(\frac{(n - \mu)e}{r}\right)^r.$$

Similarly, show that the conditions of Corollary C.9 imply that

$$\Pr\{np - X \geq r\} \leq \left(\frac{nqe}{r}\right)^r.$$

C.5-6 ★

Consider a sequence of n Bernoulli trials, where in the i th trial, for $i = 1, 2, \dots, n$, success occurs with probability p_i and failure occurs with probability $q_i = 1 - p_i$. Let X be the random variable describing the total number of successes, and let $\mu = E[X]$. Show that for $r \geq 0$,

$$\Pr\{X - \mu \geq r\} \leq e^{-r^2/2n}.$$

(Hint: Prove that $p_i e^{\alpha q_i} + q_i e^{-\alpha p_i} \leq e^{\alpha^2/2}$. Then follow the outline of the proof of Theorem C.8, using this inequality in place of inequality (C.45).)

C.5-7 ★

Show that choosing $\alpha = \ln(r/\mu)$ minimizes the right-hand side of inequality (C.47).

Problems
C-1 Balls and bins

In this problem, we investigate the effect of various assumptions on the number of ways of placing n balls into b distinct bins.

- a. Suppose that the n balls are distinct and that their order within a bin does not matter. Argue that the number of ways of placing the balls in the bins is b^n .
- b. Suppose that the balls are distinct and that the balls in each bin are ordered. Prove that there are exactly $(b + n - 1)!/(b - 1)!$ ways to place the balls in the bins. (Hint: Consider the number of ways of arranging n distinct balls and $b - 1$ indistinguishable sticks in a row.)
- c. Suppose that the balls are identical, and hence their order within a bin does not matter. Show that the number of ways of placing the balls in the bins is $\binom{b+n-1}{n}$. (Hint: Of the arrangements in part (b), how many are repeated if the balls are made identical?)
- d. Suppose that the balls are identical and that no bin may contain more than one ball, so that $n \leq b$. Show that the number of ways of placing the balls is $\binom{b}{n}$.
- e. Suppose that the balls are identical and that no bin may be left empty. Assuming that $n \geq b$, show that the number of ways of placing the balls is $\binom{n-1}{b-1}$.

Appendix notes

The first general methods for solving probability problems were discussed in a famous correspondence between B. Pascal and P. de Fermat, which began in 1654, and in a book by C. Huygens in 1657. Rigorous probability theory began with the work of J. Bernoulli in 1713 and A. De Moivre in 1730. Further developments of the theory were provided by P.-S. Laplace, S.-D. Poisson, and C. F. Gauss.

Sums of random variables were originally studied by P. L. Chebyshev and A. A. Markov. A. N. Kolmogorov axiomatized probability theory in 1933. Chernoff [66] and Hoeffding [173] provided bounds on the tails of distributions. Seminal work in random combinatorial structures was done by P. Erdős.

Knuth [209] and Liu [237] are good references for elementary combinatorics and counting. Standard textbooks such as Billingsley [46], Chung [67], Drake [95], Feller [104], and Rozanov [300] offer comprehensive introductions to probability.

D Matrices

Matrices arise in numerous applications, including, but by no means limited to, scientific computing. If you have seen matrices before, much of the material in this appendix will be familiar to you, but some of it might be new. Section D.1 covers basic matrix definitions and operations, and Section D.2 presents some basic matrix properties.

D.1 Matrices and matrix operations

In this section, we review some basic concepts of matrix theory and some fundamental properties of matrices.

Matrices and vectors

A **matrix** is a rectangular array of numbers. For example,

$$\begin{aligned} A &= \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \end{aligned} \tag{D.1}$$

is a 2×3 matrix $A = (a_{ij})$, where for $i = 1, 2$ and $j = 1, 2, 3$, we denote the element of the matrix in row i and column j by a_{ij} . We use uppercase letters to denote matrices and corresponding subscripted lowercase letters to denote their elements. We denote the set of all $m \times n$ matrices with real-valued entries by $\mathbb{R}^{m \times n}$ and, in general, the set of $m \times n$ matrices with entries drawn from a set S by $S^{m \times n}$.

The **transpose** of a matrix A is the matrix A^T obtained by exchanging the rows and columns of A . For the matrix A of equation (D.1),

$$A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}.$$

A **vector** is a one-dimensional array of numbers. For example,

$$x = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

is a vector of size 3. We sometimes call a vector of length n an **n -vector**. We use lowercase letters to denote vectors, and we denote the i th element of a size- n vector x by x_i , for $i = 1, 2, \dots, n$. We take the standard form of a vector to be as a **column vector** equivalent to an $n \times 1$ matrix; the corresponding **row vector** is obtained by taking the transpose:

$$x^T = (2 \ 3 \ 5).$$

The **unit vector** e_i is the vector whose i th element is 1 and all of whose other elements are 0. Usually, the size of a unit vector is clear from the context.

A **zero matrix** is a matrix all of whose entries are 0. Such a matrix is often denoted 0, since the ambiguity between the number 0 and a matrix of 0s is usually easily resolved from context. If a matrix of 0s is intended, then the size of the matrix also needs to be derived from the context.

Square matrices

Square $n \times n$ matrices arise frequently. Several special cases of square matrices are of particular interest:

1. A **diagonal matrix** has $a_{ij} = 0$ whenever $i \neq j$. Because all of the off-diagonal elements are zero, we can specify the matrix by listing the elements along the diagonal:

$$\text{diag}(a_{11}, a_{22}, \dots, a_{nn}) = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{pmatrix}.$$

2. The $n \times n$ **identity matrix** I_n is a diagonal matrix with 1s along the diagonal:

$$\begin{aligned} I_n &= \text{diag}(1, 1, \dots, 1) \\ &= \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix}. \end{aligned}$$

When I appears without a subscript, we derive its size from the context. The i th column of an identity matrix is the unit vector e_i .

3. A **tridiagonal matrix** T is one for which $t_{ij} = 0$ if $|i - j| > 1$. Nonzero entries appear only on the main diagonal, immediately above the main diagonal ($t_{i,i+1}$ for $i = 1, 2, \dots, n - 1$), or immediately below the main diagonal ($t_{i+1,i}$ for $i = 1, 2, \dots, n - 1$):

$$T = \begin{pmatrix} t_{11} & t_{12} & 0 & 0 & \dots & 0 & 0 & 0 \\ t_{21} & t_{22} & t_{23} & 0 & \dots & 0 & 0 & 0 \\ 0 & t_{32} & t_{33} & t_{34} & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & t_{n-2,n-2} & t_{n-2,n-1} & 0 \\ 0 & 0 & 0 & 0 & \dots & t_{n-1,n-2} & t_{n-1,n-1} & t_{n-1,n} \\ 0 & 0 & 0 & 0 & \dots & 0 & t_{n,n-1} & t_{nn} \end{pmatrix}.$$

4. An **upper-triangular matrix** U is one for which $u_{ij} = 0$ if $i > j$. All entries below the diagonal are zero:

$$U = \begin{pmatrix} u_{11} & u_{12} & \dots & u_{1n} \\ 0 & u_{22} & \dots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & u_{nn} \end{pmatrix}.$$

An upper-triangular matrix is **unit upper-triangular** if it has all 1s along the diagonal.

5. A **lower-triangular matrix** L is one for which $l_{ij} = 0$ if $i < j$. All entries above the diagonal are zero:

$$L = \begin{pmatrix} l_{11} & 0 & \dots & 0 \\ l_{21} & l_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \dots & l_{nn} \end{pmatrix}.$$

A lower-triangular matrix is **unit lower-triangular** if it has all 1s along the diagonal.

6. A **permutation matrix** P has exactly one 1 in each row or column, and 0s elsewhere. An example of a permutation matrix is

$$P = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

Such a matrix is called a permutation matrix because multiplying a vector x by a permutation matrix has the effect of permuting (rearranging) the elements of x . Exercise D.1-4 explores additional properties of permutation matrices.

7. A **symmetric matrix** A satisfies the condition $A = A^T$. For example,

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 6 & 4 \\ 3 & 4 & 5 \end{pmatrix}$$

is a symmetric matrix.

Basic matrix operations

The elements of a matrix or vector are numbers from a number system, such as the real numbers, the complex numbers, or integers modulo a prime. The number system defines how to add and multiply numbers. We can extend these definitions to encompass addition and multiplication of matrices.

We define **matrix addition** as follows. If $A = (a_{ij})$ and $B = (b_{ij})$ are $m \times n$ matrices, then their matrix sum $C = (c_{ij}) = A + B$ is the $m \times n$ matrix defined by

$$c_{ij} = a_{ij} + b_{ij}$$

for $i = 1, 2, \dots, m$ and $j = 1, 2, \dots, n$. That is, matrix addition is performed componentwise. A zero matrix is the identity for matrix addition:

$$A + 0 = A = 0 + A.$$

If λ is a number and $A = (a_{ij})$ is a matrix, then $\lambda A = (\lambda a_{ij})$ is the **scalar multiple** of A obtained by multiplying each of its elements by λ . As a special case, we define the **negative** of a matrix $A = (a_{ij})$ to be $-1 \cdot A = -A$, so that the ij th entry of $-A$ is $-a_{ij}$. Thus,

$$A + (-A) = 0 = (-A) + A.$$

We use the negative of a matrix to define **matrix subtraction**: $A - B = A + (-B)$.

We define **matrix multiplication** as follows. We start with two matrices A and B that are **compatible** in the sense that the number of columns of A equals the number of rows of B . (In general, an expression containing a matrix product AB is always assumed to imply that matrices A and B are compatible.) If $A = (a_{ik})$ is an $m \times n$ matrix and $B = (b_{kj})$ is an $n \times p$ matrix, then their matrix product $C = AB$ is the $m \times p$ matrix $C = (c_{ij})$, where

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad (\text{D.2})$$

for $i = 1, 2, \dots, m$ and $j = 1, 2, \dots, p$. The procedure SQUARE-MATRIX-MULTIPLY in Section 4.2 implements matrix multiplication in the straightforward manner based on equation (D.2), assuming that the matrices are square: $m = n = p$. To multiply $n \times n$ matrices, SQUARE-MATRIX-MULTIPLY performs n^3 multiplications and $n^2(n-1)$ additions, and so its running time is $\Theta(n^3)$.

Matrices have many (but not all) of the algebraic properties typical of numbers. Identity matrices are identities for matrix multiplication:

$$I_m A = A I_n = A$$

for any $m \times n$ matrix A . Multiplying by a zero matrix gives a zero matrix:

$$A 0 = 0.$$

Matrix multiplication is associative:

$$A(BC) = (AB)C$$

for compatible matrices A , B , and C . Matrix multiplication distributes over addition:

$$\begin{aligned} A(B + C) &= AB + AC, \\ (B + C)D &= BD + CD. \end{aligned}$$

For $n > 1$, multiplication of $n \times n$ matrices is not commutative. For example, if

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \text{ and } B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \text{ then}$$

$$AB = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

and

$$BA = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

We define matrix-vector products or vector-vector products as if the vector were the equivalent $n \times 1$ matrix (or a $1 \times n$ matrix, in the case of a row vector). Thus, if A is an $m \times n$ matrix and x is an n -vector, then Ax is an m -vector. If x and y are n -vectors, then

$$x^T y = \sum_{i=1}^n x_i y_i$$

is a number (actually a 1×1 matrix) called the **inner product** of x and y . The matrix xy^T is an $n \times n$ matrix Z called the **outer product** of x and y , with $z_{ij} = x_i y_j$. The (**euclidean**) **norm** $\|x\|$ of an n -vector x is defined by

$$\begin{aligned} \|x\| &= (x_1^2 + x_2^2 + \cdots + x_n^2)^{1/2} \\ &= (x^T x)^{1/2}. \end{aligned}$$

Thus, the norm of x is its length in n -dimensional euclidean space.

Exercises

D.1-1

Show that if A and B are symmetric $n \times n$ matrices, then so are $A + B$ and $A - B$.

D.1-2

Prove that $(AB)^T = B^T A^T$ and that $A^T A$ is always a symmetric matrix.

D.1-3

Prove that the product of two lower-triangular matrices is lower-triangular.

D.1-4

Prove that if P is an $n \times n$ permutation matrix and A is an $n \times n$ matrix, then the matrix product PA is A with its rows permuted, and the matrix product AP is A with its columns permuted. Prove that the product of two permutation matrices is a permutation matrix.

D.2 Basic matrix properties

In this section, we define some basic properties pertaining to matrices: inverses, linear dependence and independence, rank, and determinants. We also define the class of positive-definite matrices.

Matrix inverses, ranks, and determinants

We define the **inverse** of an $n \times n$ matrix A to be the $n \times n$ matrix, denoted A^{-1} (if it exists), such that $AA^{-1} = I_n = A^{-1}A$. For example,

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{-1} = \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix}.$$

Many nonzero $n \times n$ matrices do not have inverses. A matrix without an inverse is called **noninvertible**, or **singular**. An example of a nonzero singular matrix is

$$\begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}.$$

If a matrix has an inverse, it is called **invertible**, or **nonsingular**. Matrix inverses, when they exist, are unique. (See Exercise D.2-1.) If A and B are nonsingular $n \times n$ matrices, then

$$(BA)^{-1} = A^{-1}B^{-1}.$$

The inverse operation commutes with the transpose operation:

$$(A^{-1})^T = (A^T)^{-1}.$$

The vectors $x_1, x_2, \dots, x_n$ are **linearly dependent** if there exist coefficients $c_1, c_2, \dots, c_n$, not all of which are zero, such that $c_1x_1 + c_2x_2 + \dots + c_nx_n = 0$. The row vectors $x_1 = (1 \ 2 \ 3)$, $x_2 = (2 \ 6 \ 4)$, and $x_3 = (4 \ 11 \ 9)$ are linearly dependent, for example, since $2x_1 + 3x_2 - 2x_3 = 0$. If vectors are not linearly dependent, they are **linearly independent**. For example, the columns of an identity matrix are linearly independent.

The **column rank** of a nonzero $m \times n$ matrix A is the size of the largest set of linearly independent columns of A . Similarly, the **row rank** of A is the size of the largest set of linearly independent rows of A . A fundamental property of any matrix A is that its row rank always equals its column rank, so that we can simply refer to the **rank** of A . The rank of an $m \times n$ matrix is an integer between 0 and $\min(m, n)$, inclusive. (The rank of a zero matrix is 0, and the rank of an $n \times n$ identity matrix is n .) An alternate, but equivalent and often more useful, definition is that the rank of a nonzero $m \times n$ matrix A is the smallest number r such that there exist matrices B and C of respective sizes $m \times r$ and $r \times n$ such that

$$A = BC.$$

A square $n \times n$ matrix has **full rank** if its rank is n . An $m \times n$ matrix has **full column rank** if its rank is n . The following theorem gives a fundamental property of ranks.

Theorem D.1

A square matrix has full rank if and only if it is nonsingular. ■

A **null vector** for a matrix A is a nonzero vector x such that $Ax = 0$. The following theorem (whose proof is left as Exercise D.2-7) and its corollary relate the notions of column rank and singularity to null vectors.

Theorem D.2

A matrix A has full column rank if and only if it does not have a null vector. ■

Corollary D.3

A square matrix A is singular if and only if it has a null vector. ■

The ij th **minor** of an $n \times n$ matrix A , for $n > 1$, is the $(n-1) \times (n-1)$ matrix $A_{[ij]}$ obtained by deleting the i th row and j th column of A . We define the **determinant** of an $n \times n$ matrix A recursively in terms of its minors by

$$\det(A) = \begin{cases} a_{11} & \text{if } n = 1, \\ \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(A_{[1j]}) & \text{if } n > 1. \end{cases}$$

The term $(-1)^{i+j} \det(A_{[ij]})$ is known as the **cofactor** of the element a_{ij} .

The following theorems, whose proofs are omitted here, express fundamental properties of the determinant.

Theorem D.4 (Determinant properties)

The determinant of a square matrix A has the following properties:

- If any row or any column of A is zero, then $\det(A) = 0$.
- The determinant of A is multiplied by λ if the entries of any one row (or any one column) of A are all multiplied by λ .
- The determinant of A is unchanged if the entries in one row (respectively, column) are added to those in another row (respectively, column).
- The determinant of A equals the determinant of A^T .
- The determinant of A is multiplied by -1 if any two rows (or any two columns) are exchanged.

Also, for any square matrices A and B , we have $\det(AB) = \det(A) \det(B)$. ■

Theorem D.5

An $n \times n$ matrix A is singular if and only if $\det(A) = 0$. ■

Positive-definite matrices

Positive-definite matrices play an important role in many applications. An $n \times n$ matrix A is **positive-definite** if $x^T A x > 0$ for all n -vectors $x \neq 0$. For example, the identity matrix is positive-definite, since for any nonzero vector $x = (x_1 \ x_2 \ \cdots \ x_n)^T$,

$$\begin{aligned} x^T I_n x &= x^T x \\ &= \sum_{i=1}^n x_i^2 \\ &> 0. \end{aligned}$$

Matrices that arise in applications are often positive-definite due to the following theorem.

Theorem D.6

For any matrix A with full column rank, the matrix $A^T A$ is positive-definite.

Proof We must show that $x^T (A^T A) x > 0$ for any nonzero vector x . For any vector x ,

$$\begin{aligned} x^T (A^T A) x &= (Ax)^T (Ax) \quad (\text{by Exercise D.1-2}) \\ &= \|Ax\|^2. \end{aligned}$$

Note that $\|Ax\|^2$ is just the sum of the squares of the elements of the vector Ax . Therefore, $\|Ax\|^2 \geq 0$. If $\|Ax\|^2 = 0$, every element of Ax is 0, which is to say $Ax = 0$. Since A has full column rank, $Ax = 0$ implies $x = 0$, by Theorem D.2. Hence, $A^T A$ is positive-definite. ■

Section 28.3 explores other properties of positive-definite matrices.

Exercises**D.2-1**

Prove that matrix inverses are unique, that is, if B and C are inverses of A , then $B = C$.

D.2-2

Prove that the determinant of a lower-triangular or upper-triangular matrix is equal to the product of its diagonal elements. Prove that the inverse of a lower-triangular matrix, if it exists, is lower-triangular.

D.2-3

Prove that if P is a permutation matrix, then P is invertible, its inverse is P^T , and P^T is a permutation matrix.

D.2-4

Let A and B be $n \times n$ matrices such that $AB = I$. Prove that if A' is obtained from A by adding row j into row i , then subtracting column i from column j of B yields the inverse B' of A' .

D.2-5

Let A be a nonsingular $n \times n$ matrix with complex entries. Show that every entry of A^{-1} is real if and only if every entry of A is real.

D.2-6

Show that if A is a nonsingular, symmetric, $n \times n$ matrix, then A^{-1} is symmetric. Show that if B is an arbitrary $m \times n$ matrix, then the $m \times m$ matrix given by the product BAB^T is symmetric.

D.2-7

Prove Theorem D.2. That is, show that a matrix A has full column rank if and only if $Ax = 0$ implies $x = 0$. (*Hint*: Express the linear dependence of one column on the others as a matrix-vector equation.)

D.2-8

Prove that for any two compatible matrices A and B ,

$$\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B)),$$

where equality holds if either A or B is a nonsingular square matrix. (*Hint*: Use the alternate definition of the rank of a matrix.)

Problems**D-1 Vandermonde matrix**

Given numbers $x_0, x_1, \dots, x_{n-1}$, prove that the determinant of the *Vandermonde matrix*

$$V(x_0, x_1, \dots, x_{n-1}) = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix}$$

is

$$\det(V(x_0, x_1, \dots, x_{n-1})) = \prod_{0 \leq j < k \leq n-1} (x_k - x_j).$$

(Hint: Multiply column i by $-x_0$ and add it to column $i + 1$ for $i = n - 1, n - 2, \dots, 1$, and then use induction.)

D-2 Permutations defined by matrix-vector multiplication over $GF(2)$

One class of permutations of the integers in the set $S_n = \{0, 1, 2, \dots, 2^n - 1\}$ is defined by matrix multiplication over $GF(2)$. For each integer x in S_n , we view its binary representation as an n -bit vector

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{pmatrix},$$

where $x = \sum_{i=0}^{n-1} x_i 2^i$. If A is an $n \times n$ matrix in which each entry is either 0 or 1, then we can define a permutation mapping each value x in S_n to the number whose binary representation is the matrix-vector product Ax . Here, we perform all arithmetic over $GF(2)$: all values are either 0 or 1, and with one exception the usual rules of addition and multiplication apply. The exception is that $1 + 1 = 0$. You can think of arithmetic over $GF(2)$ as being just like regular integer arithmetic, except that you use only the least significant bit.

As an example, for $S_2 = \{0, 1, 2, 3\}$, the matrix

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

defines the following permutation π_A : $\pi_A(0) = 0$, $\pi_A(1) = 3$, $\pi_A(2) = 2$, $\pi_A(3) = 1$. To see why $\pi_A(3) = 1$, observe that, working in $GF(2)$,

$$\begin{aligned} \pi_A(3) &= \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \cdot 1 + 0 \cdot 1 \\ 1 \cdot 1 + 1 \cdot 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \end{aligned}$$

which is the binary representation of 1.

For the remainder of this problem, we work over $GF(2)$, and all matrix and vector entries are 0 or 1. We define the rank of a 0-1 matrix (a matrix for which each entry is either 0 or 1) over $GF(2)$ the same as for a regular matrix, but with all arithmetic that determines linear independence performed over $GF(2)$. We define the **range** of an $n \times n$ 0-1 matrix A by

$$R(A) = \{y : y = Ax \text{ for some } x \in S_n\} ,$$

so that $R(A)$ is the set of numbers in S_n that we can produce by multiplying each value x in S_n by A .

- a. If r is the rank of matrix A , prove that $|R(A)| = 2^r$. Conclude that A defines a permutation on S_n only if A has full rank.

For a given $n \times n$ matrix A and a given value $y \in R(A)$, we define the **preimage** of y by

$$P(A, y) = \{x : Ax = y\} ,$$

so that $P(A, y)$ is the set of values in S_n that map to y when multiplied by A .

- b. If r is the rank of $n \times n$ matrix A and $y \in R(A)$, prove that $|P(A, y)| = 2^{n-r}$.

Let $0 \leq m \leq n$, and suppose we partition the set S_n into blocks of consecutive numbers, where the i th block consists of the 2^m numbers $i2^m, i2^m + 1, i2^m + 2, \dots, (i+1)2^m - 1$. For any subset $S \subseteq S_n$, define $B(S, m)$ to be the set of size- 2^m blocks of S_n containing some element of S . As an example, when $n = 3$, $m = 1$, and $S = \{1, 4, 5\}$, then $B(S, m)$ consists of blocks 0 (since 1 is in the 0th block) and 2 (since both 4 and 5 are in block 2).

- c. Let r be the rank of the lower left $(n - m) \times m$ submatrix of A , that is, the matrix formed by taking the intersection of the bottom $n - m$ rows and the leftmost m columns of A . Let S be any size- 2^m block of S_n , and let $S' = \{y : y = Ax \text{ for some } x \in S\}$. Prove that $|B(S', m)| = 2^r$ and that for each block in $B(S', m)$, exactly 2^{m-r} numbers in S map to that block.

Because multiplying the zero vector by any matrix yields a zero vector, the set of permutations of S_n defined by multiplying by $n \times n$ 0-1 matrices with full rank over $GF(2)$ cannot include all permutations of S_n . Let us extend the class of permutations defined by matrix-vector multiplication to include an additive term, so that $x \in S_n$ maps to $Ax + c$, where c is an n -bit vector and addition is performed over $GF(2)$. For example, when

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

and

$$c = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

we get the following permutation $\pi_{A,c}$: $\pi_{A,c}(0) = 2$, $\pi_{A,c}(1) = 1$, $\pi_{A,c}(2) = 0$, $\pi_{A,c}(3) = 3$. We call any permutation that maps $x \in S_n$ to $Ax + c$, for some $n \times n$ 0-1 matrix A with full rank and some n -bit vector c , a **linear permutation**.

- d.* Use a counting argument to show that the number of linear permutations of S_n is much less than the number of permutations of S_n .
- e.* Give an example of a value of n and a permutation of S_n that cannot be achieved by any linear permutation. (*Hint:* For a given permutation, think about how multiplying a matrix by a unit vector relates to the columns of the matrix.)

Appendix notes

Linear-algebra textbooks provide plenty of background information on matrices. The books by Strang [323, 324] are particularly good.

Bibliography

- [1] Milton Abramowitz and Irene A. Stegun, editors. *Handbook of Mathematical Functions*. Dover, 1965.
- [2] G. M. Adel'son-Vel'skiĭ and E. M. Landis. An algorithm for the organization of information. *Soviet Mathematics Doklady*, 3(5):1259–1263, 1962.
- [3] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [4] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004.
- [5] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.
- [6] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [7] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [8] Ravindra K. Ahuja, Kurt Mehlhorn, James B. Orlin, and Robert E. Tarjan. Faster algorithms for the shortest path problem. *Journal of the ACM*, 37:213–223, 1990.
- [9] Ravindra K. Ahuja and James B. Orlin. A fast and simple algorithm for the maximum flow problem. *Operations Research*, 37(5):748–759, 1989.
- [10] Ravindra K. Ahuja, James B. Orlin, and Robert E. Tarjan. Improved time bounds for the maximum flow problem. *SIAM Journal on Computing*, 18(5):939–954, 1989.
- [11] Miklós Ajtai, Nimrod Megiddo, and Orli Waarts. Improved algorithms and analysis for secretary problems and generalizations. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science*, pages 473–482, 1995.
- [12] Selim G. Akl. *The Design and Analysis of Parallel Algorithms*. Prentice Hall, 1989.
- [13] Mohamad Akra and Louay Bazzi. On the solution of linear recurrence equations. *Computational Optimization and Applications*, 10(2):195–210, 1998.
- [14] Noga Alon. Generating pseudo-random permutations and maximum flow algorithms. *Information Processing Letters*, 35:201–204, 1990.

- [15] Arne Andersson. Balanced search trees made simple. In *Proceedings of the Third Workshop on Algorithms and Data Structures*, volume 709 of *Lecture Notes in Computer Science*, pages 60–71. Springer, 1993.
- [16] Arne Andersson. Faster deterministic sorting and searching in linear space. In *Proceedings of the 37th Annual Symposium on Foundations of Computer Science*, pages 135–141, 1996.
- [17] Arne Andersson, Torben Hagerup, Stefan Nilsson, and Rajeev Raman. Sorting in linear time? *Journal of Computer and System Sciences*, 57:74–93, 1998.
- [18] Tom M. Apostol. *Calculus*, volume 1. Blaisdell Publishing Company, second edition, 1967.
- [19] Nimar S. Arora, Robert D. Blumofe, and C. Greg Plaxton. Thread scheduling for multiprogrammed multiprocessors. In *Proceedings of the 10th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 119–129, 1998.
- [20] Sanjeev Arora. *Probabilistic checking of proofs and the hardness of approximation problems*. PhD thesis, University of California, Berkeley, 1994.
- [21] Sanjeev Arora. The approximability of NP-hard problems. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pages 337–348, 1998.
- [22] Sanjeev Arora. Polynomial time approximation schemes for euclidean traveling salesman and other geometric problems. *Journal of the ACM*, 45(5):753–782, 1998.
- [23] Sanjeev Arora and Carsten Lund. Hardness of approximations. In Dorit S. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 399–446. PWS Publishing Company, 1997.
- [24] Javed A. Aslam. A simple bound on the expected height of a randomly built binary search tree. Technical Report TR2001-387, Dartmouth College Department of Computer Science, 2001.
- [25] Mikhail J. Atallah, editor. *Algorithms and Theory of Computation Handbook*. CRC Press, 1999.
- [26] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. Marchetti-Spaccamela, and M. Protasi. *Complexity and Approximation: Combinatorial Optimization Problems and Their Approximability Properties*. Springer, 1999.
- [27] Shai Avidan and Ariel Shamir. Seam carving for content-aware image resizing. *ACM Transactions on Graphics*, 26(3), article 10, 2007.
- [28] Sara Baase and Alan Van Gelder. *Computer Algorithms: Introduction to Design and Analysis*. Addison-Wesley, third edition, 2000.
- [29] Eric Bach. Private communication, 1989.
- [30] Eric Bach. Number-theoretic algorithms. In *Annual Review of Computer Science*, volume 4, pages 119–172. Annual Reviews, Inc., 1990.
- [31] Eric Bach and Jeffrey Shallit. *Algorithmic Number Theory—Volume I: Efficient Algorithms*. The MIT Press, 1996.
- [32] David H. Bailey, King Lee, and Horst D. Simon. Using Strassen’s algorithm to accelerate the solution of linear systems. *The Journal of Supercomputing*, 4(4):357–371, 1990.

- [33] Surender Baswana, Ramesh Hariharan, and Sandeep Sen. Improved decremental algorithms for maintaining transitive closure and all-pairs shortest paths. *Journal of Algorithms*, 62(2):74–92, 2007.
- [34] R. Bayer. Symmetric binary B-trees: Data structure and maintenance algorithms. *Acta Informatica*, 1(4):290–306, 1972.
- [35] R. Bayer and E. M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.
- [36] Pierre Beauchemin, Gilles Brassard, Claude Crépeau, Claude Goutier, and Carl Pomerance. The generation of random numbers that are probably prime. *Journal of Cryptology*, 1(1):53–64, 1988.
- [37] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [38] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
- [39] Michael Ben-Or. Lower bounds for algebraic computation trees. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, pages 80–86, 1983.
- [40] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Cache-oblivious B-trees. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science*, pages 399–409, 2000.
- [41] Samuel W. Bent and John W. John. Finding the median requires $2n$ comparisons. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, pages 213–216, 1985.
- [42] Jon L. Bentley. *Writing Efficient Programs*. Prentice Hall, 1982.
- [43] Jon L. Bentley. *Programming Pearls*. Addison-Wesley, 1986.
- [44] Jon L. Bentley, Dorothea Haken, and James B. Saxe. A general method for solving divide-and-conquer recurrences. *SIGACT News*, 12(3):36–44, 1980.
- [45] Daniel Bienstock and Benjamin McClosky. Tightening simplex mixed-integer sets with guaranteed bounds. *Optimization Online*, July 2008.
- [46] Patrick Billingsley. *Probability and Measure*. John Wiley & Sons, second edition, 1986.
- [47] Guy E. Blelloch. *Scan Primitives and Parallel Vector Models*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1989. Available as MIT Laboratory for Computer Science Technical Report MIT/LCS/TR-463.
- [48] Guy E. Blelloch. Programming parallel algorithms. *Communications of the ACM*, 39(3):85–97, 1996.
- [49] Guy E. Blelloch, Phillip B. Gibbons, and Yossi Matias. Provably efficient scheduling for languages with fine-grained parallelism. In *Proceedings of the 7th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 1–12, 1995.
- [50] Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.
- [51] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. Cilk: An efficient multithreaded runtime system. *Journal of Parallel and Distributed Computing*, 37(1):55–69, 1996.

- [52] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.
- [53] Béla Bollobás. *Random Graphs*. Academic Press, 1985.
- [54] Gilles Brassard and Paul Bratley. *Fundamentals of Algorithmics*. Prentice Hall, 1996.
- [55] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.
- [56] Richard P. Brent. An improved Monte Carlo factorization algorithm. *BIT*, 20(2):176–184, 1980.
- [57] J. P. Buhler, H. W. Lenstra, Jr., and Carl Pomerance. Factoring integers with the number field sieve. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 50–94. Springer, 1993.
- [58] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.
- [59] Barbara Chapman, Gabriele Jost, and Ruud van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. The MIT Press, 2007.
- [60] Bernard Chazelle. A minimum spanning tree algorithm with inverse-Ackermann type complexity. *Journal of the ACM*, 47(6):1028–1047, 2000.
- [61] Joseph Cheriyan and Torben Hagerup. A randomized maximum-flow algorithm. *SIAM Journal on Computing*, 24(2):203–226, 1995.
- [62] Joseph Cheriyan and S. N. Maheshwari. Analysis of preflow push algorithms for maximum network flow. *SIAM Journal on Computing*, 18(6):1057–1086, 1989.
- [63] Boris V. Cherkassky and Andrew V. Goldberg. On implementing the push-relabel method for the maximum flow problem. *Algorithmica*, 19(4):390–410, 1997.
- [64] Boris V. Cherkassky, Andrew V. Goldberg, and Tomasz Radzik. Shortest paths algorithms: Theory and experimental evaluation. *Mathematical Programming*, 73(2):129–174, 1996.
- [65] Boris V. Cherkassky, Andrew V. Goldberg, and Craig Silverstein. Buckets, heaps, lists and monotone priority queues. *SIAM Journal on Computing*, 28(4):1326–1346, 1999.
- [66] H. Chernoff. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Annals of Mathematical Statistics*, 23(4):493–507, 1952.
- [67] Kai Lai Chung. *Elementary Probability Theory with Stochastic Processes*. Springer, 1974.
- [68] V. Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235, 1979.
- [69] V. Chvátal. *Linear Programming*. W. H. Freeman and Company, 1983.
- [70] V. Chvátal, D. A. Klarnier, and D. E. Knuth. Selected combinatorial research problems. Technical Report STAN-CS-72-292, Computer Science Department, Stanford University, 1972.
- [71] Cilk Arts, Inc., Burlington, Massachusetts. *Cilk++ Programmer’s Guide*, 2008. Available at <http://www.cilk.com/archive/docs/cilk1guide>.

- [72] Alan Cobham. The intrinsic computational difficulty of functions. In *Proceedings of the 1964 Congress for Logic, Methodology, and the Philosophy of Science*, pages 24–30. North-Holland, 1964.
- [73] H. Cohen and H. W. Lenstra, Jr. Primality testing and Jacobi sums. *Mathematics of Computation*, 42(165):297–330, 1984.
- [74] D. Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.
- [75] Stephen Cook. The complexity of theorem proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.
- [76] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [77] Don Coppersmith. Modifications to the number field sieve. *Journal of Cryptology*, 6(3):169–180, 1993.
- [78] Don Coppersmith and Shmuel Winograd. Matrix multiplication via arithmetic progression. *Journal of Symbolic Computation*, 9(3):251–280, 1990.
- [79] Thomas H. Cormen, Thomas Sundquist, and Leonard F. Wisniewski. Asymptotically tight bounds for performing BMMC permutations on parallel disk systems. *SIAM Journal on Computing*, 28(1):105–136, 1998.
- [80] Don Dailey and Charles E. Leiserson. Using Cilk to write multiprocessor chess programs. In H. J. van den Herik and B. Monien, editors, *Advances in Computer Games*, volume 9, pages 25–52. University of Maastricht, Netherlands, 2001.
- [81] Paolo D’Alberto and Alexandru Nicolau. Adaptive Strassen’s matrix multiplication. In *Proceedings of the 21st Annual International Conference on Supercomputing*, pages 284–292, June 2007.
- [82] Sanjoy Dasgupta, Christos Papadimitriou, and Umesh Vazirani. *Algorithms*. McGraw-Hill, 2008.
- [83] Roman Dementiev, Lutz Kettner, Jens Mehnert, and Peter Sanders. Engineering a sorted list data structure for 32 bit keys. In *Proceedings of the Sixth Workshop on Algorithm Engineering and Experiments and the First Workshop on Analytic Algorithmics and Combinatorics*, pages 142–151, January 2004.
- [84] Camil Demetrescu and Giuseppe F. Italiano. Fully dynamic all pairs shortest paths with real edge weights. *Journal of Computer and System Sciences*, 72(5):813–837, 2006.
- [85] Eric V. Denardo and Bennett L. Fox. Shortest-route methods: 1. Reaching, pruning, and buckets. *Operations Research*, 27(1):161–186, 1979.
- [86] Martin Dietzfelbinger, Anna Karlin, Kurt Mehlhorn, Friedhelm Meyer auf der Heide, Hans Rohnert, and Robert E. Tarjan. Dynamic perfect hashing: Upper and lower bounds. *SIAM Journal on Computing*, 23(4):738–761, 1994.
- [87] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, IT-22(6):644–654, 1976.
- [88] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

- [89] E. A. Dinic. Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11(5):1277–1280, 1970.
- [90] Brandon Dixon, Monika Rauch, and Robert E. Tarjan. Verification and sensitivity analysis of minimum spanning trees in linear time. *SIAM Journal on Computing*, 21(6):1184–1192, 1992.
- [91] John D. Dixon. Factorization and primality tests. *The American Mathematical Monthly*, 91(6):333–352, 1984.
- [92] Dorit Dor, Johan Håstad, Staffan Ulfberg, and Uri Zwick. On lower bounds for selecting the median. *SIAM Journal on Discrete Mathematics*, 14(3):299–311, 2001.
- [93] Dorit Dor and Uri Zwick. Selecting the median. *SIAM Journal on Computing*, 28(5):1722–1758, 1999.
- [94] Dorit Dor and Uri Zwick. Median selection requires $(2 + \epsilon)n$ comparisons. *SIAM Journal on Discrete Mathematics*, 14(3):312–325, 2001.
- [95] Alvin W. Drake. *Fundamentals of Applied Probability Theory*. McGraw-Hill, 1967.
- [96] James R. Driscoll, Harold N. Gabow, Ruth Shrairman, and Robert E. Tarjan. Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation. *Communications of the ACM*, 31(11):1343–1354, 1988.
- [97] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. Making data structures persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.
- [98] Derek L. Eager, John Zahorjan, and Edward D. Lazowska. Speedup versus efficiency in parallel systems. *IEEE Transactions on Computers*, 38(3):408–423, 1989.
- [99] Herbert Edelsbrunner. *Algorithms in Combinatorial Geometry*, volume 10 of *EATCS Monographs on Theoretical Computer Science*. Springer, 1987.
- [100] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.
- [101] Jack Edmonds. Matroids and the greedy algorithm. *Mathematical Programming*, 1(1):127–136, 1971.
- [102] Jack Edmonds and Richard M. Karp. Theoretical improvements in the algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- [103] Shimon Even. *Graph Algorithms*. Computer Science Press, 1979.
- [104] William Feller. *An Introduction to Probability Theory and Its Applications*. John Wiley & Sons, third edition, 1968.
- [105] Robert W. Floyd. Algorithm 97 (SHORTEST PATH). *Communications of the ACM*, 5(6):345, 1962.
- [106] Robert W. Floyd. Algorithm 245 (TREESORT). *Communications of the ACM*, 7(12):701, 1964.
- [107] Robert W. Floyd. Permuting information in idealized two-level storage. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 105–109. Plenum Press, 1972.

- [108] Robert W. Floyd and Ronald L. Rivest. Expected time bounds for selection. *Communications of the ACM*, 18(3):165–172, 1975.
- [109] Lestor R. Ford, Jr. and D. R. Fulkerson. *Flows in Networks*. Princeton University Press, 1962.
- [110] Lestor R. Ford, Jr. and Selmer M. Johnson. A tournament problem. *The American Mathematical Monthly*, 66(5):387–389, 1959.
- [111] Michael L. Fredman. New bounds on the complexity of the shortest path problem. *SIAM Journal on Computing*, 5(1):83–89, 1976.
- [112] Michael L. Fredman, János Komlós, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM*, 31(3):538–544, 1984.
- [113] Michael L. Fredman and Michael E. Saks. The cell probe complexity of dynamic data structures. In *Proceedings of the Twenty First Annual ACM Symposium on Theory of Computing*, pages 345–354, 1989.
- [114] Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987.
- [115] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424–436, 1993.
- [116] Michael L. Fredman and Dan E. Willard. Trans-dichotomous algorithms for minimum spanning trees and shortest paths. *Journal of Computer and System Sciences*, 48(3):533–551, 1994.
- [117] Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005.
- [118] Matteo Frigo, Charles E. Leiserson, and Keith H. Randall. The implementation of the Cilk-5 multithreaded language. In *Proceedings of the 1998 ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 212–223, 1998.
- [119] Harold N. Gabow. Path-based depth-first search for strong and biconnected components. *Information Processing Letters*, 74(3–4):107–114, 2000.
- [120] Harold N. Gabow, Z. Galil, T. Spencer, and Robert E. Tarjan. Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. *Combinatorica*, 6(2):109–122, 1986.
- [121] Harold N. Gabow and Robert E. Tarjan. A linear-time algorithm for a special case of disjoint set union. *Journal of Computer and System Sciences*, 30(2):209–221, 1985.
- [122] Harold N. Gabow and Robert E. Tarjan. Faster scaling algorithms for network problems. *SIAM Journal on Computing*, 18(5):1013–1036, 1989.
- [123] Zvi Galil and Oded Margalit. All pairs shortest distances for graphs with small integer length edges. *Information and Computation*, 134(2):103–139, 1997.
- [124] Zvi Galil and Oded Margalit. All pairs shortest paths for graphs with small integer length edges. *Journal of Computer and System Sciences*, 54(2):243–254, 1997.
- [125] Zvi Galil and Kunsoo Park. Dynamic programming with convexity, concavity and sparsity. *Theoretical Computer Science*, 92(1):49–76, 1992.

- [126] Zvi Galil and Joel Seiferas. Time-space-optimal string matching. *Journal of Computer and System Sciences*, 26(3):280–294, 1983.
- [127] Igal Galperin and Ronald L. Rivest. Scapegoat trees. In *Proceedings of the 4th ACM-SIAM Symposium on Discrete Algorithms*, pages 165–174, 1993.
- [128] Michael R. Garey, R. L. Graham, and J. D. Ullman. Worst-case analysis of memory allocation algorithms. In *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*, pages 143–150, 1972.
- [129] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [130] Saul Gass. *Linear Programming: Methods and Applications*. International Thomson Publishing, fourth edition, 1975.
- [131] Fănică Gavril. Algorithms for minimum coloring, maximum clique, minimum covering by cliques, and maximum independent set of a chordal graph. *SIAM Journal on Computing*, 1(2):180–187, 1972.
- [132] Alan George and Joseph W-H Liu. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice Hall, 1981.
- [133] E. N. Gilbert and E. F. Moore. Variable-length binary encodings. *Bell System Technical Journal*, 38(4):933–967, 1959.
- [134] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.
- [135] Michel X. Goemans and David P. Williamson. The primal-dual method for approximation algorithms and its application to network design problems. In Dorit S. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 144–191. PWS Publishing Company, 1997.
- [136] Andrew V. Goldberg. *Efficient Graph Algorithms for Sequential and Parallel Computers*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1987.
- [137] Andrew V. Goldberg. Scaling algorithms for the shortest paths problem. *SIAM Journal on Computing*, 24(3):494–504, 1995.
- [138] Andrew V. Goldberg and Satish Rao. Beyond the flow decomposition barrier. *Journal of the ACM*, 45(5):783–797, 1998.
- [139] Andrew V. Goldberg, Éva Tardos, and Robert E. Tarjan. Network flow algorithms. In Bernhard Korte, László Lovász, Hans Jürgen Prömel, and Alexander Schrijver, editors, *Paths, Flows, and VLSI-Layout*, pages 101–164. Springer, 1990.
- [140] Andrew V. Goldberg and Robert E. Tarjan. A new approach to the maximum flow problem. *Journal of the ACM*, 35(4):921–940, 1988.
- [141] D. Goldfarb and M. J. Todd. Linear programming. In G. L. Nemhauser, A. H. G. Rinnooy-Kan, and M. J. Todd, editors, *Handbook in Operations Research and Management Science, Vol. 1, Optimization*, pages 73–170. Elsevier Science Publishers, 1989.
- [142] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984.

- [143] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.
- [144] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. The Johns Hopkins University Press, third edition, 1996.
- [145] G. H. Gonnet. *Handbook of Algorithms and Data Structures*. Addison-Wesley, 1984.
- [146] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Addison-Wesley, 1992.
- [147] Michael T. Goodrich and Roberto Tamassia. *Data Structures and Algorithms in Java*. John Wiley & Sons, 1998.
- [148] Michael T. Goodrich and Roberto Tamassia. *Algorithm Design: Foundations, Analysis, and Internet Examples*. John Wiley & Sons, 2001.
- [149] Ronald L. Graham. Bounds for certain multiprocessor anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.
- [150] Ronald L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Information Processing Letters*, 1(4):132–133, 1972.
- [151] Ronald L. Graham and Pavol Hell. On the history of the minimum spanning tree problem. *Annals of the History of Computing*, 7(1):43–57, 1985.
- [152] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics*. Addison-Wesley, second edition, 1994.
- [153] David Gries. *The Science of Programming*. Springer, 1981.
- [154] M. Grötschel, László Lovász, and Alexander Schrijver. *Geometric Algorithms and Combinatorial Optimization*. Springer, 1988.
- [155] Leo J. Guibas and Robert Sedgewick. A dichromatic framework for balanced trees. In *Proceedings of the 19th Annual Symposium on Foundations of Computer Science*, pages 8–21, 1978.
- [156] Dan Gusfield. *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.
- [157] H. Halberstam and R. E. Ingram, editors. *The Mathematical Papers of Sir William Rowan Hamilton*, volume III (Algebra). Cambridge University Press, 1967.
- [158] Yijie Han. Improved fast integer sorting in linear space. In *Proceedings of the 12th ACM-SIAM Symposium on Discrete Algorithms*, pages 793–796, 2001.
- [159] Yijie Han. An $O(n^3(\log \log n / \log n)^{5/4})$ time algorithm for all pairs shortest path. *Algorithmica*, 51(4):428–434, 2008.
- [160] Frank Harary. *Graph Theory*. Addison-Wesley, 1969.
- [161] Gregory C. Harfst and Edward M. Reingold. A potential-based amortized analysis of the union-find data structure. *SIGACT News*, 31(3):86–95, 2000.
- [162] J. Hartmanis and R. E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, May 1965.

- [163] Michael T. Heideman, Don H. Johnson, and C. Sidney Burrus. Gauss and the history of the Fast Fourier Transform. *IEEE ASSP Magazine*, 1(4):14–21, 1984.
- [164] Monika R. Henzinger and Valerie King. Fully dynamic biconnectivity and transitive closure. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science*, pages 664–672, 1995.
- [165] Monika R. Henzinger and Valerie King. Randomized fully dynamic graph algorithms with polylogarithmic time per operation. *Journal of the ACM*, 46(4):502–516, 1999.
- [166] Monika R. Henzinger, Satish Rao, and Harold N. Gabow. Computing vertex connectivity: New bounds from old techniques. *Journal of Algorithms*, 34(2):222–250, 2000.
- [167] Nicholas J. Higham. Exploiting fast matrix multiplication within the level 3 BLAS. *ACM Transactions on Mathematical Software*, 16(4):352–368, 1990.
- [168] W. Daniel Hillis and Jr. Guy L. Steele. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.
- [169] C. A. R. Hoare. Algorithm 63 (PARTITION) and algorithm 65 (FIND). *Communications of the ACM*, 4(7):321–322, 1961.
- [170] C. A. R. Hoare. Quicksort. *Computer Journal*, 5(1):10–15, 1962.
- [171] Dorit S. Hochbaum. Efficient bounds for the stable set, vertex cover and set packing problems. *Discrete Applied Mathematics*, 6(3):243–254, 1983.
- [172] Dorit S. Hochbaum, editor. *Approximation Algorithms for NP-Hard Problems*. PWS Publishing Company, 1997.
- [173] W. Hoeffding. On the distribution of the number of successes in independent trials. *Annals of Mathematical Statistics*, 27(3):713–721, 1956.
- [174] Micha Hofri. *Probabilistic Analysis of Algorithms*. Springer, 1987.
- [175] Micha Hofri. *Analysis of Algorithms*. Oxford University Press, 1995.
- [176] John E. Hopcroft and Richard M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973.
- [177] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison Wesley, third edition, 2006.
- [178] John E. Hopcroft and Robert E. Tarjan. Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973.
- [179] John E. Hopcroft and Jeffrey D. Ullman. Set merging algorithms. *SIAM Journal on Computing*, 2(4):294–303, 1973.
- [180] John E. Hopcroft and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 1979.
- [181] Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran. *Computer Algorithms*. Computer Science Press, 1998.
- [182] T. C. Hu and M. T. Shing. Computation of matrix chain products. Part I. *SIAM Journal on Computing*, 11(2):362–373, 1982.
- [183] T. C. Hu and M. T. Shing. Computation of matrix chain products. Part II. *SIAM Journal on Computing*, 13(2):228–251, 1984.

- [184] T. C. Hu and A. C. Tucker. Optimal computer search trees and variable-length alphabetic codes. *SIAM Journal on Applied Mathematics*, 21(4):514–532, 1971.
- [185] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- [186] Steven Huss-Lederman, Elaine M. Jacobson, Jeremy R. Johnson, Anna Tsao, and Thomas Turnbull. Implementation of Strassen’s algorithm for matrix multiplication. In *Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, article 32, 1996.
- [187] Oscar H. Ibarra and Chul E. Kim. Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM*, 22(4):463–468, 1975.
- [188] E. J. Isaac and R. C. Singleton. Sorting by address calculation. *Journal of the ACM*, 3(3):169–174, 1956.
- [189] R. A. Jarvis. On the identification of the convex hull of a finite set of points in the plane. *Information Processing Letters*, 2(1):18–21, 1973.
- [190] David S. Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3):256–278, 1974.
- [191] David S. Johnson. The NP-completeness column: An ongoing guide—The tale of the second prover. *Journal of Algorithms*, 13(3):502–524, 1992.
- [192] Donald B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1):1–13, 1977.
- [193] Richard Johnsonbaugh and Marcus Schaefer. *Algorithms*. Pearson Prentice Hall, 2004.
- [194] A. Karatsuba and Yu. Ofman. Multiplication of multidigit numbers on automata. *Soviet Physics—Doklady*, 7(7):595–596, 1963. Translation of an article in *Doklady Akademii Nauk SSSR*, 145(2), 1962.
- [195] David R. Karger, Philip N. Klein, and Robert E. Tarjan. A randomized linear-time algorithm to find minimum spanning trees. *Journal of the ACM*, 42(2):321–328, 1995.
- [196] David R. Karger, Daphne Koller, and Steven J. Phillips. Finding the hidden path: Time bounds for all-pairs shortest paths. *SIAM Journal on Computing*, 22(6):1199–1217, 1993.
- [197] Howard Karloff. *Linear Programming*. Birkhäuser, 1991.
- [198] N. Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4(4):373–395, 1984.
- [199] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.
- [200] Richard M. Karp. An introduction to randomized algorithms. *Discrete Applied Mathematics*, 34(1–3):165–201, 1991.
- [201] Richard M. Karp and Michael O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2):249–260, 1987.
- [202] A. V. Karzanov. Determining the maximal flow in a network by the method of preflows. *Soviet Mathematics Doklady*, 15(2):434–437, 1974.

- [203] Valerie King. A simpler minimum spanning tree verification algorithm. *Algorithmica*, 18(2):263–270, 1997.
- [204] Valerie King, Satish Rao, and Robert E. Tarjan. A faster deterministic maximum flow algorithm. *Journal of Algorithms*, 17(3):447–474, 1994.
- [205] Jeffrey H. Kingston. *Algorithms and Data Structures: Design, Correctness, Analysis*. Addison-Wesley, second edition, 1997.
- [206] D. G. Kirkpatrick and R. Seidel. The ultimate planar convex hull algorithm? *SIAM Journal on Computing*, 15(2):287–299, 1986.
- [207] Philip N. Klein and Neal E. Young. Approximation algorithms for NP-hard optimization problems. In *CRC Handbook on Algorithms*, pages 34-1–34-19. CRC Press, 1999.
- [208] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Addison-Wesley, 2006.
- [209] Donald E. Knuth. *Fundamental Algorithms*, volume 1 of *The Art of Computer Programming*. Addison-Wesley, 1968. Third edition, 1997.
- [210] Donald E. Knuth. *Seminumerical Algorithms*, volume 2 of *The Art of Computer Programming*. Addison-Wesley, 1969. Third edition, 1997.
- [211] Donald E. Knuth. *Sorting and Searching*, volume 3 of *The Art of Computer Programming*. Addison-Wesley, 1973. Second edition, 1998.
- [212] Donald E. Knuth. Optimum binary search trees. *Acta Informatica*, 1(1):14–25, 1971.
- [213] Donald E. Knuth. Big omicron and big omega and big theta. *SIGACT News*, 8(2):18–23, 1976.
- [214] Donald E. Knuth, James H. Morris, Jr., and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977.
- [215] J. Komlós. Linear verification for spanning trees. *Combinatorica*, 5(1):57–65, 1985.
- [216] Bernhard Korte and László Lovász. Mathematical structures underlying greedy algorithms. In F. Gecseg, editor, *Fundamentals of Computation Theory*, volume 117 of *Lecture Notes in Computer Science*, pages 205–209. Springer, 1981.
- [217] Bernhard Korte and László Lovász. Structural properties of greedoids. *Combinatorica*, 3(3–4):359–374, 1983.
- [218] Bernhard Korte and László Lovász. Greedoids—A structural framework for the greedy algorithm. In W. Pulleybank, editor, *Progress in Combinatorial Optimization*, pages 221–243. Academic Press, 1984.
- [219] Bernhard Korte and László Lovász. Greedoids and linear objective functions. *SIAM Journal on Algebraic and Discrete Methods*, 5(2):229–238, 1984.
- [220] Dexter C. Kozen. *The Design and Analysis of Algorithms*. Springer, 1992.
- [221] David W. Krumme, George Cybenko, and K. N. Venkatarman. Gossiping in minimal time. *SIAM Journal on Computing*, 21(1):111–139, 1992.
- [222] Joseph B. Kruskal, Jr. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.
- [223] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.

- [224] Eugene L. Lawler. *Combinatorial Optimization: Networks and Matroids*. Holt, Rinehart, and Winston, 1976.
- [225] Eugene L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys, editors. *The Traveling Salesman Problem*. John Wiley & Sons, 1985.
- [226] C. Y. Lee. An algorithm for path connection and its applications. *IRE Transactions on Electronic Computers*, EC-10(3):346–365, 1961.
- [227] Tom Leighton. Tight bounds on the complexity of parallel sorting. *IEEE Transactions on Computers*, C-34(4):344–354, 1985.
- [228] Tom Leighton. Notes on better master theorems for divide-and-conquer recurrences. Class notes. Available at <http://citeseer.ist.psu.edu/252350.html>, October 1996.
- [229] Tom Leighton and Satish Rao. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *Journal of the ACM*, 46(6):787–832, 1999.
- [230] Daan Leijen and Judd Hall. Optimize managed code for multi-core machines. *MSDN Magazine*, October 2007.
- [231] Debra A. Lelewer and Daniel S. Hirschberg. Data compression. *ACM Computing Surveys*, 19(3):261–296, 1987.
- [232] A. K. Lenstra, H. W. Lenstra, Jr., M. S. Manasse, and J. M. Pollard. The number field sieve. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 11–42. Springer, 1993.
- [233] H. W. Lenstra, Jr. Factoring integers with elliptic curves. *Annals of Mathematics*, 126(3):649–673, 1987.
- [234] L. A. Levin. Universal sorting problems. *Problemy Peredachi Informatsii*, 9(3):265–266, 1973. In Russian.
- [235] Anany Levitin. *Introduction to the Design & Analysis of Algorithms*. Addison-Wesley, 2007.
- [236] Harry R. Lewis and Christos H. Papadimitriou. *Elements of the Theory of Computation*. Prentice Hall, second edition, 1998.
- [237] C. L. Liu. *Introduction to Combinatorial Mathematics*. McGraw-Hill, 1968.
- [238] László Lovász. On the ratio of optimal integral and fractional covers. *Discrete Mathematics*, 13(4):383–390, 1975.
- [239] László Lovász and M. D. Plummer. *Matching Theory*, volume 121 of *Annals of Discrete Mathematics*. North Holland, 1986.
- [240] Bruce M. Maggs and Serge A. Plotkin. Minimum-cost spanning tree as a path-finding problem. *Information Processing Letters*, 26(6):291–293, 1988.
- [241] Michael Main. *Data Structures and Other Objects Using Java*. Addison-Wesley, 1999.
- [242] Udi Manber. *Introduction to Algorithms: A Creative Approach*. Addison-Wesley, 1989.
- [243] Conrado Martínez and Salvador Roura. Randomized binary search trees. *Journal of the ACM*, 45(2):288–323, 1998.
- [244] William J. Masek and Michael S. Paterson. A faster algorithm computing string edit distances. *Journal of Computer and System Sciences*, 20(1):18–31, 1980.

- [245] H. A. Maurer, Th. Ottmann, and H.-W. Six. Implementing dictionaries using binary trees of very small height. *Information Processing Letters*, 5(1):11–14, 1976.
- [246] Ernst W. Mayr, Hans Jürgen Prömel, and Angelika Steger, editors. *Lectures on Proof Verification and Approximation Algorithms*, volume 1367 of *Lecture Notes in Computer Science*. Springer, 1998.
- [247] C. C. McGeoch. All pairs shortest paths and the essential subgraph. *Algorithmica*, 13(5):426–441, 1995.
- [248] M. D. McIlroy. A killer adversary for quicksort. *Software—Practice and Experience*, 29(4):341–344, 1999.
- [249] Kurt Mehlhorn. *Sorting and Searching*, volume 1 of *Data Structures and Algorithms*. Springer, 1984.
- [250] Kurt Mehlhorn. *Graph Algorithms and NP-Completeness*, volume 2 of *Data Structures and Algorithms*. Springer, 1984.
- [251] Kurt Mehlhorn. *Multidimensional Searching and Computational Geometry*, volume 3 of *Data Structures and Algorithms*. Springer, 1984.
- [252] Kurt Mehlhorn and Stefan Näher. Bounded ordered dictionaries in $O(\log \log N)$ time and $O(n)$ space. *Information Processing Letters*, 35(4):183–189, 1990.
- [253] Kurt Mehlhorn and Stefan Näher. *LEDA: A Platform for Combinatorial and Geometric Computing*. Cambridge University Press, 1999.
- [254] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1997.
- [255] Gary L. Miller. Riemann’s hypothesis and tests for primality. *Journal of Computer and System Sciences*, 13(3):300–317, 1976.
- [256] John C. Mitchell. *Foundations for Programming Languages*. The MIT Press, 1996.
- [257] Joseph S. B. Mitchell. Guillotine subdivisions approximate polygonal subdivisions: A simple polynomial-time approximation scheme for geometric TSP, k -MST, and related problems. *SIAM Journal on Computing*, 28(4):1298–1309, 1999.
- [258] Louis Monier. *Algorithmes de Factorisation D’Entiers*. PhD thesis, L’Université Paris-Sud, 1980.
- [259] Louis Monier. Evaluation and comparison of two efficient probabilistic primality testing algorithms. *Theoretical Computer Science*, 12(1):97–108, 1980.
- [260] Edward F. Moore. The shortest path through a maze. In *Proceedings of the International Symposium on the Theory of Switching*, pages 285–292. Harvard University Press, 1959.
- [261] Rajeev Motwani, Joseph (Seffi) Naor, and Prabhakar Raghavan. Randomized approximation algorithms in combinatorial optimization. In Dorit Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, chapter 11, pages 447–481. PWS Publishing Company, 1997.
- [262] Rajeev Motwani and Prabhakar Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995.
- [263] J. I. Munro and V. Raman. Fast stable in-place sorting with $O(n)$ data moves. *Algorithmica*, 16(2):151–160, 1996.

- [264] J. Nievergelt and E. M. Reingold. Binary search trees of bounded balance. *SIAM Journal on Computing*, 2(1):33–43, 1973.
- [265] Ivan Niven and Herbert S. Zuckerman. *An Introduction to the Theory of Numbers*. John Wiley & Sons, fourth edition, 1980.
- [266] Alan V. Oppenheim and Ronald W. Schaffer, with John R. Buck. *Discrete-Time Signal Processing*. Prentice Hall, second edition, 1998.
- [267] Alan V. Oppenheim and Alan S. Willsky, with S. Hamid Nawab. *Signals and Systems*. Prentice Hall, second edition, 1997.
- [268] James B. Orlin. A polynomial time primal network simplex algorithm for minimum cost flows. *Mathematical Programming*, 78(1):109–129, 1997.
- [269] Joseph O’Rourke. *Computational Geometry in C*. Cambridge University Press, second edition, 1998.
- [270] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.
- [271] Christos H. Papadimitriou and Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice Hall, 1982.
- [272] Michael S. Paterson. Progress in selection. In *Proceedings of the Fifth Scandinavian Workshop on Algorithm Theory*, pages 368–379, 1996.
- [273] Mihai Pătraşcu and Mikkel Thorup. Time-space trade-offs for predecessor search. In *Proceedings of the 38th Annual ACM Symposium on Theory of Computing*, pages 232–240, 2006.
- [274] Mihai Pătraşcu and Mikkel Thorup. Randomization does not help searching predecessors. In *Proceedings of the 18th ACM-SIAM Symposium on Discrete Algorithms*, pages 555–564, 2007.
- [275] Pavel A. Pevzner. *Computational Molecular Biology: An Algorithmic Approach*. The MIT Press, 2000.
- [276] Steven Phillips and Jeffery Westbrook. Online load balancing and network flow. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing*, pages 402–411, 1993.
- [277] J. M. Pollard. A Monte Carlo method for factorization. *BIT*, 15(3):331–334, 1975.
- [278] J. M. Pollard. Factoring with cubic integers. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 4–10. Springer, 1993.
- [279] Carl Pomerance. On the distribution of pseudoprimes. *Mathematics of Computation*, 37(156):587–593, 1981.
- [280] Carl Pomerance, editor. *Proceedings of the AMS Symposia in Applied Mathematics: Computational Number Theory and Cryptography*. American Mathematical Society, 1990.
- [281] William K. Pratt. *Digital Image Processing*. John Wiley & Sons, fourth edition, 2007.
- [282] Franco P. Preparata and Michael Ian Shamos. *Computational Geometry: An Introduction*. Springer, 1985.

- [283] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes in C++: The Art of Scientific Computing*. Cambridge University Press, second edition, 2002.
- [284] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, third edition, 2007.
- [285] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6):1389–1401, 1957.
- [286] William Pugh. Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6):668–676, 1990.
- [287] Paul W. Purdom, Jr. and Cynthia A. Brown. *The Analysis of Algorithms*. Holt, Rinehart, and Winston, 1985.
- [288] Michael O. Rabin. Probabilistic algorithms. In J. F. Traub, editor, *Algorithms and Complexity: New Directions and Recent Results*, pages 21–39. Academic Press, 1976.
- [289] Michael O. Rabin. Probabilistic algorithm for testing primality. *Journal of Number Theory*, 12(1):128–138, 1980.
- [290] P. Raghavan and C. D. Thompson. Randomized rounding: A technique for provably good algorithms and algorithmic proofs. *Combinatorica*, 7(4):365–374, 1987.
- [291] Rajeev Raman. Recent results on the single-source shortest paths problem. *SIGACT News*, 28(2):81–87, 1997.
- [292] James Reinders. *Intel Threading Building Blocks: Outfitting C++ for Multi-core Processor Parallelism*. O'Reilly Media, Inc., 2007.
- [293] Edward M. Reingold, Jürg Nievergelt, and Narsingh Deo. *Combinatorial Algorithms: Theory and Practice*. Prentice Hall, 1977.
- [294] Edward M. Reingold, Kenneth J. Urban, and David Gries. K-M-P string matching revisited. *Information Processing Letters*, 64(5):217–223, 1997.
- [295] Hans Riesel. *Prime Numbers and Computer Methods for Factorization*, volume 126 of *Progress in Mathematics*. Birkhäuser, second edition, 1994.
- [296] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. See also U.S. Patent 4,405,829.
- [297] Herbert Robbins. A remark on Stirling's formula. *American Mathematical Monthly*, 62(1):26–29, 1955.
- [298] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977.
- [299] Salvador Roura. An improved master theorem for divide-and-conquer recurrences. In *Proceedings of Automata, Languages and Programming, 24th International Colloquium, ICALP'97*, volume 1256 of *Lecture Notes in Computer Science*, pages 449–459. Springer, 1997.
- [300] Y. A. Rozanov. *Probability Theory: A Concise Course*. Dover, 1969.

- [301] S. Sahni and T. Gonzalez. P-complete approximation problems. *Journal of the ACM*, 23(3):555–565, 1976.
- [302] A. Schönhage, M. Paterson, and N. Pippenger. Finding the median. *Journal of Computer and System Sciences*, 13(2):184–199, 1976.
- [303] Alexander Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1986.
- [304] Alexander Schrijver. Paths and flows—A historical survey. *CWI Quarterly*, 6(3):169–183, 1993.
- [305] Robert Sedgewick. Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.
- [306] Robert Sedgewick. *Algorithms*. Addison-Wesley, second edition, 1988.
- [307] Robert Sedgewick and Philippe Flajolet. *An Introduction to the Analysis of Algorithms*. Addison-Wesley, 1996.
- [308] Raimund Seidel. On the all-pairs-shortest-path problem in unweighted undirected graphs. *Journal of Computer and System Sciences*, 51(3):400–403, 1995.
- [309] Raimund Seidel and C. R. Aragon. Randomized search trees. *Algorithmica*, 16(4–5):464–497, 1996.
- [310] João Setubal and João Meidanis. *Introduction to Computational Molecular Biology*. PWS Publishing Company, 1997.
- [311] Clifford A. Shaffer. *A Practical Introduction to Data Structures and Algorithm Analysis*. Prentice Hall, second edition, 2001.
- [312] Jeffrey Shallit. Origins of the analysis of the Euclidean algorithm. *Historia Mathematica*, 21(4):401–419, 1994.
- [313] Michael I. Shamos and Dan Hoey. Geometric intersection problems. In *Proceedings of the 17th Annual Symposium on Foundations of Computer Science*, pages 208–215, 1976.
- [314] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers and Mathematics with Applications*, 7(1):67–72, 1981.
- [315] David B. Shmoys. Computing near-optimal solutions to combinatorial optimization problems. In William Cook, László Lovász, and Paul Seymour, editors, *Combinatorial Optimization*, volume 20 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, 1995.
- [316] Avi Shoshan and Uri Zwick. All pairs shortest paths in undirected graphs with integer weights. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, pages 605–614, 1999.
- [317] Michael Sipser. *Introduction to the Theory of Computation*. Thomson Course Technology, second edition, 2006.
- [318] Steven S. Skiena. *The Algorithm Design Manual*. Springer, second edition, 1998.
- [319] Daniel D. Sleator and Robert E. Tarjan. A data structure for dynamic trees. *Journal of Computer and System Sciences*, 26(3):362–391, 1983.

- [320] Daniel D. Sleator and Robert E. Tarjan. Self-adjusting binary search trees. *Journal of the ACM*, 32(3):652–686, 1985.
- [321] Joel Spencer. *Ten Lectures on the Probabilistic Method*, volume 64 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. Society for Industrial and Applied Mathematics, 1993.
- [322] Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *Journal of the ACM*, 51(3):385–463, 2004.
- [323] Gilbert Strang. *Introduction to Applied Mathematics*. Wellesley-Cambridge Press, 1986.
- [324] Gilbert Strang. *Linear Algebra and Its Applications*. Thomson Brooks/Cole, fourth edition, 2006.
- [325] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 14(3):354–356, 1969.
- [326] T. G. Szymanski. A special case of the maximal common subsequence problem. Technical Report TR-170, Computer Science Laboratory, Princeton University, 1975.
- [327] Robert E. Tarjan. Depth first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.
- [328] Robert E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.
- [329] Robert E. Tarjan. A class of algorithms which require nonlinear time to maintain disjoint sets. *Journal of Computer and System Sciences*, 18(2):110–127, 1979.
- [330] Robert E. Tarjan. *Data Structures and Network Algorithms*. Society for Industrial and Applied Mathematics, 1983.
- [331] Robert E. Tarjan. Amortized computational complexity. *SIAM Journal on Algebraic and Discrete Methods*, 6(2):306–318, 1985.
- [332] Robert E. Tarjan. Class notes: Disjoint set union. COS 423, Princeton University, 1999.
- [333] Robert E. Tarjan and Jan van Leeuwen. Worst-case analysis of set union algorithms. *Journal of the ACM*, 31(2):245–281, 1984.
- [334] George B. Thomas, Jr., Maurice D. Weir, Joel Hass, and Frank R. Giordano. *Thomas' Calculus*. Addison-Wesley, eleventh edition, 2005.
- [335] Mikkel Thorup. Faster deterministic sorting and priority queues in linear space. In *Proceedings of the 9th ACM-SIAM Symposium on Discrete Algorithms*, pages 550–555, 1998.
- [336] Mikkel Thorup. Undirected single-source shortest paths with positive integer weights in linear time. *Journal of the ACM*, 46(3):362–394, 1999.
- [337] Mikkel Thorup. On RAM priority queues. *SIAM Journal on Computing*, 30(1):86–109, 2000.
- [338] Richard Tolimieri, Myoung An, and Chao Lu. *Mathematics of Multidimensional Fourier Transform Algorithms*. Springer, second edition, 1997.
- [339] P. van Emde Boas. Preserving order in a forest in less than logarithmic time. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science*, pages 75–84, 1975.

- [340] P. van Emde Boas. Preserving order in a forest in less than logarithmic time and linear space. *Information Processing Letters*, 6(3):80–82, 1977.
- [341] P. van Emde Boas, R. Kaas, and E. Zijlstra. Design and implementation of an efficient priority queue. *Mathematical Systems Theory*, 10(1):99–127, 1976.
- [342] Jan van Leeuwen, editor. *Handbook of Theoretical Computer Science, Volume A: Algorithms and Complexity*. Elsevier Science Publishers and the MIT Press, 1990.
- [343] Charles Van Loan. *Computational Frameworks for the Fast Fourier Transform*. Society for Industrial and Applied Mathematics, 1992.
- [344] Robert J. Vanderbei. *Linear Programming: Foundations and Extensions*. Kluwer Academic Publishers, 1996.
- [345] Vijay V. Vazirani. *Approximation Algorithms*. Springer, 2001.
- [346] Rakesh M. Verma. General techniques for analyzing recursive algorithms with applications. *SIAM Journal on Computing*, 26(2):568–581, 1997.
- [347] Hao Wang and Bill Lin. Pipelined van Emde Boas tree: Algorithms, analysis, and applications. In *26th IEEE International Conference on Computer Communications*, pages 2471–2475, 2007.
- [348] Antony F. Ware. Fast approximate Fourier transforms for irregularly spaced data. *SIAM Review*, 40(4):838–856, 1998.
- [349] Stephen Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962.
- [350] Michael S. Waterman. *Introduction to Computational Biology, Maps, Sequences and Genomes*. Chapman & Hall, 1995.
- [351] Mark Allen Weiss. *Data Structures and Problem Solving Using C++*. Addison-Wesley, second edition, 2000.
- [352] Mark Allen Weiss. *Data Structures and Problem Solving Using Java*. Addison-Wesley, third edition, 2006.
- [353] Mark Allen Weiss. *Data Structures and Algorithm Analysis in C++*. Addison-Wesley, third edition, 2007.
- [354] Mark Allen Weiss. *Data Structures and Algorithm Analysis in Java*. Addison-Wesley, second edition, 2007.
- [355] Hassler Whitney. On the abstract properties of linear dependence. *American Journal of Mathematics*, 57(3):509–533, 1935.
- [356] Herbert S. Wilf. *Algorithms and Complexity*. A K Peters, second edition, 2002.
- [357] J. W. J. Williams. Algorithm 232 (HEAPSORT). *Communications of the ACM*, 7(6):347–348, 1964.
- [358] Shmuel Winograd. On the algebraic complexity of functions. In *Actes du Congrès International des Mathématiciens*, volume 3, pages 283–288, 1970.
- [359] Andrew C.-C. Yao. A lower bound to finding convex hulls. *Journal of the ACM*, 28(4):780–787, 1981.
- [360] Chee Yap. A real elementary approach to the master recurrence and generalizations. Unpublished manuscript. Available at <http://cs.nyu.edu/yap/papers/>, July 2008.

- [361] Yinyu Ye. *Interior Point Algorithms: Theory and Analysis*. John Wiley & Sons, 1997.
- [362] Daniel Zwillinger, editor. *CRC Standard Mathematical Tables and Formulae*. Chapman & Hall/CRC Press, 31st edition, 2003.

Index

This index uses the following conventions. Numbers are alphabetized as if spelled out; for example, “2-3-4 tree” is indexed as if it were “two-three-four tree.” When an entry refers to a place other than the main text, the page number is followed by a tag: ex. for exercise, pr. for problem, fig. for figure, and n. for footnote. A tagged page number often indicates the first page of an exercise or problem, which is not necessarily the page on which the reference actually appears.

- $\alpha(n)$, 574
- ϕ (golden ratio), 59, 108 pr.
- $\hat{\phi}$ (conjugate of the golden ratio), 59
- $\phi(n)$ (Euler’s phi function), 943
- $\rho(n)$ -approximation algorithm, 1106, 1123
- o -notation, 50–51, 64
- O -notation, 45 fig., 47–48, 64
- Q' -notation, 62 pr.
- $\tilde{O}$ -notation, 62 pr.
- ω -notation, 51
- Ω -notation, 45 fig., 48–49, 64
- $\tilde{\Omega}$ -notation, 62 pr.
- $\tilde{\Omega}$ -notation, 62 pr.
- Θ -notation, 44–47, 45 fig., 64
- $\tilde{\Theta}$ -notation, 62 pr.
- $\{ \}$ (set), 1158
- $\in$ (set member), 1158
- $\notin$ (not a set member), 1158
- $\emptyset$
 - (empty language), 1058
 - (empty set), 1158
- $\subseteq$ (subset), 1159
- $\subset$ (proper subset), 1159
- $:$ (such that), 1159
- $\cap$ (set intersection), 1159
- $\cup$ (set union), 1159
- $-$ (set difference), 1159
- $| |$
 - (flow value), 710
 - (length of a string), 986
 - (set cardinality), 1161
- $\times$
 - (Cartesian product), 1162
 - (cross product), 1016
- $\langle \rangle$
 - (sequence), 1166
 - (standard encoding), 1057
- $\binom{n}{k}$ (choose), 1185
- $\| \|$ (euclidean norm), 1222
- $!$ (factorial), 57
- $\lceil \rceil$ (ceiling), 54
- $\lfloor \rfloor$ (floor), 54
- $\sqrt{}$ (lower square root), 546
- $\sqrt{}$ (upper square root), 546
- $\sum$ (sum), 1145
- $\prod$ (product), 1148
- $\rightarrow$ (adjacency relation), 1169
- $\rightsquigarrow$ (reachability relation), 1170
- $\wedge$ (AND), 697, 1071
- $\neg$ (NOT), 1071
- $\vee$ (OR), 697, 1071
- $\oplus$ (group operator), 939
- $\otimes$ (convolution operator), 901

- * (closure operator), 1058
- | (divides relation), 927
- ⊥ (does-not-divide relation), 927
- $\equiv$ (equivalent modulo n), 54, 1165 ex.
- $\not\equiv$ (not equivalent modulo n), 54
- $[a]_n$ (equivalence class modulo n), 928
- $+_n$ (addition modulo n), 940
- $\cdot_n$ (multiplication modulo n), 940
- $(\frac{a}{p})$ (Legendre symbol), 982 pr.
- ε (empty string), 986, 1058
- $\sqsubset$ (prefix relation), 986
- $\sqsupset$ (suffix relation), 986
- $\succsim_x$ (above relation), 1022
- // (comment symbol), 21
- $\gg$ (much-greater-than relation), 574
- $\ll$ (much-less-than relation), 783
- $\leq_P$ (polynomial-time reducibility relation), 1067, 1077 ex.
- AA-tree, 338
- abelian group, 940
- ABOVE, 1024
- above relation ($\succsim_x$), 1022
- absent child, 1178
- absolutely convergent series, 1146
- absorption laws for sets, 1160
- abstract problem, 1054
- acceptable pair of integers, 972
- acceptance
 - by an algorithm, 1058
 - by a finite automaton, 996
- accepting state, 995
- accounting method, 456–459
 - for binary counters, 458
 - for dynamic tables, 465–466
 - for stack operations, 457–458, 458 ex.
- Ackermann's function, 585
- activity-selection problem, 415–422, 450
- acyclic graph, 1170
 - relation to matroids, 448 pr.
- add instruction, 23
- addition
 - of binary integers, 22 ex.
 - of matrices, 1220
 - modulo n ($+_n$), 940
 - of polynomials, 898
- additive group modulo n , 940
- addressing, open, *see* open-address hash table
- ADD-SUBARRAY, 805 pr.
- adjacency-list representation, 590
 - replaced by a hash table, 593 ex.
- adjacency-matrix representation, 591
- adjacency relation ($\rightarrow$), 1169
- adjacent vertices, 1169
- admissible edge, 749
- admissible network, 749–750
- adversary, 190
- aggregate analysis, 452–456
 - for binary counters, 454–455
 - for breadth-first search, 597
 - for depth-first search, 606
 - for Dijkstra's algorithm, 661
 - for disjoint-set data structures, 566–567, 568 ex.
 - for dynamic tables, 465
 - for Fibonacci heaps, 518, 522 ex.
 - for Graham's scan, 1036
 - for the Knuth-Morris-Pratt algorithm, 1006
 - for Prim's algorithm, 636
 - for rod-cutting, 367
 - for shortest paths in a dag, 655
 - for stack operations, 452–454
- aggregate flow, 863
- Akra-Bazzi method for solving a recurrence, 112–113
- algorithm, 5
 - correctness of, 6
 - origin of word, 42
 - running time of, 25
 - as a technology, 13
- Alice, 959
- ALLOCATE-NODE, 492
- ALLOCATE-OBJECT, 244
- allocation of objects, 243–244
- all-pairs shortest paths, 644, 684–707
 - in dynamic graphs, 707
 - in ϵ -dense graphs, 706 pr.
 - Floyd-Warshall algorithm for, 693–697, 706
 - Johnson's algorithm for, 700–706
 - by matrix multiplication, 686–693, 706–707
 - by repeated squaring, 689–691
- alphabet, 995, 1057
- $\alpha(n)$, 574
- amortized analysis, 451–478
 - accounting method of, 456–459
 - aggregate analysis, 367, 452–456

- for bit-reversal permutation, 472 pr.
- for breadth-first search, 597
- for depth-first search, 606
- for Dijkstra's algorithm, 661
- for disjoint-set data structures, 566–567, 568 ex., 572 ex., 575–581, 581–582 ex.
- for dynamic tables, 463–471
- for Fibonacci heaps, 509–512, 517–518, 520–522, 522 ex.
- for the generic push-relabel algorithm, 746
- for Graham's scan, 1036
- for the Knuth-Morris-Pratt algorithm, 1006
- for making binary search dynamic, 473 pr.
- potential method of, 459–463
- for restructuring red-black trees, 474 pr.
- for self-organizing lists with move-to-front, 476 pr.
- for shortest paths in a dag, 655
- for stacks on secondary storage, 502 pr.
- for weight-balanced trees, 473 pr.
- amortized cost
 - in the accounting method, 456
 - in aggregate analysis, 452
 - in the potential method, 459
- ancestor, 1176
 - least common, 584 pr.
- AND function ($\wedge$), 697, 1071
- AND gate, 1070
- and, in pseudocode, 22
- antiparallel edges, 711–712
- antisymmetric relation, 1164
- ANY-SEGMENTS-INTERSECT, 1025
- approximation
 - by least squares, 835–839
 - of summation by integrals, 1154–1156
- approximation algorithm, 10, 1105–1140
 - for bin packing, 1134 pr.
 - for MAX-CNF satisfiability, 1127 ex.
 - for maximum clique, 1111 ex., 1134 pr.
 - for maximum matching, 1135 pr.
 - for maximum spanning tree, 1137 pr.
 - for maximum-weight cut, 1127 ex.
 - for MAX-3-CNF satisfiability, 1123–1124, 1139
 - for minimum-weight vertex cover, 1124–1127, 1139
 - for parallel machine scheduling, 1136 pr.
 - randomized, 1123
 - for set cover, 1117–1122, 1139
 - for subset sum, 1128–1134, 1139
 - for traveling-salesman problem, 1111–1117, 1139
 - for vertex cover, 1108–1111, 1139
 - for weighted set cover, 1135 pr.
 - for 0-1 knapsack problem, 1137 pr., 1139
- approximation error, 836
- approximation ratio, 1106, 1123
- approximation scheme, 1107
- APPROX-MIN-WEIGHT-VC, 1126
- APPROX-SUBSET-SUM, 1131
- APPROX-TSP-TOUR, 1112
- APPROX-VERTEX-COVER, 1109
- arbitrage, 679 pr.
- arc, *see* edge
- argument of a function, 1166–1167
- arithmetic instructions, 23
- arithmetic, modular, 54, 939–946
- arithmetic series, 1146
- arithmetic with infinities, 650
- arm, 485
- array, 21
 - Monge, 110 pr.
 - passing as a parameter, 21
- articulation point, 621 pr.
- assignment
 - multiple, 21
 - satisfying, 1072, 1079
 - truth, 1072, 1079
- associative laws for sets, 1160
- associative operation, 939
- asymptotically larger, 52
- asymptotically nonnegative, 45
- asymptotically positive, 45
- asymptotically smaller, 52
- asymptotically tight bound, 45
- asymptotic efficiency, 43
- asymptotic lower bound, 48
- asymptotic notation, 43–53, 62 pr.
 - and graph algorithms, 588
 - and linearity of summations, 1146
- asymptotic upper bound, 47
- attribute of an object, 21
- augmentation of a flow, 716
- augmenting data structures, 339–355
- augmenting path, 719–720, 763 pr.
- authentication, 284 pr., 960–961, 964

- automaton
 - finite, 995
 - string-matching, 996–1002
- auxiliary hash function, 272
- auxiliary linear program, 886
- average-case running time, 28, 116
- AVL-INSERT, 333 pr.
- AVL tree, 333 pr., 337
- axioms, for probability, 1190
- babyface, 602 ex.
- back edge, 609, 613
- back substitution, 817
- BAD-SET-COVER-INSTANCE, 1122 ex.
- BALANCE, 333 pr.
- balanced search tree
 - AA-trees, 338
 - AVL trees, 333 pr., 337
 - B-trees, 484–504
 - k -neighbor trees, 338
 - red-black trees, 308–338
 - scapegoat trees, 338
 - splay trees, 338, 482
 - treaps, 333 pr., 338
 - 2-3-4 trees, 489, 503 pr.
 - 2-3 trees, 337, 504
 - weight-balanced trees, 338, 473 pr.
- balls and bins, 133–134, 1215 pr.
- base- a pseudoprime, 967
- base case, 65, 84
- base, in DNA, 391
- basic feasible solution, 866
- basic solution, 866
- basic variable, 855
- basis function, 835
- Bayes's theorem, 1194
- BELLMAN-FORD, 651
- Bellman-Ford algorithm, 651–655, 682
 - for all-pairs shortest paths, 684
 - in Johnson's algorithm, 702–704
 - and objective functions, 670 ex.
 - to solve systems of difference constraints, 668
 - Yen's improvement to, 678 pr.
- BELOW, 1024
- Bernoulli trial, 1201
 - and balls and bins, 133–134
 - and streaks, 135–139
- best-case running time, 29 ex., 49
- BFS, 595
- BIASED-RANDOM, 117 ex.
- biconnected component, 621 pr.
- big-oh notation, 45 fig., 47–48, 64
- big-omega notation, 45 fig., 48–49, 64
- bijective function, 1167
- binary character code, 428
- binary counter
 - analyzed by accounting method, 458
 - analyzed by aggregate analysis, 454–455
 - analyzed by potential method, 461–462
 - bit-reversed, 472 pr.
- binary entropy function, 1187
- binary gcd algorithm, 981 pr.
- binary heap, *see* heap
- binary relation, 1163
- binary search, 39 ex.
 - with fast insertion, 473 pr.
 - in insertion sort, 39 ex.
 - in multithreaded merging, 799–800
 - in searching B-trees, 499 ex.
- BINARY-SEARCH, 799
- binary search tree, 286–307
 - AA-trees, 338
 - AVL trees, 333 pr., 337
 - deletion from, 295–298, 299 ex.
 - with equal keys, 303 pr.
 - insertion into, 294–295
 - k -neighbor trees, 338
 - maximum key of, 291
 - minimum key of, 291
 - optimal, 397–404, 413
 - predecessor in, 291–292
 - querying, 289–294
 - randomly built, 299–303, 304 pr.
 - right-converting of, 314 ex.
 - scapegoat trees, 338
 - searching, 289–291
 - for sorting, 299 ex.
 - splay trees, 338
 - successor in, 291–292
 - and treaps, 333 pr.
 - weight-balanced trees, 338
 - see also* red-black tree
- binary-search-tree property, 287
 - in treaps, 333 pr.
 - vs. min-heap property, 289 ex.

- binary tree, 1177
 - full, 1178
 - number of different ones, 306 pr.
 - representation of, 246
 - superimposed upon a bit vector, 533–534
 - see also* binary search tree
- binomial coefficient, 1186–1187
- binomial distribution, 1203–1206
 - and balls and bins, 133
 - maximum value of, 1207 ex.
 - tails of, 1208–1215
- binomial expansion, 1186
- binomial heap, 527 pr.
- binomial tree, 527 pr.
- bin packing, 1134 pr.
- bipartite graph, 1172
 - corresponding flow network of, 732
 - d -regular, 736 ex.
 - and hypergraphs, 1173 ex.
- bipartite matching, 530, 732–736, 747 ex., 766
 - Hopcroft-Karp algorithm for, 763 pr.
- birthday paradox, 130–133, 142 ex.
- bisection of a tree, 1181 pr.
- bitonic euclidean traveling-salesman problem, 405 pr.
- bitonic sequence, 682 pr.
- bitonic tour, 405 pr.
- bit operation, 927
 - in Euclid's algorithm, 981 pr.
- bit-reversal permutation, 472 pr., 918
- BIT-REVERSE-COPY, 918
- bit-reversed binary counter, 472 pr.
- BIT-REVERSED-INCREMENT, 472 pr.
- bit vector, 255 ex., 532–536
- black-height, 309
- black vertex, 594, 603
- blocking flow, 765
- block structure in pseudocode, 20
- Bob, 959
- Boole's inequality, 1195 ex.
- boolean combinational circuit, 1071
- boolean combinational element, 1070
- boolean connective, 1079
- boolean formula, 1049, 1066 ex., 1079, 1086 ex.
- boolean function, 1187 ex.
- boolean matrix multiplication, 832 ex.
- Borůvka's algorithm, 641
- bottleneck spanning tree, 640 pr.
- bottleneck traveling-salesman problem, 1117 ex.
- bottom of a stack, 233
- BOTTOM-UP-CUT-ROD, 366
- bottom-up method, for dynamic programming, 365
- bound
 - asymptotically tight, 45
 - asymptotic lower, 48
 - asymptotic upper, 47
 - on binomial coefficients, 1186–1187
 - on binomial distributions, 1206
 - polylogarithmic, 57
 - on the tails of a binomial distribution, 1208–1215
 - see also* lower bounds
- boundary condition, in a recurrence, 67, 84
- boundary of a polygon, 1020 ex.
- bounding a summation, 1149–1156
- box, nesting, 678 pr.
- B^+ -tree, 488
- branching factor, in B-trees, 487
- branch instructions, 23
- breadth-first search, 594–602, 623
 - in maximum flow, 727–730, 766
 - and shortest paths, 597–600, 644
 - similarity to Dijkstra's algorithm, 662, 663 ex.
- breadth-first tree, 594, 600
- bridge, 621 pr.
- B^* -tree, 489 n.
- B-tree, 484–504
 - compared with red-black trees, 484, 490
 - creating, 492
 - deletion from, 499–502
 - full node in, 489
 - height of, 489–490
 - insertion into, 493–497
 - minimum degree of, 489
 - minimum key of, 497 ex.
 - properties of, 488–491
 - searching, 491–492
 - splitting a node in, 493–495
 - 2-3-4 trees, 489
- B-TREE-CREATE, 492
- B-TREE-DELETE, 499
- B-TREE-INSERT, 495

- B-TREE-INSERT-NONFULL, 496
- B-TREE-SEARCH, 492, 499 ex.
- B-TREE-SPLIT-CHILD, 494
- BUBBLESORT, 40 pr.
- bucket, 200
- bucket sort, 200–204
- BUCKET-SORT, 201
- BUILD-MAX-HEAP, 157
- BUILD-MAX-HEAP', 167 pr.
- BUILD-MIN-HEAP, 159
- butterfly operation, 915
- by**, in pseudocode, 21
- cache, 24, 449 pr.
- cache hit, 449 pr.
- cache miss, 449 pr.
- cache obliviousness, 504
- caching, off-line, 449 pr.
- call
 - in a multithreaded computation, 776
 - of a subroutine, 23, 25 n.
 - by value, 21
- call edge, 778
- cancellation lemma, 907
- cancellation of flow, 717
- canonical form for task scheduling, 444
- capacity
 - of a cut, 721
 - of an edge, 709
 - residual, 716, 719
 - of a vertex, 714 ex.
- capacity constraint, 709–710
- cardinality of a set ($| \cdot |$), 1161
- Carmichael number, 968, 975 ex.
- Cartesian product ($\times$), 1162
- Cartesian sum, 906 ex.
- cascading cut, 520
- CASCADING-CUT, 519
- Catalan numbers, 306 pr., 372
- ceiling function ($\lceil \cdot \rceil$), 54
 - in master theorem, 103–106
- ceiling instruction, 23
- certain event, 1190
- certificate
 - in a cryptosystem, 964
 - for verification algorithms, 1063
- CHAINED-HASH-DELETE, 258
- CHAINED-HASH-INSERT, 258
- CHAINED-HASH-SEARCH, 258
- chaining, 257–260, 283 pr.
- chain of a convex hull, 1038
- changing a key, in a Fibonacci heap, 529 pr.
- changing variables, in the substitution method, 86–87
- character code, 428
- chess-playing program, 790–791
- child
 - in a binary tree, 1178
 - in a multithreaded computation, 776
 - in a rooted tree, 1176
- child list in a Fibonacci heap, 507
- Chinese remainder theorem, 950–954, 983
- chip multiprocessor, 772
- chirp transform, 914 ex.
- choose $\binom{n}{k}$, 1185
- chord, 345 ex.
- Cilk, 774, 812
- Cilk++, 774, 812
- ciphertext, 960
- circuit
 - boolean combinational, 1071
 - depth of, 919
 - for fast Fourier transform, 919–920
- CIRCUIT-SAT, 1072
- circuit satisfiability, 1070–1077
- circular, doubly linked list with a sentinel, 239
- circular linked list, 236
 - see also* linked list
- class
 - complexity, 1059
 - equivalence, 1164
- classification of edges
 - in breadth-first search, 621 pr.
 - in depth-first search, 609–610, 611 ex.
 - in a multithreaded dag, 778–779
- clause, 1081–1082
- clean area, 208 pr.
- clique, 1086–1089, 1105
 - approximation algorithm for, 1111 ex., 1134 pr.
- CLIQUE, 1087
- closed interval, 348
- closed semiring, 707
- closest pair, finding, 1039–1044, 1047
- closest-point heuristic, 1117 ex.

- closure
 - group property, 939
 - of a language, 1058
 - operator (*), 1058
 - transitive, *see* transitive closure
- cluster
 - in a bit vector with a superimposed tree of constant height, 534
 - for parallel computing, 772
 - in proto van Emde Boas structures, 538
 - in van Emde Boas trees, 546
- clustering, 272
- CNF (conjunctive normal form), 1049, 1082
- CNF satisfiability, 1127 ex.
- coarsening leaves of recursion
 - in merge sort, 39 pr.
 - when recursively spawning, 787
- code, 428–429
 - Huffman, 428–437, 450
- codeword, 429
- codomain, 1166
- coefficient
 - binomial, 1186
 - of a polynomial, 55, 898
 - in slack form, 856
- coefficient representation, 900
 - and fast multiplication, 903–905
- cofactor, 1224
- coin changing, 446 pr.
- colinearity, 1016
- collision, 257
 - resolution by chaining, 257–260
 - resolution by open addressing, 269–277
- collision-resistant hash function, 964
- coloring, 1103 pr., 1180 pr.
- color, of a red-black-tree node, 308
- column-major order, 208 pr.
- column rank, 1223
- columnsort, 208 pr.
- column vector, 1218
- combination, 1185
- combinational circuit, 1071
- combinational element, 1070
- combine step, in divide-and-conquer, 30, 65
- comment, in pseudocode (//), 21
- commodity, 862
- common divisor, 929
 - greatest, *see* greatest common divisor
- common multiple, 939 ex.
- common subexpression, 915
- common subsequence, 7, 391
 - longest, 7, 390–397, 413
- commutative laws for sets, 1159
- commutative operation, 940
- COMPACTIFY-LIST, 245 ex.
- compact list, 250 pr.
- COMPACT-LIST-SEARCH, 250 pr.
- COMPACT-LIST-SEARCH', 251 pr.
- comparable line segments, 1022
- COMPARE-EXCHANGE, 208 pr.
- compare-exchange operation, 208 pr.
- comparison sort, 191
 - and binary search trees, 289 ex.
 - randomized, 205 pr.
 - and selection, 222
- compatible activities, 415
- compatible matrices, 371, 1221
- competitive analysis, 476 pr.
- complement
 - of an event, 1190
 - of a graph, 1090
 - of a language, 1058
 - Schur, 820, 834
 - of a set, 1160
- complementary slackness, 894 pr.
- complete graph, 1172
- complete k -ary tree, 1179
 - see also* heap
- completeness of a language, 1077 ex.
- complete step, 782
- completion time, 447 pr., 1136 pr.
- complexity class, 1059
 - co-NP, 1064
 - NP, 1049, 1064
 - NPC, 1050, 1069
 - P, 1049, 1055
- complexity measure, 1059
- complex numbers
 - inverting matrices of, 832 ex.
 - multiplication of, 83 ex.
- complex root of unity, 906
 - interpolation at, 912–913
- component
 - biconnected, 621 pr.
 - connected, 1170
 - strongly connected, 1170

- component graph, 617
- composite number, 928
 - witness to, 968
- composition, of multithreaded computations, 784 fig.
- computational depth, 812
- computational geometry, 1014–1047
- computational problem, 5–6
- computation dag, 777
- computation, multithreaded, 777
- COMPUTE-PREFIX-FUNCTION, 1006
- COMPUTE-TRANSITION-FUNCTION, 1001
- concatenation
 - of languages, 1058
 - of strings, 986
- concrete problem, 1055
- concurrency keywords, 774, 776, 785
- concurrency platform, 773
- conditional branch instruction, 23
- conditional independence, 1195 ex.
- conditional probability, 1192, 1194
- configuration, 1074
- conjugate of the golden ratio ($\hat{\phi}$), 59
- conjugate transpose, 832 ex.
- conjunctive normal form, 1049, 1082
- connected component, 1170
 - identified using depth-first search, 612 ex.
 - identified using disjoint-set data structures, 562–564
- CONNECTED-COMPONENTS, 563
- connected graph, 1170
- connective, 1079
- co-NP (complexity class), 1064
- conquer step, in divide-and-conquer, 30, 65
- conservation of flow, 709–710
- consistency
 - of literals, 1088
 - sequential, 779, 812
- CONSOLIDATE, 516
- consolidating a Fibonacci-heap root list, 513–517
- constraint, 851
 - difference, 665
 - equality, 670 ex., 852–853
 - inequality, 852–853
 - linear, 846
 - nonnegativity, 851, 853
 - tight, 865
 - violation of, 865
- constraint graph, 666–668
- contain, in a path, 1170
- continuation edge, 778
- continuous uniform probability distribution, 1192
- contraction
 - of a dynamic table, 467–471
 - of a matroid, 442
 - of an undirected graph by an edge, 1172
- control instructions, 23
- convergence property, 650, 672–673
- convergent series, 1146
- converting binary to decimal, 933 ex.
- convex combination of points, 1015
- convex function, 1199
- convex hull, 8, 1029–1039, 1046 pr.
- convex layers, 1044 pr.
- convex polygon, 1020 ex.
- convex set, 714 ex.
- convolution ($\otimes$), 901
- convolution theorem, 913
- copy instruction, 23
- correctness of an algorithm, 6
- corresponding flow network for bipartite matching, 732
- countably infinite set, 1161
- counter, *see* binary counter
- counting, 1183–1189
 - probabilistic, 143 pr.
- counting sort, 194–197
 - in radix sort, 198
- COUNTING-SORT, 195
- coupon collector's problem, 134
- cover
 - path, 761 pr.
 - by a subset, 1118
 - vertex, 1089, 1108, 1124–1127, 1139
- covertical, 1024
- CREATE-NEW-RS-VEB-TREE, 557 pr.
- credit, 456
- critical edge, 729
- critical path
 - of a dag, 657
 - of a multithreaded computation, 779
- cross a cut, 626
- cross edge, 609
- cross product ($\times$), 1016

- cryptosystem, 958–965, 983
- cubic spline, 840 pr.
- currency exchange, 390 ex., 679 pr.
- curve fitting, 835–839
- cut
 - capacity of, 721
 - cascading, 520
 - of a flow network, 720–724
 - minimum, 721, 731 ex.
 - net flow across, 720
 - of an undirected graph, 626
 - weight of, 1127 ex.
- CUT, 519
- CUT-ROD, 363
- cutting, in a Fibonacci heap, 519
- cycle of a graph, 1170
 - hamiltonian, 1049, 1061
 - minimum mean-weight, 680 pr.
 - negative-weight, *see* negative-weight cycle
 - and shortest paths, 646–647
- cyclic group, 955
- cyclic rotation, 1012 ex.
- cycling, of simplex algorithm, 875
- dag, *see* directed acyclic graph
- DAG-SHORTEST-PATHS, 655
- d -ary heap, 167 pr.
 - in shortest-paths algorithms, 706 pr.
- data-movement instructions, 23
- data-parallel model, 811
- data structure, 9, 229–355, 481–585
 - AA-trees, 338
 - augmentation of, 339–355
 - AVL trees, 333 pr., 337
 - binary search trees, 286–307
 - binomial heaps, 527 pr.
 - bit vectors, 255 ex., 532–536
 - B-trees, 484–504
 - deques, 236 ex.
 - dictionaries, 229
 - direct-address tables, 254–255
 - for disjoint sets, 561–585
 - for dynamic graphs, 483
 - dynamic sets, 229–231
 - dynamic trees, 482
 - exponential search trees, 212, 483
 - Fibonacci heaps, 505–530
 - fusion trees, 212, 483
 - hash tables, 256–261
 - heaps, 151–169
 - interval trees, 348–354
 - k -neighbor trees, 338
 - linked lists, 236–241
 - mergeable heap, 505
 - order-statistic trees, 339–345
 - persistent, 331 pr., 482
 - potential of, 459
 - priority queues, 162–166
 - proto van Emde Boas structures, 538–545
 - queues, 232, 234–235
 - radix trees, 304 pr.
 - red-black trees, 308–338
 - relaxed heaps, 530
 - rooted trees, 246–249
 - scapegoat trees, 338
 - on secondary storage, 484–487
 - skip lists, 338
 - splay trees, 338, 482
 - stacks, 232–233
 - treaps, 333 pr., 338
 - 2-3-4 heaps, 529 pr.
 - 2-3-4 trees, 489, 503 pr.
 - 2-3 trees, 337, 504
 - van Emde Boas trees, 531–560
 - weight-balanced trees, 338
- data type, 23
- deadline, 444
- deallocation of objects, 243–244
- decision by an algorithm, 1058–1059
- decision problem, 1051, 1054
 - and optimization problems, 1051
- decision tree, 192–193
- DECREASE-KEY, 162, 505
- decreasing a key
 - in Fibonacci heaps, 519–522
 - in 2-3-4 heaps, 529 pr.
- DECREMENT, 456 ex.
- degeneracy, 874
- degree
 - of a binomial-tree root, 527 pr.
 - maximum, of a Fibonacci heap, 509, 523–526
 - minimum, of a B-tree, 489
 - of a node, 1177
 - of a polynomial, 55, 898
 - of a vertex, 1169

- degree-bound, 898
- DELETE, 230, 505
- DELETE-LARGER-HALF, 463 ex.
- deletion
 - from binary search trees, 295–298, 299 ex.
 - from a bit vector with a superimposed binary tree, 534
 - from a bit vector with a superimposed tree of constant height, 535
 - from B-trees, 499–502
 - from chained hash tables, 258
 - from direct-address tables, 254
 - from dynamic tables, 467–471
 - from Fibonacci heaps, 522, 526 pr.
 - from heaps, 166 ex.
 - from interval trees, 349
 - from linked lists, 238
 - from open-address hash tables, 271
 - from order-statistic trees, 343–344
 - from proto van Emde Boas structures, 544
 - from queues, 234
 - from red-black trees, 323–330
 - from stacks, 232
 - from sweep-line statuses, 1024
 - from 2-3-4 heaps, 529 pr.
 - from van Emde Boas trees, 554–556
- DeMorgan's laws
 - for propositional logic, 1083
 - for sets, 1160, 1162 ex.
- dense graph, 589
- ϵ -dense, 706 pr.
- density
 - of prime numbers, 965–966
 - of a rod, 370 ex.
- dependence
 - and indicator random variables, 119
 - linear, 1223
 - see also* independence
- depth
 - average, of a node in a randomly built binary search tree, 304 pr.
 - of a circuit, 919
 - of a node in a rooted tree, 1177
 - of quicksort recursion tree, 178 ex.
 - of a stack, 188 pr.
- depth-determination problem, 583 pr.
- depth-first forest, 603
- depth-first search, 603–612, 623
 - in finding articulation points, bridges, and biconnected components, 621 pr.
 - in finding strongly connected components, 615–621, 623
 - in topological sorting, 612–615
- depth-first tree, 603
- deque, 236 ex.
- DEQUEUE, 235
- derivative of a series, 1147
- descendant, 1176
- destination vertex, 644
- det, *see* determinant
- determinacy race, 788
- determinant, 1224–1225
 - and matrix multiplication, 832 ex.
- deterministic algorithm, 123
 - multithreaded, 787
- DETERMINISTIC-SEARCH, 143 pr.
- DFS, 604
- DFS-VISIT, 604
- DFT (discrete Fourier transform), 9, 909
- diagonal matrix, 1218
 - LUP decomposition of, 827 ex.
- diameter of a tree, 602 ex.
- dictionary, 229
- difference constraints, 664–670
- difference equation, *see* recurrence
- difference of sets ($-$), 1159
 - symmetric, 763 pr.
- differentiation of a series, 1147
- digital signature, 960
- digraph, *see* directed graph
- DIJKSTRA, 658
- Dijkstra's algorithm, 658–664, 682
 - for all-pairs shortest paths, 684, 704
 - implemented with a Fibonacci heap, 662
 - implemented with a min-heap, 662
 - with integer edge weights, 664 ex.
 - in Johnson's algorithm, 702
 - similarity to breadth-first search, 662, 663 ex.
 - similarity to Prim's algorithm, 634, 662
- DIRECT-ADDRESS-DELETE, 254
- direct addressing, 254–255, 532–536
- DIRECT-ADDRESS-INSERT, 254
- DIRECT-ADDRESS-SEARCH, 254
- direct-address table, 254–255
- directed acyclic graph (dag), 1172

- and back edges, 613
- and component graphs, 617
- and hamiltonian paths, 1066 ex.
- longest simple path in, 404 pr.
- for representing a multithreaded computation, 777
- single-source shortest-paths algorithm for, 655–658
- topological sort of, 612–615, 623
- directed graph, 1168
 - all-pairs shortest paths in, 684–707
 - constraint graph, 666
 - Euler tour of, 623 pr., 1048
 - hamiltonian cycle of, 1049
 - and longest paths, 1048
 - path cover of, 761 pr.
 - PERT chart, 657, 657 ex.
 - semiconnected, 621 ex.
 - shortest path in, 643
 - single-source shortest paths in, 643–683
 - singly connected, 612 ex.
 - square of, 593 ex.
 - transitive closure of, 697
 - transpose of, 592 ex.
 - universal sink in, 593 ex.
 - see also* directed acyclic graph, graph, network
- directed segment, 1015–1017
- directed version of an undirected graph, 1172
- DIRECTION, 1018
- dirty area, 208 pr.
- DISCHARGE, 751
- discharge of an overflowing vertex, 751
- discovered vertex, 594, 603
- discovery time, in depth-first search, 605
- discrete Fourier transform, 9, 909
- discrete logarithm, 955
- discrete logarithm theorem, 955
- discrete probability distribution, 1191
- discrete random variable, 1196–1201
- disjoint-set data structure, 561–585
 - analysis of, 575–581, 581 ex.
 - in connected components, 562–564
 - in depth determination, 583 pr.
 - disjoint-set-forest implementation of, 568–572
 - in Kruskal’s algorithm, 631
 - linear-time special case of, 585
 - linked-list implementation of, 564–568
 - in off-line least common ancestors, 584 pr.
 - in off-line minimum, 582 pr.
 - in task scheduling, 448 pr.
- disjoint-set forest, 568–572
 - analysis of, 575–581, 581 ex.
 - rank properties of, 575, 581 ex.
 - see also* disjoint-set data structure
- disjoint sets, 1161
- disjunctive normal form, 1083
- disk, 1028 ex.
- disk drive, 485–487
 - see also* secondary storage
- DISK-READ, 487
- DISK-WRITE, 487
- distance
 - edit, 406 pr.
 - euclidean, 1039
 - L_m , 1044 ex.
 - Manhattan, 225 pr., 1044 ex.
 - of a shortest path, 597
- distributed memory, 772
- distribution
 - binomial, 1203–1206
 - continuous uniform, 1192
 - discrete, 1191
 - geometric, 1202–1203
 - of inputs, 116, 122
 - of prime numbers, 965
 - probability, 1190
 - sparse-hulled, 1046 pr.
 - uniform, 1191
- distributive laws for sets, 1160
- divergent series, 1146
- divide-and-conquer method, 30–35, 65
 - analysis of, 34–35
 - for binary search, 39 ex.
 - for conversion of binary to decimal, 933 ex.
 - for fast Fourier transform, 909–912
 - for finding the closest pair of points, 1040–1043
 - for finding the convex hull, 1030
 - for matrix inversion, 829–831
 - for matrix multiplication, 76–83, 792–797
 - for maximum-subarray problem, 68–75
 - for merge sort, 30–37, 797–805
 - for multiplication, 920 pr.

- for multithreaded matrix multiplication, 792–797
- for multithreaded merge sort, 797–805
- for quicksort, 170–190
- relation to dynamic programming, 359
- for selection, 215–224
- solving recurrences for, 83–106, 112–113
- for Strassen’s algorithm, 79–83
- divide instruction, 23
- divides relation ($\mid$), 927
- divide step, in divide-and-conquer, 30, 65
- division method, 263, 268–269 ex.
- division theorem, 928
- divisor, 927–928
 - common, 929
 - see also* greatest common divisor
- DNA, 6–7, 390–391, 406 pr.
- DNF (disjunctive normal form), 1083
- does-not-divide relation ($\nmid$), 927
- domain, 1166
- dominates relation, 1045 pr.
- double hashing, 272–274, 277 ex.
- doubly linked list, 236
 - see also* linked list
- downto**, in pseudocode, 21
- d -regular graph, 736 ex.
- duality, 879–886, 895 pr.
 - weak, 880–881, 886 ex.
- dual linear program, 879
- dummy key, 397
- dynamic graph, 562 n.
 - all-pairs shortest paths algorithms for, 707
 - data structures for, 483
 - minimum-spanning-tree algorithm for, 637 ex.
 - transitive closure of, 705 pr., 707
- dynamic multithreaded algorithm, *see* multithreaded algorithm
- dynamic multithreading, 773
- dynamic order statistics, 339–345
- dynamic-programming method, 359–413
 - for activity selection, 421 ex.
 - for all-pairs shortest paths, 686–697
 - for bitonic euclidean traveling-salesman problem, 405 pr.
 - bottom-up, 365
 - for breaking a string, 410 pr.
 - compared with greedy algorithms, 381, 390 ex., 418, 423–427
 - for edit distance, 406 pr.
 - elements of, 378–390
 - for Floyd-Warshall algorithm, 693–697
 - for inventory planning, 411 pr.
 - for longest common subsequence, 390–397
 - for longest palindrome subsequence, 405 pr.
 - for longest simple path in a weighted directed acyclic graph, 404 pr.
 - for matrix-chain multiplication, 370–378
 - and memoization, 387–389
 - for optimal binary search trees, 397–404
 - optimal substructure in, 379–384
 - overlapping subproblems in, 384–386
 - for printing neatly, 405 pr.
 - reconstructing an optimal solution in, 387
 - relation to divide-and-conquer, 359
 - for rod-cutting, 360–370
 - for seam carving, 409 pr.
 - for signing free agents, 411 pr.
 - top-down with memoization, 365
 - for transitive closure, 697–699
 - for Viterbi algorithm, 408 pr.
 - for 0-1 knapsack problem, 427 ex.
- dynamic set, 229–231
 - see also* data structure
- dynamic table, 463–471
 - analyzed by accounting method, 465–466
 - analyzed by aggregate analysis, 465
 - analyzed by potential method, 466–471
 - load factor of, 463
- dynamic tree, 482
- e , 55
- $E[]$ (expected value), 1197
- early-first form, 444
- early task, 444
- edge, 1168
 - admissible, 749
 - antiparallel, 711–712
 - attributes of, 592
 - back, 609
 - bridge, 621 pr.
 - call, 778
 - capacity of, 709
 - classification in breadth-first search, 621 pr.
 - classification in depth-first search, 609–610

- continuation, 778
- critical, 729
- cross, 609
- forward, 609
- inadmissible, 749
- light, 626
- negative-weight, 645–646
- residual, 716
- return, 779
- safe, 626
- saturated, 739
- spawn, 778
- tree, 601, 603, 609
- weight of, 591
- edge connectivity, 731 ex.
- edge set, 1168
- edit distance, 406 pr.
- Edmonds-Karp algorithm, 727–730
- elementary event, 1189
- elementary insertion, 465
- element of a set ($\in$), 1158
- ellipsoid algorithm, 850, 897
- elliptic-curve factorization method, 984
- elseif**, in pseudocode, 20 n.
- else**, in pseudocode, 20
- empty language ($\emptyset$), 1058
- empty set ($\emptyset$), 1158
- empty set laws, 1159
- empty stack, 233
- empty string (ϵ), 986, 1058
- empty tree, 1178
- encoding of problem instances, 1055–1057
- endpoint
 - of an interval, 348
 - of a line segment, 1015
- ENQUEUE, 235
- entering a vertex, 1169
- entering variable, 867
- entropy function, 1187
- ϵ -dense graph, 706 pr.
- ϵ -universal hash function, 269 ex.
- equality
 - of functions, 1166
 - linear, 845
 - of sets, 1158
- equality constraint, 670 ex., 852
 - and inequality constraints, 853
 - tight, 865
 - violation of, 865
- equation
 - and asymptotic notation, 49–50
 - normal, 837
 - recurrence, *see* recurrence
- equivalence class, 1164
 - modulo n ($[a]_n$), 928
- equivalence, modular ($\equiv$), 54, 1165 ex.
- equivalence relation, 1164
 - and modular equivalence, 1165 ex.
- equivalent linear programs, 852
- error**, in pseudocode, 22
- escape problem, 760 pr.
- EUCLID, 935
- Euclid's algorithm, 933–939, 981 pr., 983
- euclidean distance, 1039
- euclidean norm ($\| \cdot \|$), 1222
- Euler's constant, 943
- Euler's phi function, 943
- Euler's theorem, 954, 975 ex.
- Euler tour, 623 pr., 1048
 - and hamiltonian cycles, 1048
- evaluation of a polynomial, 41 pr., 900, 905 ex.
 - derivatives of, 922 pr.
 - at multiple points, 923 pr.
- event, 1190
- event point, 1023
- event-point schedule, 1023
- EXACT-SUBSET-SUM, 1129
- excess flow, 736
- exchange property, 437
- exclusion and inclusion, 1163 ex.
- execute a subroutine, 25 n.
- expansion of a dynamic table, 464–467
- expectation, *see* expected value
- expected running time, 28, 117
- expected value, 1197–1199
 - of a binomial distribution, 1204
 - of a geometric distribution, 1202
 - of an indicator random variable, 118
- explored vertex, 605
- exponential function, 55–56
- exponential height, 300
- exponential search tree, 212, 483
- exponential series, 1147
- exponentiation instruction, 24
- exponentiation, modular, 956
- EXTENDED-BOTTOM-UP-CUT-ROD, 369

- EXTENDED-EUCLID, 937
- EXTEND-SHORTEST-PATHS, 688
- extension of a set, 438
- exterior of a polygon, 1020 ex.
- external node, 1176
- external path length, 1180 ex.
- extracting the maximum key
 - from d -ary heaps, 167 pr.
 - from max-heaps, 163
- extracting the minimum key
 - from Fibonacci heaps, 512–518
 - from 2-3-4 heaps, 529 pr.
 - from Young tableaux, 167 pr.
- EXTRACT-MAX, 162–163
- EXTRACT-MIN, 162, 505
- factor, 928
 - twiddle, 912
- factorial function (!), 57–58
- factorization, 975–980, 984
 - unique, 931
- failure, in a Bernoulli trial, 1201
- fair coin, 1191
- fan-out, 1071
- Farkas's lemma, 895 pr.
- farthest-pair problem, 1030
- FASTER-ALL-PAIRS-SHORTEST-PATHS, 691, 692 ex.
- fast Fourier transform (FFT), 898–925
 - circuit for, 919–920
 - iterative implementation of, 915–918
 - multidimensional, 921 pr.
 - multithreaded algorithm for, 804 ex.
 - recursive implementation of, 909–912
 - using modular arithmetic, 923 pr.
- feasibility problem, 665, 894 pr.
- feasible linear program, 851
- feasible region, 847
- feasible solution, 665, 846, 851
- Fermat's theorem, 954
- FFT, *see* fast Fourier transform
- FFTW, 924
- FIB, 775
- FIB-HEAP-CHANGE-KEY, 529 pr.
- FIB-HEAP-DECREASE-KEY, 519
- FIB-HEAP-DELETE, 522
- FIB-HEAP-EXTRACT-MIN, 513
- FIB-HEAP-INSERT, 510
- FIB-HEAP-LINK, 516
- FIB-HEAP-PRUNE, 529 pr.
- FIB-HEAP-UNION, 512
- Fibonacci heap, 505–530
 - changing a key in, 529 pr.
 - compared with binary heaps, 506–507
 - creating, 510
 - decreasing a key in, 519–522
 - deletion from, 522, 526 pr.
 - in Dijkstra's algorithm, 662
 - extracting the minimum key from, 512–518
 - insertion into, 510–511
 - in Johnson's algorithm, 704
 - maximum degree of, 509, 523–526
 - minimum key of, 511
 - potential function for, 509
 - in Prim's algorithm, 636
 - pruning, 529 pr.
 - running times of operations on, 506 fig.
 - uniting, 511–512
- Fibonacci numbers, 59–60, 108 pr., 523
 - computation of, 774–780, 981 pr.
- FIFO (first-in, first-out), 232
 - see also* queue
- final-state function, 996
- final strand, 779
- FIND-DEPTH, 583 pr.
- FIND-MAX-CROSSING-SUBARRAY, 71
- FIND-MAXIMUM-SUBARRAY, 72
- find path, 569
- FIND-SET, 562
 - disjoint-set-forest implementation of, 571, 585
 - linked-list implementation of, 564
- finished vertex, 603
- finishing time, in depth-first search, 605
 - and strongly connected components, 618
- finish time, in activity selection, 415
- finite automaton, 995
 - for string matching, 996–1002
- FINITE-AUTOMATON-MATCHER, 999
- finite group, 940
- finite sequence, 1166
- finite set, 1161
- first-fit heuristic, 1134 pr.
- first-in, first-out, 232
 - see also* queue
- fixed-length code, 429

- floating-point data type, 23
- floor function ($\lfloor \rfloor$), 54
 - in master theorem, 103–106
- floor instruction, 23
- flow, 709–714
 - aggregate, 863
 - augmentation of, 716
 - blocking, 765
 - cancellation of, 717
 - excess, 736
 - integer-valued, 733
 - net, across a cut, 720
 - value of, 710
- flow conservation, 709–710
- flow network, 709–714
 - corresponding to a bipartite graph, 732
 - cut of, 720–724
 - with multiple sources and sinks, 712
- FLOYD-WARSHALL, 695
- FLOYD-WARSHALL', 699 ex.
- Floyd-Warshall algorithm, 693–697, 699–700 ex., 706
 - multithreaded, 797 ex.
- FORD-FULKERSON, 724
- Ford-Fulkerson method, 714–731, 765
- FORD-FULKERSON-METHOD, 715
- forest, 1172–1173
 - depth-first, 603
 - disjoint-set, 568–572
- for**, in pseudocode, 20–21
 - and loop invariants, 19 n.
- formal power series, 108 pr.
- formula satisfiability, 1079–1081, 1105
- forward edge, 609
- forward substitution, 816–817
- Fourier transform, *see* discrete Fourier transform, fast Fourier transform
- fractional knapsack problem, 426, 428 ex.
- free agent, 411 pr.
- freeing of objects, 243–244
- free list, 243
- FREE-OBJECT, 244
- free tree, 1172–1176
- frequency domain, 898
- full binary tree, 1178, 1180 ex.
 - relation to optimal code, 430
- full node, 489
- full rank, 1223
- full walk of a tree, 1114
- fully parenthesized matrix-chain product, 370
- fully polynomial-time approximation scheme, 1107
 - for subset sum, 1128–1134, 1139
- function, 1166–1168
 - Ackermann's, 585
 - basis, 835
 - convex, 1199
 - final-state, 996
 - hash, *see* hash function
 - linear, 26, 845
 - objective, 664, 847, 851
 - potential, 459
 - prefix, 1003–1004
 - quadratic, 27
 - reduction, 1067
 - suffix, 996
 - transition, 995, 1001–1002, 1012 ex.
- functional iteration, 58
- fundamental theorem of linear programming, 892
- furthest-in-future strategy, 449 pr.
- fusion tree, 212, 483
- fuzzy sorting, 189 pr.
- Gabow's scaling algorithm for single-source shortest paths, 679 pr.
- gap character, 989 ex., 1002 ex.
- gap heuristic, 760 ex., 766
- garbage collection, 151, 243
- gate, 1070
- Gaussian elimination, 819, 842
- gcd, *see* greatest common divisor
- general number-field sieve, 984
- generating function, 108 pr.
- generator
 - of a subgroup, 944
 - of $\mathbb{Z}_n^*$, 955
- GENERIC-MST, 626
- GENERIC-PUSH-RELABEL, 741
- generic push-relabel algorithm, 740–748
- geometric distribution, 1202–1203
 - and balls and bins, 134
- geometric series, 1147
- geometry, computational, 1014–1047
- $GF(2)$, 1227 pr.
- gift wrapping, 1037, 1047

- global variable, 21
- Goldberg's algorithm, *see* push-relabel algorithm
- golden ratio (ϕ), 59, 108 pr.
- gossiping, 478
- GRAFT, 583 pr.
- Graham's scan, 1030–1036, 1047
- GRAHAM-SCAN, 1031
- graph, 1168–1173
 - adjacency-list representation of, 590
 - adjacency-matrix representation of, 591
 - algorithms for, 587–766
 - and asymptotic notation, 588
 - attributes of, 588, 592
 - breadth-first search of, 594–602, 623
 - coloring of, 1103 pr.
 - complement of, 1090
 - component, 617
 - constraint, 666–668
 - dense, 589
 - depth-first search of, 603–612, 623
 - dynamic, 562 n.
 - ϵ -dense, 706 pr.
 - hamiltonian, 1061
 - incidence matrix of, 448 pr., 593 ex.
 - interval, 422 ex.
 - nonhamiltonian, 1061
 - shortest path in, 597
 - singly connected, 612 ex.
 - sparse, 589
 - static, 562 n.
 - subproblem, 367–368
 - tour of, 1096
 - weighted, 591
 - see also* directed acyclic graph, directed graph, flow network, undirected graph, tree
- graphic matroid, 437–438, 642
- GRAPH-ISOMORPHISM, 1065 ex.
- gray vertex, 594, 603
- greatest common divisor (gcd), 929–930, 933 ex.
 - binary gcd algorithm for, 981 pr.
 - Euclid's algorithm for, 933–939, 981 pr., 983
 - with more than two arguments, 939 ex.
 - recursion theorem for, 934
- greedoid, 450
- GREEDY, 440
- GREEDY-ACTIVITY-SELECTOR, 421
- greedy algorithm, 414–450
 - for activity selection, 415–422
 - for coin changing, 446 pr.
 - compared with dynamic programming, 381, 390 ex., 418, 423–427
 - Dijkstra's algorithm, 658–664
 - elements of, 423–428
 - for fractional knapsack problem, 426
 - greedy-choice property in, 424–425
 - for Huffman code, 428–437
 - Kruskal's algorithm, 631–633
 - and matroids, 437–443
 - for minimum spanning tree, 631–638
 - for multithreaded scheduling, 781–783
 - for off-line caching, 449 pr.
 - optimal substructure in, 425
 - Prim's algorithm, 634–636
 - for set cover, 1117–1122, 1139
 - for task scheduling, 443–446, 447–448 pr.
 - on a weighted matroid, 439–442
 - for weighted set cover, 1135 pr.
- greedy-choice property, 424–425
 - of activity selection, 417–418
 - of Huffman codes, 433–434
 - of a weighted matroid, 441
- greedy scheduler, 782
- GREEDY-SET-COVER, 1119
- grid, 760 pr.
- group, 939–946
 - cyclic, 955
 - operator ($\oplus$), 939
- guessing the solution, in the substitution method, 84–85
- half 3-CNF satisfiability, 1101 ex.
- half-open interval, 348
- Hall's theorem, 735 ex.
- halting problem, 1048
- halving lemma, 908
- HAM-CYCLE, 1062
- hamiltonian cycle, 1049, 1061, 1091–1096, 1105
- hamiltonian graph, 1061
- hamiltonian path, 1066 ex., 1101 ex.
- HAM-PATH, 1066 ex.
- handle, 163, 507
- handshaking lemma, 1172 ex.

- harmonic number, 1147, 1153–1154
- harmonic series, 1147, 1153–1154
- HASH-DELETE, 277 ex.
- hash function, 256, 262–269
 - auxiliary, 272
 - collision-resistant, 964
 - division method for, 263, 268–269 ex.
 - ϵ -universal, 269 ex.
 - multiplication method for, 263–264
 - universal, 265–268
- hashing, 253–285
 - with chaining, 257–260, 283 pr.
 - double, 272–274, 277 ex.
 - k -universal, 284 pr.
 - in memoization, 365, 387
 - with open addressing, 269–277
 - perfect, 277–282, 285
 - to replace adjacency lists, 593 ex.
 - universal, 265–268
- HASH-INSERT, 270, 277 ex.
- HASH-SEARCH, 271, 277 ex.
- hash table, 256–261
 - dynamic, 471 ex.
 - secondary, 278
 - see also* hashing
- hash value, 256
- hat-check problem, 122 ex.
- head
 - in a disk drive, 485
 - of a linked list, 236
 - of a queue, 234
- heap, 151–169
 - analyzed by potential method, 462 ex.
 - binomial, 527 pr.
 - building, 156–159, 166 pr.
 - compared with Fibonacci heaps, 506–507
 - d -ary, 167 pr., 706 pr.
 - deletion from, 166 ex.
 - in Dijkstra's algorithm, 662
 - extracting the maximum key from, 163
 - Fibonacci, *see* Fibonacci heap
 - as garbage-collected storage, 151
 - height of, 153
 - in Huffman's algorithm, 433
 - to implement a mergeable heap, 506
 - increasing a key in, 163–164
 - insertion into, 164
 - in Johnson's algorithm, 704
 - max-heap, 152
 - maximum key of, 163
 - mergeable, *see* mergeable heap
 - min-heap, 153
 - in Prim's algorithm, 636
 - as a priority queue, 162–166
 - relaxed, 530
 - running times of operations on, 506 fig.
 - and treaps, 333 pr.
 - 2-3-4, 529 pr.
- HEAP-DECREASE-KEY, 165 ex.
- HEAP-DELETE, 166 ex.
- HEAP-EXTRACT-MAX, 163
- HEAP-EXTRACT-MIN, 165 ex.
- HEAP-INCREASE-KEY, 164
- HEAP-MAXIMUM, 163
- HEAP-MINIMUM, 165 ex.
- heap property, 152
 - maintenance of, 154–156
 - vs. binary-search-tree property, 289 ex.
- heapsort, 151–169
- HEAPSORT, 160
- heel, 602 ex.
- height
 - of a binomial tree, 527 pr.
 - black-, 309
 - of a B-tree, 489–490
 - of a d -ary heap, 167 pr.
 - of a decision tree, 193
 - exponential, 300
 - of a heap, 153
 - of a node in a heap, 153, 159 ex.
 - of a node in a tree, 1177
 - of a red-black tree, 309
 - of a tree, 1177
- height-balanced tree, 333 pr.
- height function, in push-relabel algorithms, 738
- hereditary family of subsets, 437
- Hermitian matrix, 832 ex.
- high endpoint of an interval, 348
- high function, 537, 546
- HIRE-ASSISTANT, 115
- hiring problem, 114–115, 123–124, 145
 - on-line, 139–141
 - probabilistic analysis of, 120–121
- hit
 - cache, 449 pr.
 - spurious, 991

- HOARE-PARTITION, 185 pr.
- HOPCROFT-KARP, 764 pr.
- Hopcroft-Karp bipartite matching algorithm, 763 pr.
- horizontal ray, 1021 ex.
- Horner's rule, 41 pr., 900
 - in the Rabin-Karp algorithm, 990
- HUFFMAN, 431
- Huffman code, 428–437, 450
- hull, convex, 8, 1029–1039, 1046 pr.
- Human Genome Project, 6
- hyperedge, 1172
- hypergraph, 1172
 - and bipartite graphs, 1173 ex.
- ideal parallel computer, 779
- idempotency laws for sets, 1159
- identity, 939
- identity matrix, 1218
- if**, in pseudocode, 20
- image, 1167
- image compression, 409 pr., 413
- inadmissible edge, 749
- incidence, 1169
- incidence matrix
 - and difference constraints, 666
 - of a directed graph, 448 pr., 593 ex.
 - of an undirected graph, 448 pr.
- inclusion and exclusion, 1163 ex.
- incomplete step, 782
- INCREASE-KEY, 162
- increasing a key, in a max-heap, 163–164
- INCREMENT, 454
- incremental design method, 29
 - for finding the convex hull, 1030
- in-degree, 1169
- indentation in pseudocode, 20
- independence
 - of events, 1192–1193, 1195 ex.
 - of random variables, 1197
 - of subproblems in dynamic programming, 383–384
- independent family of subsets, 437
- independent set, 1101 pr.
 - of tasks, 444
- independent strands, 789
- index function, 537, 546
- index of an element of $\mathbb{Z}_n^*$, 955
- indicator random variable, 118–121
 - in analysis of expected height of a randomly built binary search tree, 300–303
 - in analysis of inserting into a treap, 333 pr.
 - in analysis of streaks, 138–139
 - in analysis of the birthday paradox, 132–133
 - in approximation algorithm for MAX-3-CNF satisfiability, 1124
 - in bounding the right tail of the binomial distribution, 1212–1213
 - in bucket sort analysis, 202–204
 - expected value of, 118
 - in hashing analysis, 259–260
 - in hiring-problem analysis, 120–121
 - and linearity of expectation, 119
 - in quicksort analysis, 182–184, 187 pr.
 - in randomized-selection analysis, 217–219, 226 pr.
 - in universal-hashing analysis, 265–266
- induced subgraph, 1171
- inequality constraint, 852
 - and equality constraints, 853
- inequality, linear, 846
- infeasible linear program, 851
- infeasible solution, 851
- infinite sequence, 1166
- infinite set, 1161
- infinite sum, 1145
- infinity, arithmetic with, 650
- INITIALIZE-PREFLOW, 740
- INITIALIZE-SIMPLEX, 871, 887
- INITIALIZE-SINGLE-SOURCE, 648
- initial strand, 779
- injective function, 1167
- inner product, 1222
- inorder tree walk, 287, 293 ex., 342
- INORDER-TREE-WALK, 288
- in-place sorting, 17, 148, 206 pr.
- input
 - to an algorithm, 5
 - to a combinational circuit, 1071
 - distribution of, 116, 122
 - to a logic gate, 1070
 - size of, 25
- input alphabet, 995
- INSERT, 162, 230, 463 ex., 505
- insertion
 - into binary search trees, 294–295

- into a bit vector with a superimposed binary tree, 534
- into a bit vector with a superimposed tree of constant height, 534
- into B-trees, 493–497
- into chained hash tables, 258
- into d -ary heaps, 167 pr.
- into direct-address tables, 254
- into dynamic tables, 464–467
- elementary, 465
- into Fibonacci heaps, 510–511
- into heaps, 164
- into interval trees, 349
- into linked lists, 237–238
- into open-address hash tables, 270
- into order-statistic trees, 343
- into proto van Emde Boas structures, 544
- into queues, 234
- into red-black trees, 315–323
- into stacks, 232
- into sweep-line statuses, 1024
- into treaps, 333 pr.
- into 2-3-4 heaps, 529 pr.
- into van Emde Boas trees, 552–554
- into Young tableaux, 167 pr.
- insertion sort, 12, 16–20, 25–27
 - in bucket sort, 201–204
 - compared with merge sort, 14 ex.
 - compared with quicksort, 178 ex.
 - decision tree for, 192 fig.
 - in merge sort, 39 pr.
 - in quicksort, 185 ex.
 - using binary search, 39 ex.
- INSERTION-SORT, 18, 26, 208 pr.
- instance
 - of an abstract problem, 1051, 1054
 - of a problem, 5
- instructions of the RAM model, 23
- integer data type, 23
- integer linear programming, 850, 895 pr., 1101 ex.
- integers ($\mathbb{Z}$), 1158
- integer-valued flow, 733
- integrality theorem, 734
- integral, to approximate summations, 1154–1156
- integration of a series, 1147
- interior of a polygon, 1020 ex.
- interior-point method, 850, 897
- intermediate vertex, 693
- internal node, 1176
- internal path length, 1180 ex.
- interpolation by a cubic spline, 840 pr.
- interpolation by a polynomial, 901, 906 ex.
 - at complex roots of unity, 912–913
- intersection
 - of chords, 345 ex.
 - determining, for a set of line segments, 1021–1029, 1047
 - determining, for two line segments, 1017–1019
 - of languages, 1058
 - of sets ($\cap$), 1159
- interval, 348
 - fuzzy sorting of, 189 pr.
- INTERVAL-DELETE, 349
- interval graph, 422 ex.
- INTERVAL-INSERT, 349
- INTERVAL-SEARCH, 349, 351
- INTERVAL-SEARCH-EXACTLY, 354 ex.
- interval tree, 348–354
- interval trichotomy, 348
- intractability, 1048
- invalid shift, 985
- inventory planning, 411 pr.
- inverse
 - of a bijective function, 1167
 - in a group, 940
 - of a matrix, 827–831, 842, 1223, 1225 ex.
 - multiplicative, modulo n , 949
- inversion
 - in a self-organizing list, 476 pr.
 - in a sequence, 41 pr., 122 ex., 345 ex.
- inverter, 1070
- invertible matrix, 1223
- isolated vertex, 1169
- isomorphic graphs, 1171
- iterated function, 63 pr.
- iterated logarithm function, 58–59
- ITERATIVE-FFT, 917
- ITERATIVE-TREE-SEARCH, 291
- iter function, 577
- Jarvis's march, 1037–1038, 1047
- Jensen's inequality, 1199
- JOHNSON, 704

- Johnson's algorithm, 700–706
- joining
 - of red-black trees, 332 pr.
 - of 2-3-4 trees, 503 pr.
- joint probability density function, 1197
- Josephus permutation, 355 pr.
- Karmarkar's algorithm, 897
- Karp's minimum mean-weight cycle algorithm, 680 pr.
- k -ary tree, 1179
- k -CNF, 1049
- k -coloring, 1103 pr., 1180 pr.
- k -combination, 1185
- k -conjunctive normal form, 1049
- kernel of a polygon, 1038 ex.
- key, 16, 147, 162, 229
 - dummy, 397
 - interpreted as a natural number, 263
 - median, of a B-tree node, 493
 - public, 959, 962
 - secret, 959, 962
 - static, 277
- keywords, in pseudocode, 20–22
 - multithreaded, 774, 776–777, 785–786
- "killer adversary" for quicksort, 190
- Kirchhoff's current law, 708
- Kleene star (*), 1058
- KMP algorithm, 1002–1013
- KMP-MATCHER, 1005
- knapsack problem
 - fractional, 426, 428 ex.
 - 0-1, 425, 427 ex., 1137 pr., 1139
- k -neighbor tree, 338
- knot, of a spline, 840 pr.
- Knuth-Morris-Pratt algorithm, 1002–1013
- k -permutation, 126, 1184
- Kraft inequality, 1180 ex.
- Kruskal's algorithm, 631–633, 642
 - with integer edge weights, 637 ex.
- k -sorted, 207 pr.
- k -string, 1184
- k -subset, 1161
- k -substring, 1184
- k th power, 933 ex.
- k -universal hashing, 284 pr.
- Lagrange's formula, 902
- Lagrange's theorem, 944
- Lamé's theorem, 936
- language, 1057
 - completeness of, 1077 ex.
 - proving NP-completeness of, 1078–1079
 - verification of, 1063
- last-in, first-out, 232
 - see also* stack
- late task, 444
- layers
 - convex, 1044 pr.
 - maximal, 1045 pr.
- LCA, 584 pr.
- lcm (least common multiple), 939 ex.
- LCS, 7, 390–397, 413
- LCS-LENGTH, 394
- leading submatrix, 833, 839 ex.
- leaf, 1176
- least common ancestor, 584 pr.
- least common multiple, 939 ex.
- least-squares approximation, 835–839
- leaving a vertex, 1169
- leaving variable, 867
- LEFT, 152
- left child, 1178
- left-child, right-sibling representation, 246, 249 ex.
- LEFT-ROTATE, 313, 353 ex.
- left rotation, 312
- left spine, 333 pr.
- left subtree, 1178
- Legendre symbol ($\frac{a}{p}$), 982 pr.
- length
 - of a path, 1170
 - of a sequence, 1166
 - of a spine, 333 pr.
 - of a string, 986, 1184
- level
 - of a function, 573
 - of a tree, 1177
- level function, 576
- lexicographically less than, 304 pr.
- lexicographic sorting, 304 pr.
- lg (binary logarithm), 56
- lg* (iterated logarithm function), 58–59
- lg^k (exponentiation of logarithms), 56
- lg lg (composition of logarithms), 56
- LIFO (last-in, first-out), 232

- see also* stack
- light edge, 626
- linear constraint, 846
- linear dependence, 1223
- linear equality, 845
- linear equations
 - solving modular, 946–950
 - solving systems of, 813–827
 - solving tridiagonal systems of, 840 pr.
- linear function, 26, 845
- linear independence, 1223
- linear inequality, 846
- linear-inequality feasibility problem, 894 pr.
- linearity of expectation, 1198
 - and indicator random variables, 119
- linearity of summations, 1146
- linear order, 1165
- linear permutation, 1229 pr.
- linear probing, 272
- linear programming, 7, 843–897
 - algorithms for, 850
 - applications of, 849
 - duality in, 879–886
 - ellipsoid algorithm for, 850, 897
 - finding an initial solution in, 886–891
 - fundamental theorem of, 892
 - interior-point methods for, 850, 897
 - Karmarkar’s algorithm for, 897
 - and maximum flow, 860–861
 - and minimum-cost circulation, 896 pr.
 - and minimum-cost flow, 861–862
 - and minimum-cost multicommodity flow, 864 ex.
 - and multicommodity flow, 862–863
 - simplex algorithm for, 864–879, 896
 - and single-pair shortest path, 859–860
 - and single-source shortest paths, 664–670, 863 ex.
 - slack form for, 854–857
 - standard form for, 850–854
 - see also* integer linear programming, 0-1 integer programming
- linear-programming relaxation, 1125
- linear search, 22 ex.
- linear speedup, 780
- line segment, 1015
 - comparable, 1022
 - determining turn of, 1017
 - determining whether any intersect, 1021–1029, 1047
 - determining whether two intersect, 1017–1019
- link
 - of binomial trees, 527 pr.
 - of Fibonacci-heap roots, 513
 - of trees in a disjoint-set forest, 570–571
- LINK, 571
- linked list, 236–241
 - compact, 245 ex., 250 pr.
 - deletion from, 238
 - to implement disjoint sets, 564–568
 - insertion into, 237–238
 - neighbor list, 750
 - searching, 237, 268 ex.
 - self-organizing, 476 pr.
- list, *see* linked list
- LIST-DELETE, 238
- LIST-DELETE’, 238
- LIST-INSERT, 238
- LIST-INSERT’, 240
- LIST-SEARCH, 237
- LIST-SEARCH’, 239
- literal, 1082
- little-oh notation, 50–51, 64
- little-omega notation, 51
- L_m -distance, 1044 ex.
- ln (natural logarithm), 56
- load factor
 - of a dynamic table, 463
 - of a hash table, 258
- load instruction, 23
- local variable, 21
- logarithm function (log), 56–57
 - discrete, 955
 - iterated ($\lg^*$), 58–59
- logical parallelism, 777
- logic gate, 1070
- longest common subsequence, 7, 390–397, 413
- longest palindrome subsequence, 405 pr.
- LONGEST-PATH, 1060 ex.
- LONGEST-PATH-LENGTH, 1060 ex.
- longest simple cycle, 1101 ex.
- longest simple path, 1048
 - in an unweighted graph, 382
 - in a weighted directed acyclic graph, 404 pr.
- LOOKUP-CHAIN, 388

- loop, in pseudocode, 20
 - parallel, 785–787
- loop invariant, 18–19
 - for breadth-first search, 595
 - for building a heap, 157
 - for consolidating the root list of a Fibonacci heap, 517
 - for determining the rank of an element in an order-statistic tree, 342
 - for Dijkstra’s algorithm, 660
 - and **for** loops, 19 n.
 - for the generic minimum-spanning-tree method, 625
 - for the generic push-relabel algorithm, 743
 - for Graham’s scan, 1034
 - for heapsort, 160 ex.
 - for Horner’s rule, 41 pr.
 - for increasing a key in a heap, 166 ex.
 - initialization of, 19
 - for insertion sort, 18
 - maintenance of, 19
 - for merging, 32
 - for modular exponentiation, 957
 - origin of, 42
 - for partitioning, 171
 - for Prim’s algorithm, 636
 - for the Rabin-Karp algorithm, 993
 - for randomly permuting an array, 127, 128 ex.
 - for red-black tree insertion, 318
 - for the relabel-to-front algorithm, 755
 - for searching an interval tree, 352
 - for the simplex algorithm, 872
 - for string-matching automata, 998, 1000
 - and termination, 19
- low endpoint of an interval, 348
- lower bounds
 - on approximations, 1140
 - asymptotic, 48
 - for average sorting, 207 pr.
 - on binomial coefficients, 1186
 - for comparting water jugs, 206 pr.
 - for convex hull, 1038 ex., 1047
 - for disjoint-set data structures, 585
 - for finding the minimum, 214
 - for finding the predecessor, 560
 - for length of an optimal traveling-salesman tour, 1112–1115
 - for median finding, 227
 - for merging, 208 pr.
 - for minimum-weight vertex cover, 1124–1126
 - for multithreaded computations, 780
 - and potential functions, 478
 - for priority-queue operations, 531
 - and recurrences, 67
 - for simultaneous minimum and maximum, 215 ex.
 - for size of an optimal vertex cover, 1110, 1135 pr.
 - for sorting, 191–194, 205 pr., 211, 531
 - for streaks, 136–138, 142 ex.
 - on summations, 1152, 1154
- lower median, 213
- lower square root ($\sqrt{}$), 546
- lower-triangular matrix, 1219, 1222 ex., 1225 ex.
- low function, 537, 546
- LU decomposition, 806 pr., 819–822
- LU-DECOMPOSITION, 821
- LUP decomposition, 806 pr., 815
 - computation of, 822–825
 - of a diagonal matrix, 827 ex.
 - in matrix inversion, 828
 - and matrix multiplication, 832 ex.
 - of a permutation matrix, 827 ex.
 - use of, 815–819
- LUP-DECOMPOSITION, 824
- LUP-SOLVE, 817
- main memory, 484
- MAKE-HEAP, 505
- MAKE-SET, 561
 - disjoint-set-forest implementation of, 571
 - linked-list implementation of, 564
- makespan, 1136 pr.
- MAKE-TREE, 583 pr.
- Manhattan distance, 225 pr., 1044 ex.
- marked node, 508, 519–520
- Markov’s inequality, 1201 ex.
- master method for solving a recurrence, 93–97
- master theorem, 94
 - proof of, 97–106
- matched vertex, 732
- matching
 - bipartite, 732, 763 pr.

- maximal, 1110, 1135 pr.
- maximum, 1135 pr.
- and maximum flow, 732–736, 747 ex.
- perfect, 735 ex.
- of strings, 985–1013
- weighted bipartite, 530
- matric matroid, 437
- matrix, 1217–1229
 - addition of, 1220
 - adjacency, 591
 - conjugate transpose of, 832 ex.
 - determinant of, 1224–1225
 - diagonal, 1218
 - Hermitian, 832 ex.
 - identity, 1218
 - incidence, 448 pr., 593 ex.
 - inversion of, 806 pr., 827–831, 842
 - lower-triangular, 1219, 1222 ex., 1225 ex.
 - multiplication of, *see* matrix multiplication
 - negative of, 1220
 - permutation, 1220, 1222 ex.
 - predecessor, 685
 - product of, with a vector, 785–787, 789–790, 792 ex.
 - pseudoinverse of, 837
 - scalar multiple of, 1220
 - subtraction of, 1221
 - symmetric, 1220
 - symmetric positive-definite, 832–835, 842
 - Toeplitz, 921 pr.
 - transpose of, 797 ex., 1217
 - transpose of, multithreaded, 792 ex.
 - tridiagonal, 1219
 - unit lower-triangular, 1219
 - unit upper-triangular, 1219
 - upper-triangular, 1219, 1225 ex.
 - Vandermonde, 902, 1226 pr.
- matrix-chain multiplication, 370–378
- MATRIX-CHAIN-MULTIPLY
- MATRIX-CHAIN-ORDER, 375
- matrix multiplication, 75–83, 1221
 - for all-pairs shortest paths, 686–693, 706–707
 - boolean, 832 ex.
 - and computing the determinant, 832 ex.
 - divide-and-conquer method for, 76–83
 - and LUP decomposition, 832 ex.
 - and matrix inversion, 828–831, 842
 - multithreaded algorithm for, 792–797, 806 pr.
 - Pan’s method for, 82 ex.
 - Strassen’s algorithm for, 79–83, 111–112
- MATRIX-MULTIPLY, 371
- matrix-vector multiplication, multithreaded, 785–787, 792 ex.
 - with race, 789–790
- matroid, 437–443, 448 pr., 450, 642
 - for task scheduling, 443–446
- MAT-VEC, 785
- MAT-VEC-MAIN-LOOP, 786
- MAT-VEC-WRONG, 790
- MAX-CNF satisfiability, 1127 ex.
- MAX-CUT problem, 1127 ex.
- MAX-FLOW-BY-SCALING, 763 pr.
- max-flow min-cut theorem, 723
- max-heap, 152
 - building, 156–159
 - d -ary, 167 pr.
 - deletion from, 166 ex.
 - extracting the maximum key from, 163
 - in heapsort, 159–162
 - increasing a key in, 163–164
 - insertion into, 164
 - maximum key of, 163
 - as a max-priority queue, 162–166
 - mergeable, 250 n., 481 n., 505 n.
- MAX-HEAPIFY, 154
- MAX-HEAP-INSERT, 164
 - building a heap with, 166 pr.
- max-heap property, 152
 - maintenance of, 154–156
- maximal element, of a partially ordered set, 1165
- maximal layers, 1045 pr.
- maximal matching, 1110, 1135 pr.
- maximal point, 1045 pr.
- maximal subset, in a matroid, 438
- maximization linear program, 846
 - and minimization linear programs, 852
- maximum, 213
 - in binary search trees, 291
 - of a binomial distribution, 1207 ex.
 - in a bit vector with a superimposed binary tree, 533
 - in a bit vector with a superimposed tree of constant height, 535

- finding, 214–215
 - in heaps, 163
 - in order-statistic trees, 347 ex.
 - in proto van Emde Boas structures, 544 ex.
 - in red-black trees, 311
 - in van Emde Boas trees, 550
- MAXIMUM, 162–163, 230
- maximum bipartite matching, 732–736, 747 ex., 766
 - Hopcroft-Karp algorithm for, 763 pr.
- maximum degree, in a Fibonacci heap, 509, 523–526
- maximum flow, 708–766
 - Edmonds-Karp algorithm for, 727–730
 - Ford-Fulkerson method for, 714–731, 765
 - as a linear program, 860–861
 - and maximum bipartite matching, 732–736, 747 ex.
 - push-relabel algorithms for, 736–760, 765
 - relabel-to-front algorithm for, 748–760
 - scaling algorithm for, 762 pr., 765
 - updating, 762 pr.
- maximum matching, 1135 pr.
- maximum spanning tree, 1137 pr.
- maximum-subarray problem, 68–75, 111
- max-priority queue, 162
- MAX-3-CNF satisfiability, 1123–1124, 1139
- MAYBE-MST-A, 641 pr.
- MAYBE-MST-B, 641 pr.
- MAYBE-MST-C, 641 pr.
- mean, *see* expected value
- mean weight of a cycle, 680 pr.
- median, 213–227
 - multithreaded algorithm for, 805 ex.
 - of sorted lists, 223 ex.
 - of two sorted lists, 804 ex.
 - weighted, 225 pr.
- median key, of a B-tree node, 493
- median-of-3 method, 188 pr.
- member of a set ($\in$), 1158
- membership
 - in proto van Emde Boas structures, 540–541
 - in Van Emde Boas trees, 550
- memoization, 365, 387–389
- MEMOIZED-CUT-ROD, 365
- MEMOIZED-CUT-ROD-AUX, 366
- MEMOIZED-MATRIX-CHAIN, 388
- memory, 484
- memory hierarchy, 24
- MERGE, 31
- mergeable heap, 481, 505
 - binomial heaps, 527 pr.
 - linked-list implementation of, 250 pr.
 - relaxed heaps, 530
 - running times of operations on, 506 fig.
 - 2-3-4 heaps, 529 pr.
 - see also* Fibonacci heap
- mergeable max-heap, 250 n., 481 n., 505 n.
- mergeable min-heap, 250 n., 481 n., 505
- MERGE-LISTS, 1129
- merge sort, 12, 30–37
 - compared with insertion sort, 14 ex.
 - multithreaded algorithm for, 797–805, 812
 - use of insertion sort in, 39 pr.
- MERGE-SORT, 34
- MERGE-SORT', 797
- merging
 - of k sorted lists, 166 ex.
 - lower bounds for, 208 pr.
 - multithreaded algorithm for, 798–801
 - of two sorted arrays, 30
- MILLER-RABIN, 970
- Miller-Rabin primality test, 968–975, 983
- MIN-GAP, 354 ex.
- min-heap, 153
 - analyzed by potential method, 462 ex.
 - building, 156–159
 - d -ary, 706 pr.
 - in Dijkstra's algorithm, 662
 - in Huffman's algorithm, 433
 - in Johnson's algorithm, 704
 - mergeable, 250 n., 481 n., 505
 - as a min-priority queue, 165 ex.
 - in Prim's algorithm, 636
- MIN-HEAPIFY, 156 ex.
- MIN-HEAP-INSERT, 165 ex.
- min-heap ordering, 507
- min-heap property, 153, 507
- maintenance of, 156 ex.
 - in treaps, 333 pr.
 - vs. binary-search-tree property, 289 ex.
- minimization linear program, 846
 - and maximization linear programs, 852
- minimum, 213
 - in binary search trees, 291

- in a bit vector with a superimposed binary tree, 533
- in a bit vector with a superimposed tree of constant height, 535
- in B-trees, 497 ex.
- in Fibonacci heaps, 511
- finding, 214–215
- off-line, 582 pr.
- in order-statistic trees, 347 ex.
- in proto van Emde Boas structures, 541–542
- in red-black trees, 311
- in 2-3-4 heaps, 529 pr.
- in van Emde Boas trees, 550
- MINIMUM, 162, 214, 230, 505
- minimum-cost circulation, 896 pr.
- minimum-cost flow, 861–862
- minimum-cost multicommodity flow, 864 ex.
- minimum-cost spanning tree, *see* minimum spanning tree
- minimum cut, 721, 731 ex.
- minimum degree, of a B-tree, 489
- minimum mean-weight cycle, 680 pr.
- minimum node, of a Fibonacci heap, 508
- minimum path cover, 761 pr.
- minimum spanning tree, 624–642
 - in approximation algorithm for traveling-salesman problem, 1112
 - Borůvka's algorithm for, 641
 - on dynamic graphs, 637 ex.
 - generic method for, 625–630
 - Kruskal's algorithm for, 631–633
 - Prim's algorithm for, 634–636
 - relation to matroids, 437, 439–440
 - second-best, 638 pr.
- minimum-weight spanning tree, *see* minimum spanning tree
- minimum-weight vertex cover, 1124–1127, 1139
- minor of a matrix, 1224
- min-priority queue, 162
 - in constructing Huffman codes, 431
 - in Dijkstra's algorithm, 661
 - in Prim's algorithm, 634, 636
- miss, 449 pr.
- missing child, 1178
- mod, 54, 928
- modifying operation, 230
- modular arithmetic, 54, 923 pr., 939–946
- modular equivalence, 54, 1165 ex.
- modular exponentiation, 956
- MODULAR-EXPONENTIATION, 957
- modular linear equations, 946–950
- MODULAR-LINEAR-EQUATION-SOLVER, 949
- modulo, 54, 928
- Monge array, 110 pr.
- monotone sequence, 168
- monotonically decreasing, 53
- monotonically increasing, 53
- Monty Hall problem, 1195 ex.
- move-to-front heuristic, 476 pr., 478
- MST-KRUSKAL, 631
- MST-PRIM, 634
- MST-REDUCE, 639 pr.
- much-greater-than ($\gg$), 574
- much-less-than ($\ll$), 783
- multicommodity flow, 862–863
 - minimum-cost, 864 ex.
- multicore computer, 772
- multidimensional fast Fourier transform, 921 pr.
- multigraph, 1172
 - converting to equivalent undirected graph, 593 ex.
- multiple, 927
 - of an element modulo n , 946–950
 - least common, 939 ex.
 - scalar, 1220
- multiple assignment, 21
- multiple sources and sinks, 712
- multiplication
 - of complex numbers, 83 ex.
 - divide-and-conquer method for, 920 pr.
 - of matrices, *see* matrix multiplication
 - of a matrix chain, 370–378
 - matrix-vector, multithreaded, 785–787, 789–790, 792 ex.
 - modulo n ($\cdot_n$), 940
 - of polynomials, 899
- multiplication method, 263–264
- multiplicative group modulo n , 941
- multiplicative inverse, modulo n , 949
- multiply instruction, 23
- MULTIPOP, 453
- multiprocessor, 772
- MULTIPUSH, 456 ex.

- multiset, 1158 n.
- multithreaded algorithm, 10, 772–812
 - for computing Fibonacci numbers, 774–780
 - for fast Fourier transform, 804 ex.
 - Floyd-Warshall algorithm, 797 ex.
 - for LU decomposition, 806 pr.
 - for LUP decomposition, 806 pr.
 - for matrix inversion, 806 pr.
 - for matrix multiplication, 792–797, 806 pr.
 - for matrix transpose, 792 ex., 797 ex.
 - for matrix-vector product, 785–787, 789–790, 792 ex.
 - for median, 805 ex.
 - for merge sorting, 797–805, 812
 - for merging, 798–801
 - for order statistics, 805 ex.
 - for partitioning, 804 ex.
 - for prefix computation, 807 pr.
 - for quicksort, 811 pr.
 - for reduction, 807 pr.
 - for a simple stencil calculation, 809 pr.
 - for solving systems of linear equations, 806 pr.
 - Strassen's algorithm, 795–796
- multithreaded composition, 784 fig.
- multithreaded computation, 777
- multithreaded scheduling, 781–783
- mutually exclusive events, 1190
- mutually independent events, 1193
- $\mathbb{N}$ (set of natural numbers), 1158
- naive algorithm, for string matching, 988–990
- NAIVE-STRING-MATCHER, 988
- natural cubic spline, 840 pr.
- natural numbers ($\mathbb{N}$), 1158
 - keys interpreted as, 263
- negative of a matrix, 1220
- negative-weight cycle
 - and difference constraints, 667
 - and relaxation, 677 ex.
 - and shortest paths, 645, 653–654, 692 ex., 700 ex.
- negative-weight edges, 645–646
- neighbor, 1172
- neighborhood, 735 ex.
- neighbor list, 750
- nested parallelism, 776, 805 pr.
- nesting boxes, 678 pr.
- net flow across a cut, 720
- network
 - admissible, 749–750
 - flow, *see* flow network
 - residual, 715–719
 - for sorting, 811
- NEXT-TO-TOP, 1031
- NIL, 21
- node, 1176
 - see also* vertex
- nonbasic variable, 855
- nondeterministic multithreaded algorithm, 787
- nondeterministic polynomial time, 1064 n.
 - see also* NP
- nonhamiltonian graph, 1061
- noninstance, 1056 n.
- noninvertible matrix, 1223
- nonnegativity constraint, 851, 853
- nonoverlappable string pattern, 1002 ex.
- nonsaturating push, 739, 745
- nonsingular matrix, 1223
- nontrivial power, 933 ex.
- nontrivial square root of 1, modulo n , 956
- no-path property, 650, 672
- normal equation, 837
- norm of a vector, 1222
- NOT function ($\neg$), 1071
- not a set member ($\notin$), 1158
- not equivalent ($\not\equiv$), 54
- NOT gate, 1070
- NP (complexity class), 1049, 1064, 1066 ex., 1105
- NPC (complexity class), 1050, 1069
- NP-complete, 1050, 1069
- NP-completeness, 9–10, 1048–1105
 - of the circuit-satisfiability problem, 1070–1077
 - of the clique problem, 1086–1089, 1105
 - of determining whether a boolean formula is a tautology, 1086 ex.
 - of the formula-satisfiability problem, 1079–1081, 1105
 - of the graph-coloring problem, 1103 pr.
 - of the half 3-CNF satisfiability problem, 1101 ex.
 - of the hamiltonian-cycle problem, 1091–1096, 1105
 - of the hamiltonian-path problem, 1101 ex.

- of the independent-set problem, 1101 pr.
- of integer linear programming, 1101 ex.
- of the longest-simple-cycle problem, 1101 ex.
- proving, of a language, 1078–1079
- of scheduling with profits and deadlines, 1104 pr.
- of the set-covering problem, 1122 ex.
- of the set-partition problem, 1101 ex.
- of the subgraph-isomorphism problem, 1100 ex.
- of the subset-sum problem, 1097–1100
- of the 3-CNF-satisfiability problem, 1082–1085, 1105
- of the traveling-salesman problem, 1096–1097
- of the vertex-cover problem, 1089–1091, 1105
- of 0-1 integer programming, 1100 ex.
- NP-hard, 1069
- n -set, 1161
- n -tuple, 1162
- null event, 1190
- null tree, 1178
- null vector, 1224
- number-field sieve, 984
- numerical stability, 813, 815, 842
- n -vector, 1218
- o -notation, 50–51, 64
- O -notation, 45 fig., 47–48, 64
- Q' -notation, 62 pr.
- $\widetilde{O}$ -notation, 62 pr.
- object, 21
 - allocation and freeing of, 243–244
 - array implementation of, 241–246
 - passing as parameter, 21
- objective function, 664, 847, 851
- objective value, 847, 851
- oblivious compare-exchange algorithm, 208 pr.
- occurrence of a pattern, 985
- OFF-LINE-MINIMUM, 583 pr.
- off-line problem
 - caching, 449 pr.
 - least common ancestors, 584 pr.
 - minimum, 582 pr.
- Omega-notation, 45 fig., 48–49, 64
- 1-approximation algorithm, 1107
- one-pass method, 585
- one-to-one correspondence, 1167
- one-to-one function, 1167
- on-line convex-hull problem, 1039 ex.
- on-line hiring problem, 139–141
- ON-LINE-MAXIMUM, 140
- on-line multithreaded scheduler, 781
- ON-SEGMENT, 1018
- onto function, 1167
- open-address hash table, 269–277
 - with double hashing, 272–274, 277 ex.
 - with linear probing, 272
 - with quadratic probing, 272, 283 pr.
- open interval, 348
- OpenMP, 774
- optimal binary search tree, 397–404, 413
- OPTIMAL-BST, 402
- optimal objective value, 851
- optimal solution, 851
- optimal subset, of a matroid, 439
- optimal substructure
 - of activity selection, 416
 - of binary search trees, 399–400
 - in dynamic programming, 379–384
 - of the fractional knapsack problem, 426
 - in greedy algorithms, 425
 - of Huffman codes, 435
 - of longest common subsequences, 392–393
 - of matrix-chain multiplication, 373
 - of rod-cutting, 362
 - of shortest paths, 644–645, 687, 693–694
 - of unweighted shortest paths, 382
 - of weighted matroids, 442
 - of the 0-1 knapsack problem, 426
- optimal vertex cover, 1108
- optimization problem, 359, 1050, 1054
 - approximation algorithms for, 10, 1106–1140
 - and decision problems, 1051
- OR function ($\vee$), 697, 1071
- order
 - of a group, 945
 - linear, 1165
 - partial, 1165
 - total, 1165
- ordered pair, 1161
- ordered tree, 1177
- order of growth, 28

- order statistics, 213–227
 - dynamic, 339–345
 - multithreaded algorithm for, 805 ex.
- order-statistic tree, 339–345
 - querying, 347 ex.
- OR gate, 1070
- origin, 1015
- or, in pseudocode, 22
- orthonormal, 842
- OS-KEY-RANK, 344 ex.
- OS-RANK, 342
- OS-SELECT, 341
- out-degree, 1169
- outer product, 1222
- output
 - of an algorithm, 5
 - of a combinational circuit, 1071
 - of a logic gate, 1070
- overdetermined system of linear equations, 814
- overflow
 - of a queue, 235
 - of a stack, 233
- overflowing vertex, 736
 - discharge of, 751
- overlapping intervals, 348
 - finding all, 354 ex.
 - point of maximum overlap, 354 pr.
- overlapping rectangles, 354 ex.
- overlapping subproblems, 384–386
- overlapping-suffix lemma, 987
- P (complexity class), 1049, 1055, 1059, 1061 ex., 1105
- package wrapping, 1037, 1047
- page on a disk, 486, 499 ex., 502 pr.
- pair, ordered, 1161
- pairwise disjoint sets, 1161
- pairwise independence, 1193
- pairwise relatively prime, 931
- palindrome, 405 pr.
- Pan's method for matrix multiplication, 82 ex.
- parallel algorithm, 10, 772
 - see also* multithreaded algorithm
- parallel computer, 772
 - ideal, 779
- parallel for**, in pseudocode, 785–786
- parallelism
 - logical, 777
 - of a multithreaded computation, 780
 - nested, 776
 - of a randomized multithreaded algorithm, 811 pr.
- parallel loop, 785–787, 805 pr.
- parallel-machine-scheduling problem, 1136 pr.
- parallel prefix, 807 pr.
- parallel random-access machine, 811
- parallel slackness, 781
 - rule of thumb, 783
- parallel, strands being logically in, 778
- parameter, 21
 - costs of passing, 107 pr.
- parent
 - in a breadth-first tree, 594
 - in a multithreaded computation, 776
 - in a rooted tree, 1176
- PARENT, 152
- parenthesis structure of depth-first search, 606
- parenthesis theorem, 606
- parenthesization of a matrix-chain product, 370
- parse tree, 1082
- partially ordered set, 1165
- partial order, 1165
- PARTITION, 171
- PARTITION', 186 pr.
- partition function, 361 n.
- partitioning, 171–173
 - around median of 3 elements, 185 ex.
 - Hoare's method for, 185 pr.
 - multithreaded algorithm for, 804 ex.
 - randomized, 179
- partition of a set, 1161, 1164
- Pascal's triangle, 1188 ex.
- path, 1170
 - augmenting, 719–720, 763 pr.
 - critical, 657
 - find, 569
 - hamiltonian, 1066 ex.
 - longest, 382, 1048
 - shortest, *see* shortest paths
 - simple, 1170
 - weight of, 643
- PATH, 1051, 1058
- path compression, 569
- path cover, 761 pr.
- path length, of a tree, 304 pr., 1180 ex.
- path-relaxation property, 650, 673

- pattern, in string matching, 985
 - nonoverlappable, 1002 ex.
- pattern matching, *see* string matching
- penalty, 444
- perfect hashing, 277–282, 285
- perfect linear speedup, 780
- perfect matching, 735 ex.
- permutation, 1167
 - bit-reversal, 472 pr., 918
 - Josephus, 355 pr.
 - k -permutation, 126, 1184
 - linear, 1229 pr.
 - in place, 126
 - random, 124–128
 - of a set, 1184
 - uniform random, 116, 125
- permutation matrix, 1220, 1222 ex., 1226 ex.
 - LUP decomposition of, 827 ex.
- PERMUTE-BY-CYCLIC, 129 ex.
- PERMUTE-BY-SORTING, 125
- PERMUTE-WITH-ALL, 129 ex.
- PERMUTE-WITHOUT-IDENTITY, 128 ex.
- persistent data structure, 331 pr., 482
- PERSISTENT-TREE-INSERT, 331 pr.
- PERT chart, 657, 657 ex.
- P-FIB, 776
- phase, of the relabel-to-front algorithm, 758
- phi function ($\phi(n)$), 943
- PISANO-DELETE, 526 pr.
- pivot
 - in linear programming, 867, 869–870, 878 ex.
 - in LU decomposition, 821
 - in quicksort, 171
- PIVOT, 869
- platter, 485
- P-MATRIX-MULTIPLY-RECURSIVE, 794
- P-MERGE, 800
- P-MERGE-SORT, 803
- pointer, 21
 - array implementation of, 241–246
 - trailing, 295
- point-value representation, 901
- polar angle, 1020 ex.
- Pollard's rho heuristic, 976–980, 980 ex., 984
- POLLARD-RHO, 976
- polygon, 1020 ex.
 - kernel of, 1038 ex.
 - star-shaped, 1038 ex.
- polylogarithmically bounded, 57
- polynomial, 55, 898
 - addition of, 898
 - asymptotic behavior of, 61 pr.
 - coefficient representation of, 900
 - derivatives of, 922 pr.
 - evaluation of, 41 pr., 900, 905 ex., 923 pr.
 - interpolation by, 901, 906 ex.
 - multiplication of, 899, 903–905, 920 pr.
 - point-value representation of, 901
- polynomial-growth condition, 113
- polynomially bounded, 55
- polynomially related, 1056
- polynomial-time acceptance, 1058
- polynomial-time algorithm, 927, 1048
- polynomial-time approximation scheme, 1107
 - for maximum clique, 1134 pr.
- polynomial-time computability, 1056
- polynomial-time decision, 1059
- polynomial-time reducibility ($\leq_p$), 1067, 1077 ex.
- polynomial-time solvability, 1055
- polynomial-time verification, 1061–1066
- POP, 233, 452
- pop from a run-time stack, 188 pr.
- positional tree, 1178
- positive-definite matrix, 1225
- post-office location problem, 225 pr.
- postorder tree walk, 287
- potential function, 459
 - for lower bounds, 478
- potential method, 459–463
 - for binary counters, 461–462
 - for disjoint-set data structures, 575–581, 582 ex.
 - for dynamic tables, 466–471
 - for Fibonacci heaps, 509–512, 517–518, 520–522
 - for the generic push-relabel algorithm, 746
 - for min-heaps, 462 ex.
 - for restructuring red-black trees, 474 pr.
 - for self-organizing lists with move-to-front, 476 pr.
 - for stack operations, 460–461
- potential, of a data structure, 459
- power
 - of an element, modulo n , 954–958

- k th, 933 ex.
- nontrivial, 933 ex.
- power series, 108 pr.
- power set, 1161
- $\Pr\{\}$ (probability distribution), 1190
- PRAM, 811
- predecessor
 - in binary search trees, 291–292
 - in a bit vector with a superimposed binary tree, 534
 - in a bit vector with a superimposed tree of constant height, 535
 - in breadth-first trees, 594
 - in B-trees, 497 ex.
 - in linked lists, 236
 - in order-statistic trees, 347 ex.
 - in proto van Emde Boas structures, 544 ex.
 - in red-black trees, 311
 - in shortest-paths trees, 647
 - in Van Emde Boas trees, 551–552
- PREDECESSOR, 230
- predecessor matrix, 685
- predecessor subgraph
 - in all-pairs shortest paths, 685
 - in breadth-first search, 600
 - in depth-first search, 603
 - in single-source shortest paths, 647
- predecessor-subgraph property, 650, 676
- preemption, 447 pr.
- prefix
 - of a sequence, 392
 - of a string ($\square$), 986
- prefix code, 429
- prefix computation, 807 pr.
- prefix function, 1003–1004
- prefix-function iteration lemma, 1007
- preflow, 736, 765
- preimage of a matrix, 1228 pr.
- preorder, total, 1165
- preorder tree walk, 287
- presorting, 1043
- Prim's algorithm, 634–636, 642
 - with an adjacency matrix, 637 ex.
 - in approximation algorithm for traveling-salesman problem, 1112
 - implemented with a Fibonacci heap, 636
 - implemented with a min-heap, 636
 - with integer edge weights, 637 ex.
 - similarity to Dijkstra's algorithm, 634, 662
 - for sparse graphs, 638 pr.
- primality testing, 965–975, 983
 - Miller-Rabin test, 968–975, 983
 - pseudoprimalty testing, 966–968
- primal linear program, 880
- primary clustering, 272
- primary memory, 484
- prime distribution function, 965
- prime number, 928
 - density of, 965–966
- prime number theorem, 965
- primitive root of $\mathbb{Z}_n^*$, 955
- principal root of unity, 907
- principle of inclusion and exclusion, 1163 ex.
- PRINT-ALL-PAIRS-SHORTEST-PATH, 685
- PRINT-CUT-ROD-SOLUTION, 369
- PRINT-INTERSECTING-SEGMENTS, 1028 ex.
- PRINT-LCS, 395
- PRINT-OPTIMAL-PARENS, 377
- PRINT-PATH, 601
- PRINT-SET, 572 ex.
- priority queue, 162–166
 - in constructing Huffman codes, 431
 - in Dijkstra's algorithm, 661
 - heap implementation of, 162–166
 - lower bounds for, 531
 - max-priority queue, 162
 - min-priority queue, 162, 165 ex.
 - with monotone extractions, 168
 - in Prim's algorithm, 634, 636
 - proto van Emde Boas structure
 - implementation of, 538–545
 - van Emde Boas tree implementation of, 531–560
 - see also* binary search tree, binomial heap, Fibonacci heap
- probabilistically checkable proof, 1105, 1140
- probabilistic analysis, 115–116, 130–142
 - of approximation algorithm for MAX-3-CNF satisfiability, 1124
 - and average inputs, 28
 - of average node depth in a randomly built binary search tree, 304 pr.
 - of balls and bins, 133–134
 - of birthday paradox, 130–133
 - of bucket sort, 201–204, 204 ex.
 - of collisions, 261 ex., 282 ex.

- of convex hull over a sparse-hulled distribution, 1046 pr.
 - of file comparison, 995 ex.
 - of fuzzy sorting of intervals, 189 pr.
 - of hashing with chaining, 258–260
 - of height of a randomly built binary search tree, 299–303
 - of hiring problem, 120–121, 139–141
 - of insertion into a binary search tree with equal keys, 303 pr.
 - of longest-probe bound for hashing, 282 pr.
 - of lower bound for sorting, 205 pr.
 - of Miller-Rabin primality test, 971–975
 - and multithreaded algorithms, 811 pr.
 - of on-line hiring problem, 139–141
 - of open-address hashing, 274–276, 277 ex.
 - of partitioning, 179 ex., 185 ex., 187–188 pr.
 - of perfect hashing, 279–282
 - of Pollard’s rho heuristic, 977–980
 - of probabilistic counting, 143 pr.
 - of quicksort, 181–184, 187–188 pr., 303 ex.
 - of Rabin-Karp algorithm, 994
 - and randomized algorithms, 123–124
 - of randomized selection, 217–219, 226 pr.
 - of searching a compact list, 250 pr.
 - of slot-size bound for chaining, 283 pr.
 - of sorting points by distance from origin, 204 ex.
 - of streaks, 135–139
 - of universal hashing, 265–268
- probabilistic counting, 143 pr.
- probability, 1189–1196
- probability density function, 1196
- probability distribution, 1190
- probability distribution function, 204 ex.
- probe sequence, 270
- probing, 270, 282 pr.
 - see also* linear probing, quadratic probing, double hashing
- problem
 - abstract, 1054
 - computational, 5–6
 - concrete, 1055
 - decision, 1051, 1054
 - intractable, 1048
 - optimization, 359, 1050, 1054
 - solution to, 6, 1054–1055
 - tractable, 1048
- procedure, 6, 16–17
- product ($\prod$), 1148
 - Cartesian, 1162
 - cross, 1016
 - inner, 1222
 - of matrices, 1221, 1226 ex.
 - outer, 1222
 - of polynomials, 899
 - rule of, 1184
 - scalar flow, 714 ex.
- professional wrestler, 602 ex.
- program counter, 1073
- programming, *see* dynamic programming, linear programming
- proper ancestor, 1176
- proper descendant, 1176
- proper subgroup, 944
- proper subset ($\subset$), 1159
- proto van Emde Boas structure, 538–545
 - cluster in, 538
 - compared with van Emde Boas trees, 547
 - deletion from, 544
 - insertion into, 544
 - maximum in, 544 ex.
 - membership in, 540–541
 - minimum in, 541–542
 - predecessor in, 544 ex.
 - successor in, 543–544
 - summary in, 540
- PROTO-VEB-INSERT, 544
- PROTO-VEB-MEMBER, 541
- PROTO-VEB-MINIMUM, 542
- proto-veB structure, *see* proto van Emde Boas structure
- PROTO-VEB-SUCCESSOR, 543
- prune-and-search method, 1030
- pruning a Fibonacci heap, 529 pr.
- P-SCAN-1, 808 pr.
- P-SCAN-2, 808 pr.
- P-SCAN-3, 809 pr.
- P-SCAN-DOWN, 809 pr.
- P-SCAN-UP, 809 pr.
- pseudocode, 16, 20–22
- pseudoinverse, 837
- pseudoprime, 966–968
- PSEUDOPRIME, 967
- pseudorandom-number generator, 117
- P-SQUARE-MATRIX-MULTIPLY, 793

- P-TRANPOSE, 792 ex.
- public key, 959, 962
- public-key cryptosystem, 958–965, 983
- PUSH
 - push-relabel operation, 739
 - stack operation, 233, 452
- push onto a run-time stack, 188 pr.
- push operation (in push-relabel algorithms), 738–739
 - nonsaturating, 739, 745
 - saturating, 739, 745
- push-relabel algorithm, 736–760, 765
 - basic operations in, 738–740
 - by discharging an overflowing vertex of maximum height, 760 ex.
 - to find a maximum bipartite matching, 747 ex.
 - gap heuristic for, 760 ex., 766
 - generic algorithm, 740–748
 - with a queue of overflowing vertices, 759 ex.
 - relabel-to-front algorithm, 748–760
- quadratic function, 27
- quadratic probing, 272, 283 pr.
- quadratic residue, 982 pr.
- quantile, 223 ex.
- query, 230
- queue, 232, 234–235
 - in breadth-first search, 595
 - implemented by stacks, 236 ex.
 - linked-list implementation of, 240 ex.
 - priority, *see* priority queue
 - in push-relabel algorithms, 759 ex.
- quicksort, 170–190
 - analysis of, 174–185
 - average-case analysis of, 181–184
 - compared with insertion sort, 178 ex.
 - compared with radix sort, 199
 - with equal element values, 186 pr.
 - good worst-case implementation of, 223 ex.
 - “killer adversary” for, 190
 - with median-of-3 method, 188 pr.
 - multithreaded algorithm for, 811 pr.
 - randomized version of, 179–180, 187 pr.
 - stack depth of, 188 pr.
 - tail-recursive version of, 188 pr.
 - use of insertion sort in, 185 ex.
 - worst-case analysis of, 180–181
- QUICKSORT, 171
- QUICKSORT', 186 pr.
- quotient, 928
- $\mathbb{R}$ (set of real numbers), 1158
- Rabin-Karp algorithm, 990–995, 1013
- RABIN-KARP-MATCHER, 993
- race, 787–790
- RACE-EXAMPLE, 788
- radix sort, 197–200
 - compared with quicksort, 199
- RADIX-SORT, 198
- radix tree, 304 pr.
- RAM, 23–24
- RANDOM, 117
- random-access machine, 23–24
 - parallel, 811
- randomized algorithm, 116–117, 122–130
 - and average inputs, 28
 - comparison sort, 205 pr.
 - for fuzzy sorting of intervals, 189 pr.
 - for hiring problem, 123–124
 - for insertion into a binary search tree with equal keys, 303 pr.
 - for MAX-3-CNF satisfiability, 1123–1124, 1139
 - Miller-Rabin primality test, 968–975, 983
 - multithreaded, 811 pr.
 - for partitioning, 179, 185 ex., 187–188 pr.
 - for permuting an array, 124–128
 - Pollard's rho heuristic, 976–980, 980 ex., 984
 - and probabilistic analysis, 123–124
 - quicksort, 179–180, 185 ex., 187–188 pr.
 - randomized rounding, 1139
 - for searching a compact list, 250 pr.
 - for selection, 215–220
 - universal hashing, 265–268
 - worst-case performance of, 180 ex.
- RANDOMIZED-HIRE-ASSISTANT, 124
- RANDOMIZED-PARTITION, 179
- RANDOMIZED-QUICKSORT, 179, 303 ex.
 - relation to randomly built binary search trees, 304 pr.
- randomized rounding, 1139
- RANDOMIZED-SELECT, 216
- RANDOMIZE-IN-PLACE, 126

- randomly built binary search tree, 299–303, 304 pr.
- random-number generator, 117
- random permutation, 124–128
 - uniform, 116, 125
- RANDOM-SAMPLE, 130 ex.
- random sampling, 129 ex., 179
- RANDOM-SEARCH, 143 pr.
- random variable, 1196–1201
 - indicator, *see* indicator random variable
- range, 1167
 - of a matrix, 1228 pr.
- rank
 - column, 1223
 - full, 1223
 - of a matrix, 1223, 1226 ex.
 - of a node in a disjoint-set forest, 569, 575, 581 ex.
 - of a number in an ordered set, 300, 339
 - in order-statistic trees, 341–343, 344–345 ex.
 - row, 1223
- rate of growth, 28
- ray, 1021 ex.
- RB-DELETE, 324
- RB-DELETE-FIXUP, 326
- RB-ENUMERATE, 348 ex.
- RB-INSERT, 315
- RB-INSERT-FIXUP, 316
- RB-JOIN, 332 pr.
- RB-TRANSPLANT, 323
- reachability in a graph ($\leadsto$), 1170
- real numbers ($\mathbb{R}$), 1158
- reconstructing an optimal solution, in dynamic programming, 387
- record, 147
- rectangle, 354 ex.
- recurrence, 34, 65–67, 83–113
 - solution by Akra-Bazzi method, 112–113
 - solution by master method, 93–97
 - solution by recursion-tree method, 88–93
 - solution by substitution method, 83–88
- recurrence equation, *see* recurrence
- recursion, 30
- recursion tree, 37, 88–93
 - in proof of master theorem, 98–100
 - and the substitution method, 91–92
- RECURSIVE-ACTIVITY-SELECTOR, 419
- recursive case, 65
- RECURSIVE-FFT, 911
- RECURSIVE-MATRIX-CHAIN, 385
- red-black tree, 308–338
 - augmentation of, 346–347
 - compared with B-trees, 484, 490
 - deletion from, 323–330
 - in determining whether any line segments intersect, 1024
 - for enumerating keys in a range, 348 ex.
 - height of, 309
 - insertion into, 315–323
 - joining of, 332 pr.
 - maximum key of, 311
 - minimum key of, 311
 - predecessor in, 311
 - properties of, 308–312
 - relaxed, 311 ex.
 - restructuring, 474 pr.
 - rotation in, 312–314
 - searching in, 311
 - successor in, 311
 - see also* interval tree, order-statistic tree
- REDUCE, 807 pr.
- reduced-space van Emde Boas tree, 557 pr.
- reducibility, 1067–1068
- reduction algorithm, 1052, 1067
- reduction function, 1067
- reduction, of an array, 807 pr.
- reflexive relation, 1163
- reflexivity of asymptotic notation, 51
- region, feasible, 847
- regularity condition, 95
- rejection
 - by an algorithm, 1058
 - by a finite automaton, 996
- RELABEL, 740
- relabeled vertex, 740
- relabel operation, in push-relabel algorithms, 740, 745
- RELABEL-TO-FRONT, 755
- relabel-to-front algorithm, 748–760
 - phase of, 758
- relation, 1163–1166
- relatively prime, 931
- RELAX, 649
- relaxation
 - of an edge, 648–650
 - linear programming, 1125

- relaxed heap, 530
- relaxed red-black tree, 311 ex.
- release time, 447 pr.
- remainder, 54, 928
- remainder instruction, 23
- repeated squaring
 - for all-pairs shortest paths, 689–691
 - for raising a number to a power, 956
- repeat**, in pseudocode, 20
- repetition factor, of a string, 1012 pr.
- REPETITION-MATCHER, 1013 pr.
- representative of a set, 561
- RESET, 459 ex.
- residual capacity, 716, 719
- residual edge, 716
- residual network, 715–719
- residue, 54, 928, 982 pr.
- respecting a set of edges, 626
- return edge, 779
- return**, in pseudocode, 22
- return instruction, 23
- reweighting
 - in all-pairs shortest paths, 700–702
 - in single-source shortest paths, 679 pr.
- rho heuristic, 976–980, 980 ex., 984
- $\rho(n)$ -approximation algorithm, 1106, 1123
- RIGHT, 152
- right child, 1178
- right-conversion, 314 ex.
- right horizontal ray, 1021 ex.
- RIGHT-ROTATE, 313
- right rotation, 312
- right spine, 333 pr.
- right subtree, 1178
- rod-cutting, 360–370, 390 ex.
- root
 - of a tree, 1176
 - of unity, 906–907
 - of $\mathbb{Z}_n^*$, 955
- rooted tree, 1176
 - representation of, 246–249
- root list, of a Fibonacci heap, 509
- rotation
 - cyclic, 1012 ex.
 - in a red-black tree, 312–314
- rotational sweep, 1030–1038
- rounding, 1126
 - randomized, 1139
- row-major order, 394
- row rank, 1223
- row vector, 1218
- RSA public-key cryptosystem, 958–965, 983
- RS-vEB tree, 557 pr.
- rule of product, 1184
- rule of sum, 1183
- running time, 25
 - average-case, 28, 116
 - best-case, 29 ex., 49
 - expected, 28, 117
 - of a graph algorithm, 588
 - and multithreaded computation, 779–780
 - order of growth, 28
 - rate of growth, 28
 - worst-case, 27, 49
- sabermetrics, 412 n.
- safe edge, 626
- SAME-COMPONENT, 563
- sample space, 1189
- sampling, 129 ex., 179
- SAT, 1079
- satellite data, 147, 229
- satisfiability, 1072, 1079–1081, 1105, 1123–1124, 1127 ex., 1139
- satisfiable formula, 1049, 1079
- satisfying assignment, 1072, 1079
- saturated edge, 739
- saturating push, 739, 745
- scalar flow product, 714 ex.
- scalar multiple, 1220
- scaling
 - in maximum flow, 762 pr., 765
 - in single-source shortest paths, 679 pr.
- scan, 807 pr.
- SCAN, 807 pr.
- scapegoat tree, 338
- schedule, 444, 1136 pr.
 - event-point, 1023
- scheduler, for multithreaded computations, 777, 781–783, 812
 - centralized, 782
 - greedy, 782
 - work-stealing algorithm for, 812
- scheduling, 443–446, 447 pr., 450, 1104 pr., 1136 pr.
- Schur complement, 820, 834

- Schur complement lemma, 834
- SCRAMBLE-SEARCH, 143 pr.
- seam carving, 409 pr., 413
- SEARCH, 230
- searching, 22 ex.
 - binary search, 39 ex., 799–800
 - in binary search trees, 289–291
 - in B-trees, 491–492
 - in chained hash tables, 258
 - in compact lists, 250 pr.
 - in direct-address tables, 254
 - for an exact interval, 354 ex.
 - in interval trees, 350–353
 - linear search, 22 ex.
 - in linked lists, 237
 - in open-address hash tables, 270–271
 - in proto van Emde Boas structures, 540–541
 - in red-black trees, 311
 - in an unsorted array, 143 pr.
 - in Van Emde Boas trees, 550
- search tree, *see* balanced search tree, binary search tree, B-tree, exponential search tree, interval tree, optimal binary search tree, order-statistic tree, red-black tree, splay tree, 2-3 tree, 2-3-4 tree
- secondary clustering, 272
- secondary hash table, 278
- secondary storage
 - search tree for, 484–504
 - stacks on, 502 pr.
- second-best minimum spanning tree, 638 pr.
- secret key, 959, 962
- segment, *see* directed segment, line segment
- SEGMENTS-INTERSECT, 1018
- SELECT, 220
- selection, 213
 - of activities, 415–422, 450
 - and comparison sorts, 222
 - in expected linear time, 215–220
 - multithreaded, 805 ex.
 - in order-statistic trees, 340–341
 - in worst-case linear time, 220–224
- selection sort, 29 ex.
- selector vertex, 1093
- self-loop, 1168
- self-organizing list, 476 pr., 478
- semiconnected graph, 621 ex.
- sentinel, 31, 238–240, 309
- sequence ($()$)
 - bitonic, 682 pr.
 - finite, 1166
 - infinite, 1166
 - inversion in, 41 pr., 122 ex., 345 ex.
 - probe, 270
- sequential consistency, 779, 812
- serial algorithm versus parallel algorithm, 772
- serialization, of a multithreaded algorithm, 774, 776
- series, 108 pr., 1146–1148
 - strands being logically in, 778
- set ($\{ \}$), 1158–1163
 - cardinality ($||$), 1161
 - convex, 714 ex.
 - difference ($-$), 1159
 - independent, 1101 pr.
 - intersection ($\cap$), 1159
 - member ($\in$), 1158
 - not a member ($\notin$), 1158
 - union ($\cup$), 1159
- set-covering problem, 1117–1122, 1139
 - weighted, 1135 pr.
- set-partition problem, 1101 ex.
- shadow of a point, 1038 ex.
- shared memory, 772
- Shell's sort, 42
- shift, in string matching, 985
- shift instruction, 24
- short-circuiting operator, 22
- SHORTEST-PATH, 1050
- shortest paths, 7, 643–707
 - all-pairs, 644, 684–707
 - Bellman-Ford algorithm for, 651–655
 - with bitonic paths, 682 pr.
 - and breadth-first search, 597–600, 644
 - convergence property of, 650, 672–673
 - and difference constraints, 664–670
 - Dijkstra's algorithm for, 658–664
 - in a directed acyclic graph, 655–658
 - in ϵ -dense graphs, 706 pr.
 - estimate of, 648
 - Floyd-Warshall algorithm for, 693–697, 700 ex., 706
 - Gabow's scaling algorithm for, 679 pr.
 - Johnson's algorithm for, 700–706
 - as a linear program, 859–860
 - and longest paths, 1048

- by matrix multiplication, 686–693, 706–707
- and negative-weight cycles, 645, 653–654, 692 ex., 700 ex.
- with negative-weight edges, 645–646
- no-path property of, 650, 672
- optimal substructure of, 644–645, 687, 693–694
- path-relaxation property of, 650, 673
- predecessor-subgraph property of, 650, 676
- problem variants, 644
 - and relaxation, 648–650
- by repeated squaring, 689–691
- single-destination, 644
- single-pair, 381, 644
- single-source, 643–683
- tree of, 647–648, 673–676
- triangle inequality of, 650, 671
- in an unweighted graph, 381, 597
- upper-bound property of, 650, 671–672
- in a weighted graph, 643
- sibling, 1176
- side of a polygon, 1020 ex.
- signature, 960
- simple cycle, 1170
- simple graph, 1170
- simple path, 1170
 - longest, 382, 1048
- simple polygon, 1020 ex.
- simple stencil calculation, 809 pr.
- simple uniform hashing, 259
- simplex, 848
- SIMPLEX, 871
- simplex algorithm, 848, 864–879, 896–897
- single-destination shortest paths, 644
- single-pair shortest path, 381, 644
 - as a linear program, 859–860
- single-source shortest paths, 643–683
 - Bellman-Ford algorithm for, 651–655
 - with bitonic paths, 682 pr.
 - and difference constraints, 664–670
 - Dijkstra's algorithm for, 658–664
 - in a directed acyclic graph, 655–658
 - in ϵ -dense graphs, 706 pr.
 - Gabow's scaling algorithm for, 679 pr.
 - as a linear program, 863 ex.
 - and longest paths, 1048
- singleton, 1161
- singly connected graph, 612 ex.
- singly linked list, 236
 - see also* linked list
- singular matrix, 1223
- singular value decomposition, 842
- sink vertex, 593 ex., 709, 712
- size
 - of an algorithm's input, 25, 926–927, 1055–1057
 - of a binomial tree, 527 pr.
 - of a boolean combinational circuit, 1072
 - of a clique, 1086
 - of a set, 1161
 - of a subtree in a Fibonacci heap, 524
 - of a vertex cover, 1089, 1108
- skip list, 338
- slack, 855
- slack form, 846, 854–857
 - uniqueness of, 876
- slackness
 - complementary, 894 pr.
 - parallel, 781
- slack variable, 855
- slot
 - of a direct-access table, 254
 - of a hash table, 256
- SLOW-ALL-PAIRS-SHORTEST-PATHS, 689
- smoothed analysis, 897
- *Socrates, 790
- solution
 - to an abstract problem, 1054
 - basic, 866
 - to a computational problem, 6
 - to a concrete problem, 1055
 - feasible, 665, 846, 851
 - infeasible, 851
 - optimal, 851
 - to a system of linear equations, 814
- sorted linked list, 236
 - see also* linked list
- sorting, 5, 16–20, 30–37, 147–212, 797–805
 - bubblesort, 40 pr.
 - bucket sort, 200–204
 - columnsort, 208 pr.
 - comparison sort, 191
 - counting sort, 194–197
 - fuzzy, 189 pr.
 - heapsort, 151–169
 - insertion sort, 12, 16–20

- k*-sorting, 207 pr.
- lexicographic, 304 pr.
- in linear time, 194–204, 206 pr.
- lower bounds for, 191–194, 211, 531
- merge sort, 12, 30–37, 797–805
- by oblivious compare-exchange algorithms, 208 pr.
- in place, 17, 148, 206 pr.
- of points by polar angle, 1020 ex.
- probabilistic lower bound for, 205 pr.
- quicksort, 170–190
- radix sort, 197–200
- selection sort, 29 ex.
- Shell's sort, 42
- stable, 196
- table of running times, 149
- topological, 8, 612–615, 623
- using a binary search tree, 299 ex.
- with variable-length items, 206 pr.
- 0-1 sorting lemma, 208 pr.
- sorting network, 811
- source vertex, 594, 644, 709, 712
- span law, 780
- spanning tree, 439, 624
 - bottleneck, 640 pr.
 - maximum, 1137 pr.
 - verification of, 642
 - see also* minimum spanning tree
- span, of a multithreaded computation, 779
- sparse graph, 589
 - all-pairs shortest paths for, 700–705
 - and Prim's algorithm, 638 pr.
- sparse-hulled distribution, 1046 pr.
- spawn**, in pseudocode, 776–777
- spawn edge, 778
- speedup, 780
 - of a randomized multithreaded algorithm, 811 pr.
- spindle, 485
- spine
 - of a string-matching automaton, 997 fig.
 - of a treap, 333 pr.
- splay tree, 338, 482
- spline, 840 pr.
- splitting
 - of B-tree nodes, 493–495
 - of 2-3-4 trees, 503 pr.
- splitting summations, 1152–1154
- spurious hit, 991
- square matrix, 1218
- SQUARE-MATRIX-MULTIPLY, 75, 689
- SQUARE-MATRIX-MULTIPLY-RECURSIVE, 77
- square of a directed graph, 593 ex.
- square root, modulo a prime, 982 pr.
- squaring, repeated
 - for all-pairs shortest paths, 689–691
 - for raising a number to a power, 956
- stability
 - numerical, 813, 815, 842
 - of sorting algorithms, 196, 200 ex.
- stack, 232–233
 - in Graham's scan, 1030
 - implemented by queues, 236 ex.
 - linked-list implementation of, 240 ex.
 - operations analyzed by accounting method, 457–458
 - operations analyzed by aggregate analysis, 452–454
 - operations analyzed by potential method, 460–461
 - for procedure execution, 188 pr.
 - on secondary storage, 502 pr.
- STACK-EMPTY, 233
- standard deviation, 1200
- standard encoding ($\langle \rangle$), 1057
- standard form, 846, 850–854
- star-shaped polygon, 1038 ex.
- start state, 995
- start time, 415
- state of a finite automaton, 995
- static graph, 562 n.
- static set of keys, 277
- static threading, 773
- stencil, 809 pr.
- stencil calculation, 809 pr.
- Stirling's approximation, 57
- storage management, 151, 243–244, 245 ex., 261 ex.
- store instruction, 23
- straddle, 1017
- strand, 777
 - final, 779
 - independent, 789
 - initial, 779
 - logically in parallel, 778

- logically in series, 778
- Strassen's algorithm, 79–83, 111–112
 - multithreaded, 795–796
- streaks, 135–139
- strictly decreasing, 53
- strictly increasing, 53
- string, 985, 1184
- string matching, 985–1013
 - based on repetition factors, 1012 pr.
 - by finite automata, 995–1002
 - with gap characters, 989 ex., 1002 ex.
 - Knuth-Morris-Pratt algorithm for, 1002–1013
 - naive algorithm for, 988–990
 - Rabin-Karp algorithm for, 990–995, 1013
- string-matching automaton, 996–1002, 1002 ex.
- strongly connected component, 1170
 - decomposition into, 615–621, 623
- STRONGLY-CONNECTED-COMPONENTS, 617
- strongly connected graph, 1170
- subgraph, 1171
 - predecessor, *see* predecessor subgraph
- subgraph-isomorphism problem, 1100 ex.
- subgroup, 943–946
- subpath, 1170
- subproblem graph, 367–368
- subroutine
 - calling, 21, 23, 25 n.
 - executing, 25 n.
- subsequence, 391
- subset ($\subseteq$), 1159, 1161
 - hereditary family of, 437
 - independent family of, 437
- SUBSET-SUM, 1097
- subset-sum problem
 - approximation algorithm for, 1128–1134, 1139
 - NP-completeness of, 1097–1100
 - with unary target, 1101 ex.
- substitution method, 83–88
 - and recursion trees, 91–92
- substring, 1184
- subtract instruction, 23
- subtraction of matrices, 1221
- subtree, 1176
 - maintaining sizes of, in order-statistic trees, 343–344
- success, in a Bernoulli trial, 1201
- successor
 - in binary search trees, 291–292
 - in a bit vector with a superimposed binary tree, 533
 - in a bit vector with a superimposed tree of constant height, 535
 - finding i th, of a node in an order-statistic tree, 344 ex.
 - in linked lists, 236
 - in order-statistic trees, 347 ex.
 - in proto van Emde Boas structures, 543–544
 - in red-black trees, 311
 - in Van Emde Boas trees, 550–551
- SUCCESSOR, 230
- such that ($:$), 1159
- suffix ($\square$), 986
- suffix function, 996
- suffix-function inequality, 999
- suffix-function recursion lemma, 1000
- sum ($\sum$), 1145
 - Cartesian, 906 ex.
 - infinite, 1145
 - of matrices, 1220
 - of polynomials, 898
 - rule of, 1183
 - telescoping, 1148
- SUM-ARRAYS, 805 pr.
- SUM-ARRAYS', 805 pr.
- summary
 - in a bit vector with a superimposed tree of constant height, 534
 - in proto van Emde Boas structures, 540
 - in van Emde Boas trees, 546
- summation, 1145–1157
 - in asymptotic notation, 49–50, 1146
 - bounding, 1149–1156
 - formulas and properties of, 1145–1149
 - linearity of, 1146
- summation lemma, 908
- supercomputer, 772
- superpolynomial time, 1048
- supersink, 712
- supersource, 712
- surjection, 1167
- SVD, 842
- sweeping, 1021–1029, 1045 pr.
 - rotational, 1030–1038

- sweep line, 1022
- sweep-line status, 1023–1024
- symbol table, 253, 262, 265
- symmetric difference, 763 pr.
- symmetric matrix, 1220, 1222 ex., 1226 ex.
- symmetric positive-definite matrix, 832–835, 842
- symmetric relation, 1163
- symmetry of Θ -notation, 52
- sync**, in pseudocode, 776–777
- system of difference constraints, 664–670
- system of linear equations, 806 pr., 813–827, 840 pr.
- TABLE-DELETE, 468
- TABLE-INSERT, 464
- tail
 - of a binomial distribution, 1208–1215
 - of a linked list, 236
 - of a queue, 234
- tail recursion, 188 pr., 419
- TAIL-RECURSIVE-QUICKSORT, 188 pr.
- target, 1097
- Tarjan's off-line least-common-ancestors algorithm, 584 pr.
- task, 443
- Task Parallel Library, 774
- task scheduling, 443–446, 448 pr., 450
- tautology, 1066 ex., 1086 ex.
- Taylor series, 306 pr.
- telescoping series, 1148
- telescoping sum, 1148
- testing
 - of primality, 965–975, 983
 - of pseudoprimalty, 966–968
- text, in string matching, 985
- then** clause, 20 n.
- Theta-notation, 44–47, 64
- thread, 773
- Threading Building Blocks, 774
- 3-CNF, 1082
- 3-CNF-SAT, 1082
- 3-CNF satisfiability, 1082–1085, 1105
 - approximation algorithm for, 1123–1124, 1139
 - and 2-CNF satisfiability, 1049
- 3-COLOR, 1103 pr.
- 3-conjunctive normal form, 1082
- tight constraint, 865
- time, *see* running time
- time domain, 898
- time-memory trade-off, 365
- timestamp, 603, 611 ex.
- Toeplitz matrix, 921 pr.
- to**, in pseudocode, 20
- TOP, 1031
- top-down method, for dynamic programming, 365
- top of a stack, 232
- topological sort, 8, 612–615, 623
 - in computing single-source shortest paths in a dag, 655
- TOPOLOGICAL-SORT, 613
- total order, 1165
- total path length, 304 pr.
- total preorder, 1165
- total relation, 1165
- tour
 - bitonic, 405 pr.
 - Euler, 623 pr., 1048
 - of a graph, 1096
- track, 486
- tractability, 1048
- trailing pointer, 295
- transition function, 995, 1001–1002, 1012 ex.
- transitive closure, 697–699
 - and boolean matrix multiplication, 832 ex.
 - of dynamic graphs, 705 pr., 707
- TRANSITIVE-CLOSURE, 698
- transitive relation, 1163
- transitivity of asymptotic notation, 51
- TRANSPLANT, 296, 323
- transpose
 - conjugate, 832 ex.
 - of a directed graph, 592 ex.
 - of a matrix, 1217
 - of a matrix, multithreaded, 792 ex.
- transpose symmetry of asymptotic notation, 52
- traveling-salesman problem
 - approximation algorithm for, 1111–1117, 1139
 - bitonic euclidean, 405 pr.
 - bottleneck, 1117 ex.
 - NP-completeness of, 1096–1097
 - with the triangle inequality, 1112–1115
 - without the triangle inequality, 1115–1116

- traversal of a tree, 287, 293 ex., 342, 1114
- treap, 333 pr., 338
- TREAP-INSERT, 333 pr.
- tree, 1173–1180
 - AA-trees, 338
 - AVL, 333 pr., 337
 - binary, *see* binary tree
 - binomial, 527 pr.
 - bisection of, 1181 pr.
 - breadth-first, 594, 600
 - B-trees, 484–504
 - decision, 192–193
 - depth-first, 603
 - diameter of, 602 ex.
 - dynamic, 482
 - free, 1172–1176
 - full walk of, 1114
 - fusion, 212, 483
 - heap, 151–169
 - height-balanced, 333 pr.
 - height of, 1177
 - interval, 348–354
 - k -neighbor, 338
 - minimum spanning, *see* minimum spanning tree
 - optimal binary search, 397–404, 413
 - order-statistic, 339–345
 - parse, 1082
 - recursion, 37, 88–93
 - red-black, *see* red-black tree
 - rooted, 246–249, 1176
 - scapegoat, 338
 - search, *see* search tree
 - shortest-paths, 647–648, 673–676
 - spanning, *see* minimum spanning tree, spanning tree
 - splay, 338, 482
 - treap, 333 pr., 338
 - 2-3, 337, 504
 - 2-3-4, 489, 503 pr.
 - van Emde Boas, 531–560
 - walk of, 287, 293 ex., 342, 1114
 - weight-balanced trees, 338
- TREE-DELETE, 298, 299 ex., 323–324
- tree edge, 601, 603, 609
- TREE-INSERT, 294, 315
- TREE-MAXIMUM, 291
- TREE-MINIMUM, 291
- TREE-PREDECESSOR, 292
- TREE-SEARCH, 290
- TREE-SUCCESSOR, 292
- tree walk, 287, 293 ex., 342, 1114
- trial, Bernoulli, 1201
- trial division, 966
- triangle inequality, 1112
 - for shortest paths, 650, 671
- triangular matrix, 1219, 1222 ex., 1225 ex.
- trichotomy, interval, 348
- trichotomy property of real numbers, 52
- tridiagonal linear systems, 840 pr.
- tridiagonal matrix, 1219
- trie (radix tree), 304 pr.
 - y-fast, 558 pr.
- TRIM, 1130
- trimming a list, 1130
- trivial divisor, 928
- truth assignment, 1072, 1079
- truth table, 1070
- TSP, 1096
- tuple, 1162
- twiddle factor, 912
- 2-CNF-SAT, 1086 ex.
- 2-CNF satisfiability, 1086 ex.
 - and 3-CNF satisfiability, 1049
- two-pass method, 571
- 2-3-4 heap, 529 pr.
- 2-3-4 tree, 489
 - joining, 503 pr.
 - splitting, 503 pr.
- 2-3 tree, 337, 504
- unary, 1056
- unbounded linear program, 851
- unconditional branch instruction, 23
- uncountable set, 1161
- underdetermined system of linear equations, 814
- underflow
 - of a queue, 234
 - of a stack, 233
- undirected graph, 1168
 - articulation point of, 621 pr.
 - biconnected component of, 621 pr.
 - bridge of, 621 pr.
 - clique in, 1086
 - coloring of, 1103 pr., 1180 pr.

- computing a minimum spanning tree in, 624–642
 - converting to, from a multigraph, 593 ex.
 - d -regular, 736 ex.
 - grid, 760 pr.
 - hamiltonian, 1061
 - independent set of, 1101 pr.
 - matching of, 732
 - nonhamiltonian, 1061
 - vertex cover of, 1089, 1108
 - see also* graph
- undirected version of a directed graph, 1172
- uniform hashing, 271
- uniform probability distribution, 1191–1192
- uniform random permutation, 116, 125
- union
 - of dynamic sets, *see* uniting
 - of languages, 1058
 - of sets ($\cup$), 1159
- UNION, 505, 562
 - disjoint-set-forest implementation of, 571
 - linked-list implementation of, 565–567, 568 ex.
- union by rank, 569
- unique factorization of integers, 931
- unit (1), 928
- uniting
 - of Fibonacci heaps, 511–512
 - of heaps, 506
 - of linked lists, 241 ex.
 - of 2-3-4 heaps, 529 pr.
- unit lower-triangular matrix, 1219
- unit-time task, 443
- unit upper-triangular matrix, 1219
- unit vector, 1218
- universal collection of hash functions, 265
- universal hashing, 265–268
- universal sink, 593 ex.
- universe, 1160
 - of keys in van Emde Boas trees, 532
- universe size, 532
- unmatched vertex, 732
- unsorted linked list, 236
 - see also* linked list
- until**, in pseudocode, 20
- unweighted longest simple paths, 382
- unweighted shortest paths, 381
- upper bound, 47
- upper-bound property, 650, 671–672
- upper median, 213
- upper square root ($\sqrt{}$), 546
- upper-triangular matrix, 1219, 1225 ex.
- valid shift, 985
- value
 - of a flow, 710
 - of a function, 1166
 - objective, 847, 851
- value over replacement player, 411 pr.
- Vandermonde matrix, 902, 1226 pr.
- van Emde Boas tree, 531–560
 - cluster in, 546
 - compared with proto van Emde Boas structures, 547
 - deletion from, 554–556
 - insertion into, 552–554
 - maximum in, 550
 - membership in, 550
 - minimum in, 550
 - predecessor in, 551–552
 - with reduced space, 557 pr.
 - successor in, 550–551
 - summary in, 546
- Var [] (variance), 1199
- variable
 - basic, 855
 - entering, 867
 - leaving, 867
 - nonbasic, 855
 - in pseudocode, 21
 - random, 1196–1201
 - slack, 855
 - see also* indicator random variable
- variable-length code, 429
- variance, 1199
 - of a binomial distribution, 1205
 - of a geometric distribution, 1203
- VEB-EMPTY-TREE-INSERT, 553
- vEB tree, *see* van Emde Boas tree
- VEB-TREE-DELETE, 554
- VEB-TREE-INSERT, 553
- VEB-TREE-MAXIMUM, 550
- VEB-TREE-MEMBER, 550
- VEB-TREE-MINIMUM, 550
- VEB-TREE-PREDECESSOR, 552
- VEB-TREE-SUCCESSOR, 551

- vector, 1218, 1222–1224
 - convolution of, 901
 - cross product of, 1016
 - orthonormal, 842
 - in the plane, 1015
- Venn diagram, 1160
- verification, 1061–1066
 - of spanning trees, 642
- verification algorithm, 1063
- vertex
 - articulation point, 621 pr.
 - attributes of, 592
 - capacity of, 714 ex.
 - in a graph, 1168
 - intermediate, 693
 - isolated, 1169
 - overflowing, 736
 - of a polygon, 1020 ex.
 - relabeled, 740
 - selector, 1093
- vertex cover, 1089, 1108, 1124–1127, 1139
- VERTEX-COVER, 1090
- vertex-cover problem
 - approximation algorithm for, 1108–1111, 1139
 - NP-completeness of, 1089–1091, 1105
- vertex set, 1168
- violation, of an equality constraint, 865
- virtual memory, 24
- Viterbi algorithm, 408 pr.
- VORP, 411 pr.
- walk of a tree, 287, 293 ex., 342, 1114
- weak duality, 880–881, 886 ex., 895 pr.
- weight
 - of a cut, 1127 ex.
 - of an edge, 591
 - mean, 680 pr.
 - of a path, 643
- weight-balanced tree, 338, 473 pr.
- weighted bipartite matching, 530
- weighted matroid, 439–442
- weighted median, 225 pr.
- weighted set-covering problem, 1135 pr.
- weighted-union heuristic, 566
- weighted vertex cover, 1124–1127, 1139
- weight function
 - for a graph, 591
 - in a weighted matroid, 439
- while**, in pseudocode, 20
- white-path theorem, 608
- white vertex, 594, 603
- widget, 1092
- wire, 1071
- WITNESS, 969
- witness, to the compositeness of a number, 968
- work law, 780
- work, of a multithreaded computation, 779
- work-stealing scheduling algorithm, 812
- worst-case running time, 27, 49
- Yen's improvement to the Bellman-Ford algorithm, 678 pr.
- y-fast trie, 558 pr.
- Young tableau, 167 pr.
- $\mathbb{Z}$ (set of integers), 1158
- $\mathbb{Z}_n$ (equivalence classes modulo n), 928
- $\mathbb{Z}_n^*$ (elements of multiplicative group modulo n), 941
- $\mathbb{Z}_n^+$ (nonzero elements of $\mathbb{Z}_n$), 967
- zero matrix, 1218
- zero of a polynomial modulo a prime, 950 ex.
- 0-1 integer programming, 1100 ex., 1125
- 0-1 knapsack problem, 425, 427 ex., 1137 pr., 1139
- 0-1 sorting lemma, 208 pr.
- zonk, 1195 ex.